Sri Lanka Institute of Information Technology

BSc (Hons) in Information Technology

Specializing in Data Science

IT3021 – Data Warehousing and Business Intelligence

Year 3

Assignment 1

Semester 1, 2026

EKANAYAKA E.J.M.C.U - IT23809024

Step 1: Data set selection 2

Step 2: Preparation of data sources 4

Step 3: Solution architecture 12

Step 4: Data warehouse design & development 15

Step 5: : ETL development 16

Step 6: : ETL development- Accumulating fact tables 38

Step 1: Data set selection

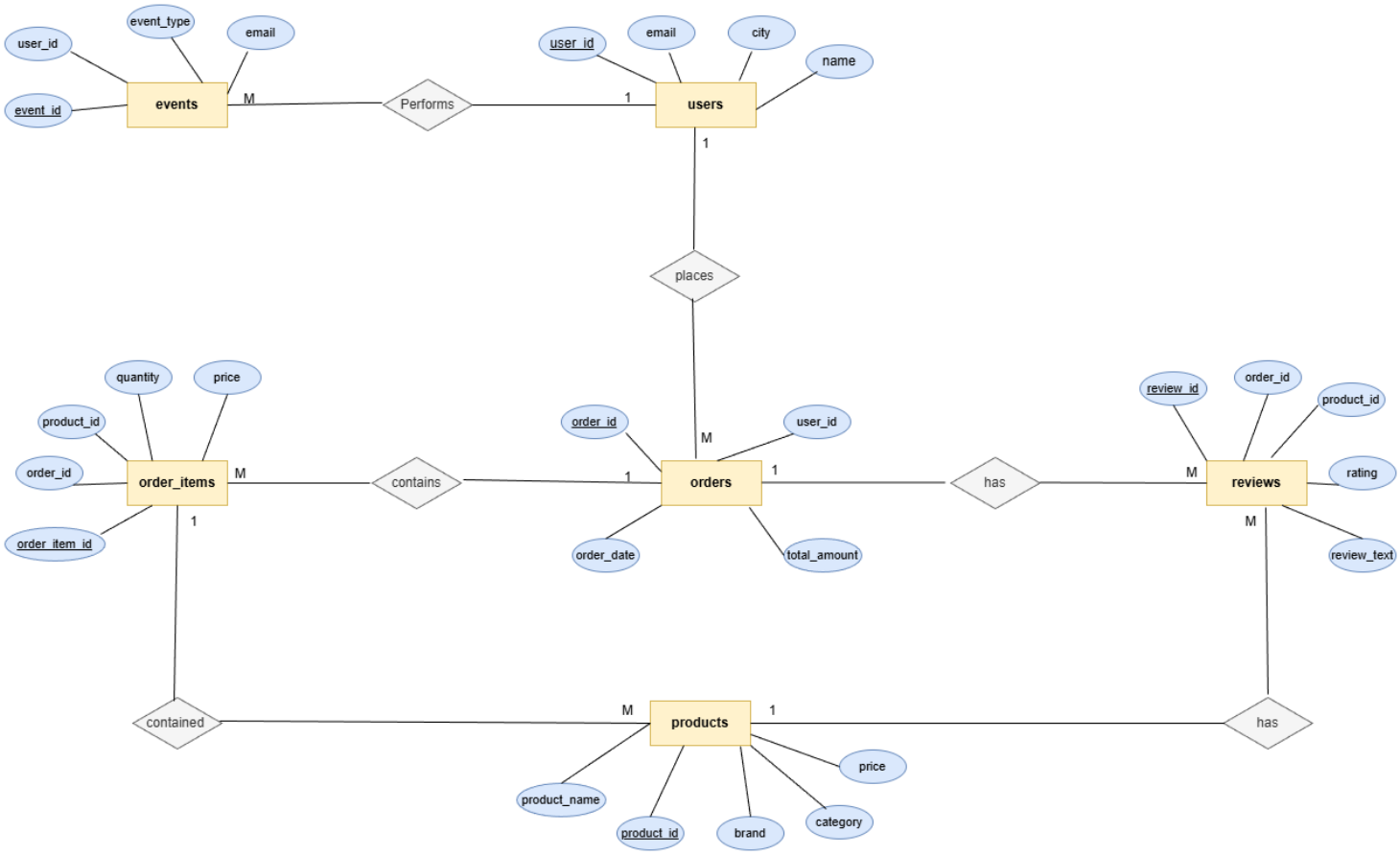
For this assignment, I have selected a comprehensive e-commerce retail dataset representing the daily transactional operations and web interactions of a modern online retail platform. This is a purely OLTP (Online Transactional Processing) dataset consisting of highly normalized tables that capture live, day-to-day business events. The dataset presents a novel and modern business scenario by combining standard sales data with web analytics, making it distinctly different from standard educational datasets like AdventureWorks.

Original data set link: https://www.kaggle.com/datasets/abhayayare/e-commerce-dataset

Table Name	Description	Attributes
Users	Contains demographic and geographical information about registered customers.	user_id, name, email, gender, city, signup_date
Products	Acts as the primary product catalog, detailing retail prices, categories, and brands.	product_id, product_name, category, brand, price, rating
Orders	Records the high-level transactional data and status of customer purchases.	order_id, user_id, order_date, order_status, total_amount

Table Name	Description	Attributes
Events	Captures user behavioral data on the website, tracking specific web interactions.	event_id, user_id, product_id, event_type, event_timestamp
Reviews	Contains customer feedback, textual reviews, and ratings for the products they purchased.	review_id, order_id, product_id, user_id, rating, review_text, review_date
Order_Items	The core transactional table containing line-item details and actual sale prices for each order.	id, order_id, user_id, product_id, sale_price

ER-Diagram



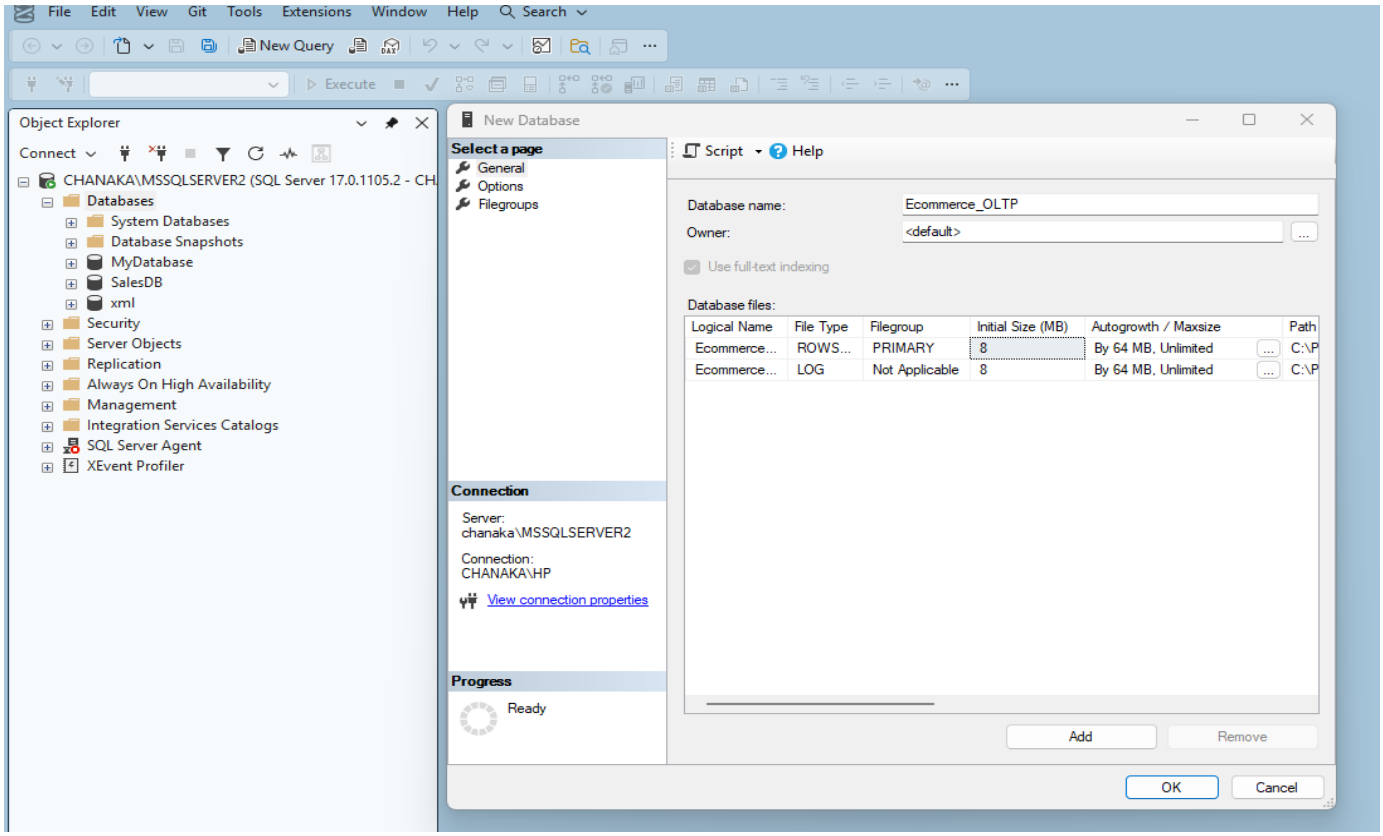
Step 2: Preparation of data sources

The preparation of data sources involves setting up the primary operational database (OLTP) that will feed data into the Data Warehouse. For this project, a new relational database named Ecommerce_OLTP was created in Microsoft SQL Server.

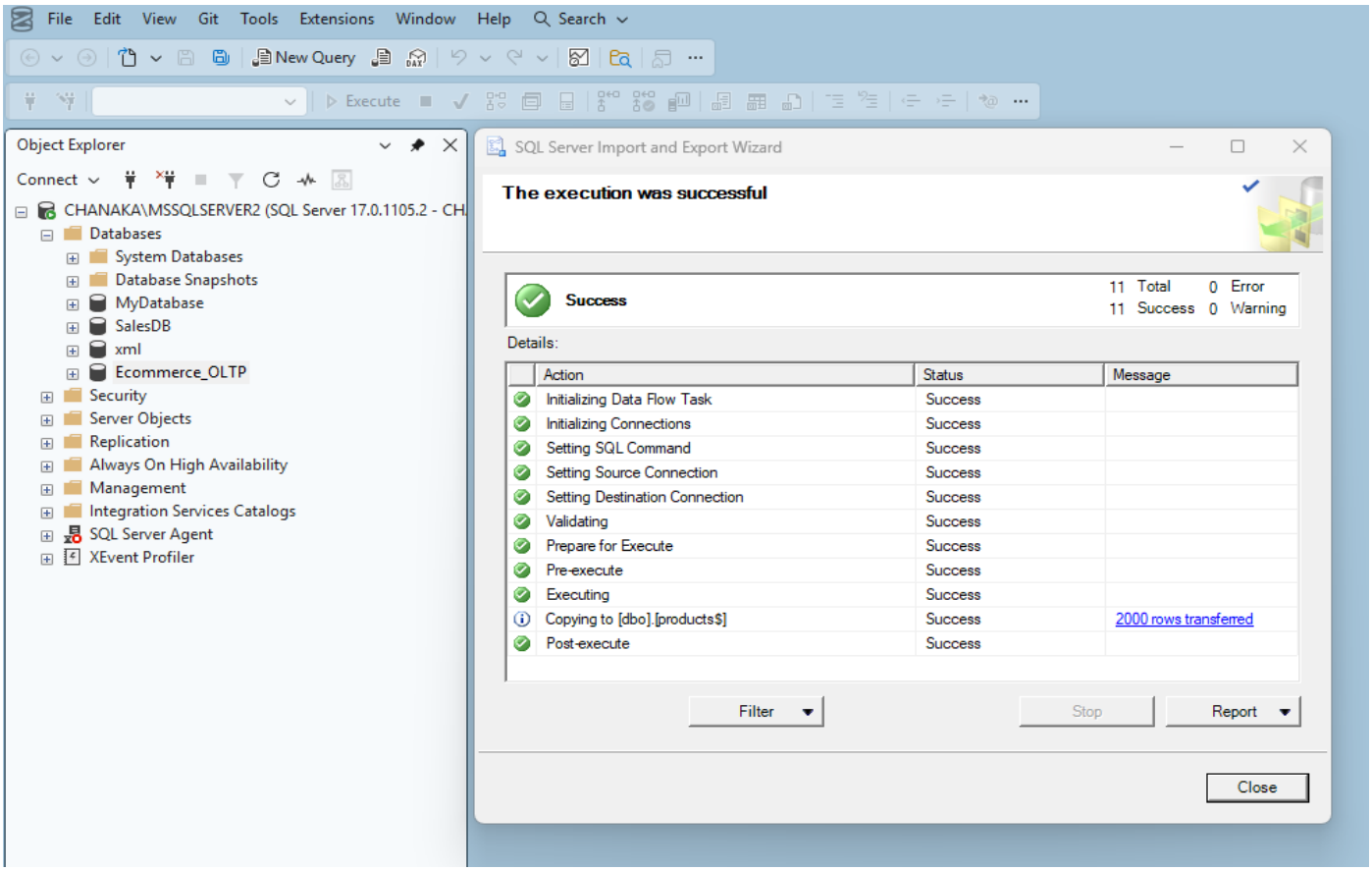
The final data sources prepared include four Excel files and two CSV files, containing the following information:

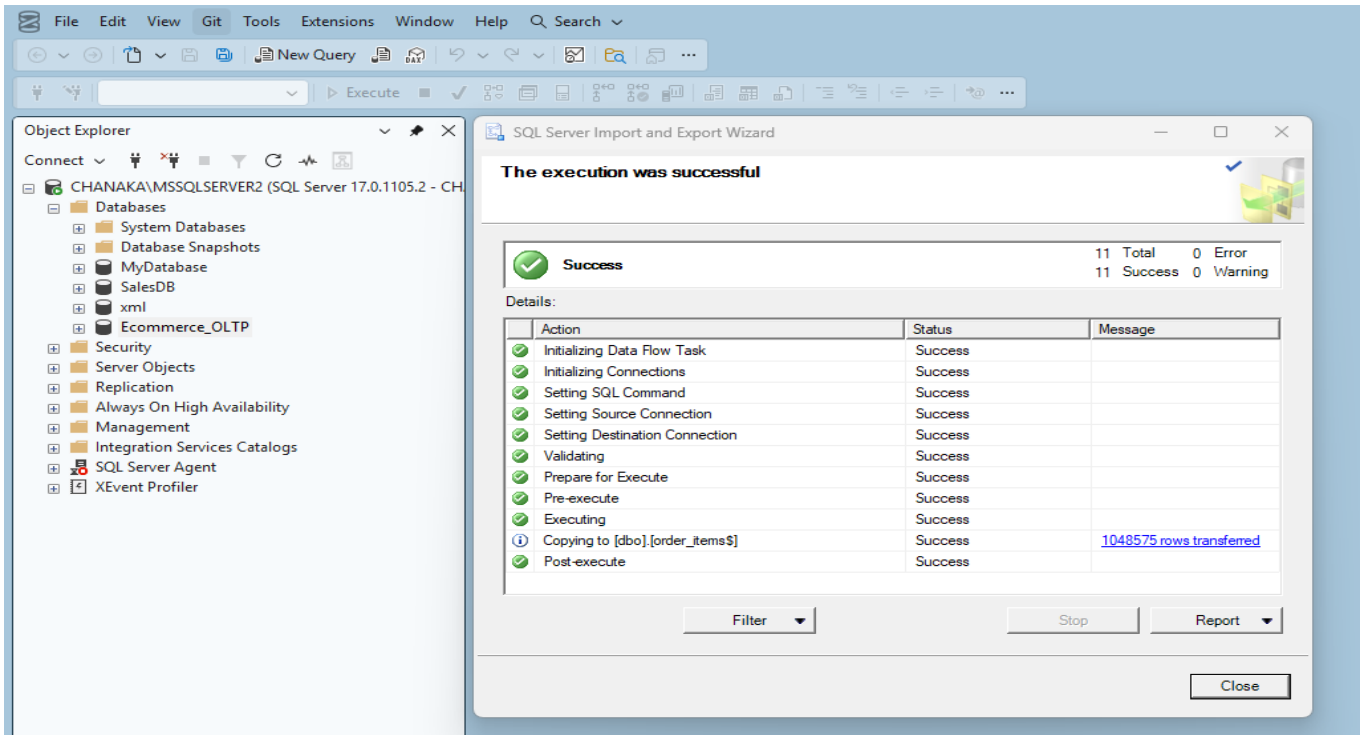
Source Name	Description	Source Type
users.csv	Contains customer demographic information, account details, and registration dates.	Flat File (CSV)
products.xlsx	Contains the complete product catalog including categories, brands, prices, and ratings.	Excel File (.xlsx)
orders.csv	Contains transactional records of customer orders, including order status, amounts, and timestamps.	Flat File (CSV)
order_items.xlsx	Contains the individual products purchased within each order, including quantities and unit prices.	Excel File (.xlsx)
reviews.csv	Contains customer feedback, textual reviews, and individual product ratings given by users.	Flat File (CSV)
events.csv	Contains user interaction logs on the platform, such as product views and add-to-cart activities.	Flat File (CSV)

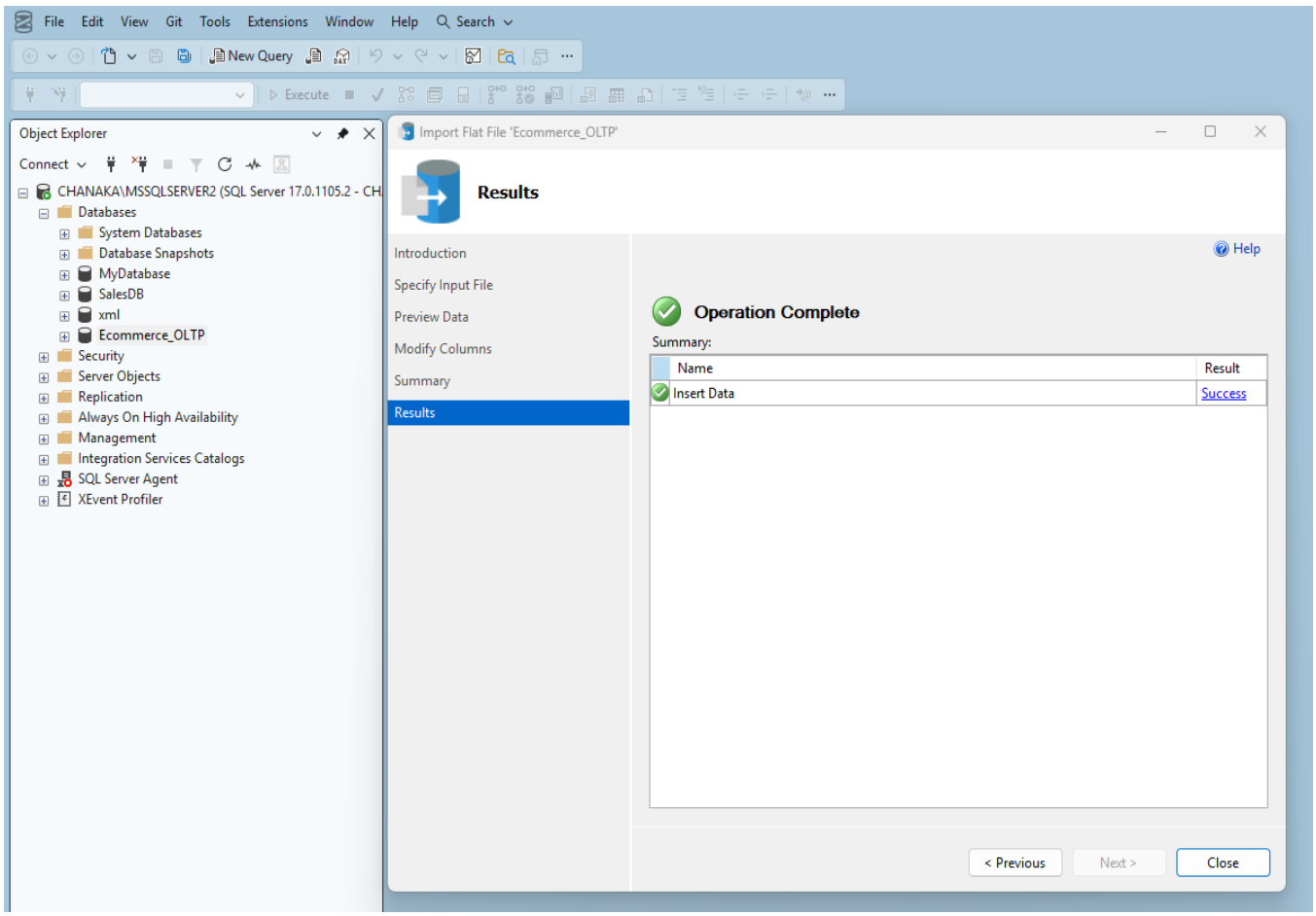
2.1Database creation(Ecommerce_OLTP)



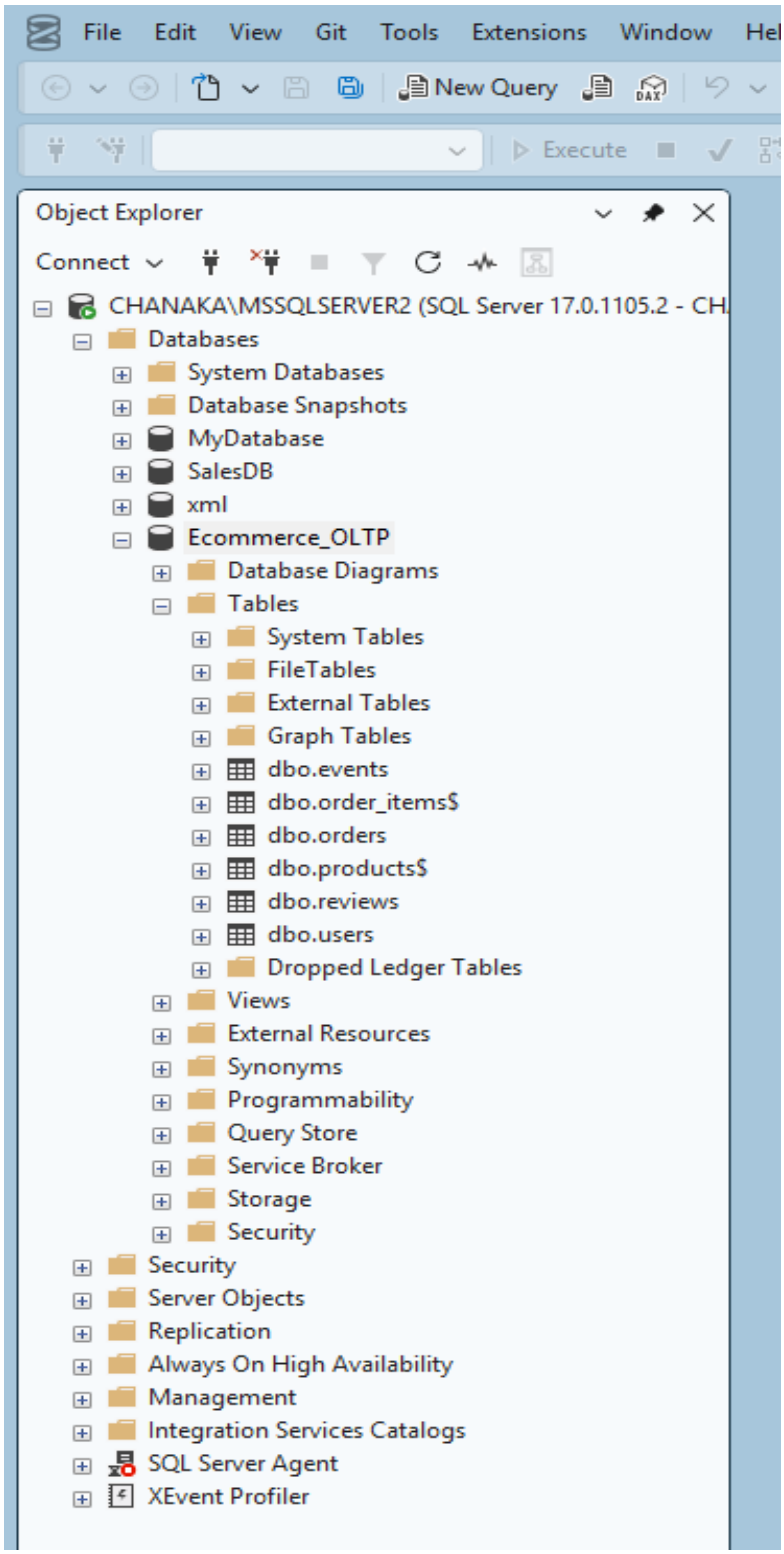
2.2 Import wizard setup for each table







2.3 Tables listed in Object Explorer(Ecommerce_OLTP)



2.4 Query result for sample data

File Edit View Query Git Project Tools Extensions Window Help Search

New Query

Ecommerce_OLTP Execute

Object Explorer

Connect

CHANAKA\MSSQLSERVER2 (SQL Server 17.0.1105.2 - CH

- Databases
  - System Databases
  - Database Snapshots
  - MyDatabase
  - SalesDB
  - xml
  - Ecommerce_OLTP
    - Database Diagrams
    - Tables
      - System Tables
      - FileTables
      - External Tables
      - Graph Tables
      - dbo.events
      - dbo.order_items$
      - dbo.orders
      - dbo.products$
      - dbo.reviews
      - dbo.users
      - Dropped Ledger Tables
    - Views
    - External Resources
    - Synonyms
    - Programmability
    - Query Store
    - Service Broker
    - Storage
    - Security
- Security
- Server Objects
- Replication
- Always On High Availability
- Management
- Integration Services Catalogs
- SQL Server Agent
- XEvent Profiler

SQLQuery1.s...KA\HP (54))*

```sql
USE Ecommerce_OLTP;
GO

-- Checking the imported data in each table
SELECT TOP 5 * FROM [users];
SELECT TOP 5 * FROM [products$];
SELECT TOP 5 * FROM [orders];
SELECT TOP 5 * FROM [order_items$];
SELECT TOP 5 * FROM [reviews];
SELECT TOP 5 * FROM [events];
```

100 % No issues found

Results Messages

	user_id	name	email	gender	city	signup_date
1	U000001	Angel Hill	donaldgarcia@example.net	Other	New Roberttown	2025-03-13
2	U000002	Jesse Guzman	jennifermiles@example.com	Male	South Bridget	2024-03-05
3	U000003	Adam Shaffer	jpeterson@example.org	Male	Curtisfurt	2025-07-07
4	U000004	Melanie Munoz	blairamanda@example.com	Other	New Kellystad	2024-03-07
5	U000005	Janet Williams	kendragalloway@example.org	Female	South Joshuastad	2025-01-29

	product_id	product_name	category	brand	price	rating
1	P000001	Astra Be	Clothing	Astra	157.89	4.08
2	P000002	NeoTech Someone	Groceries	NeoTech	21.46	3.87
3	P000003	Acme Discuss	Sports	Acme	265.37	3.46
4	P000004	Nimbus South	Electronics	Nimbus	541.41	4.14
5	P000005	Astra Capital	Home & Kitchen	Astra	198	3.97

	order_id	user_id	order_date	order_status	total_amount
1	O00000001	U009310	2025-09-09T14:52:37.292731	processing	689.659973144531
2	O00000002	U003247	2025-04-15T01:18:27.193404	completed	1666.84997558594
3	O00000003	U007252	2025-04-27T15:37:48.008624	processing	665.059997558594
4	O00000004	U008986	2025-10-04T20:35:22.204857	cancelled	689.5
5	O00000005	U008537	2024-11-13T08:15:18.498252	cancelled	860.5

	order_item_id	order_id	product_id	user_id	quantity	item_price	item_total
1	I00019741	O00009126	P000180	U008870	1	400.2	400.2
2	I00019742	O00009126	P001161	U008870	1	12.66	12.66
3	I00019743	O00009127	P001452	U002970	1	505.82	505.82
4	I00019744	O00009127	P000161	U002970	1	701.36	701.36
5	I00019745	O00009127	P001470	U002970	1	7.41	7.41

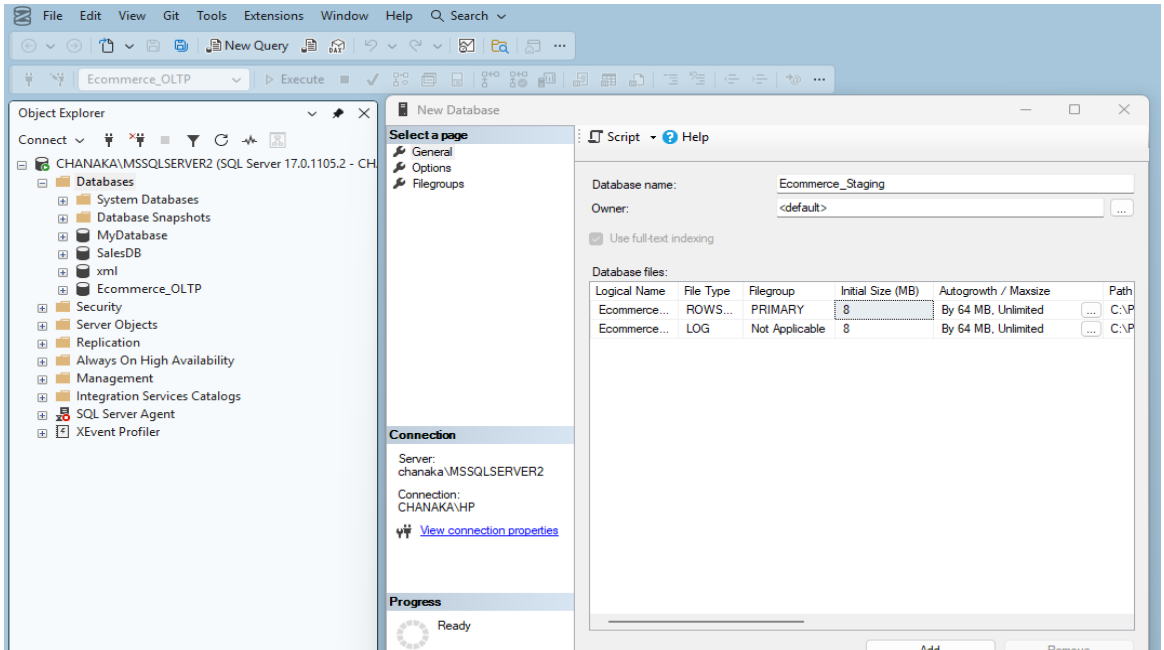
	review_id	order_id	product_id	user_id	rating	review_text	review_date
1	528.00	O00000237	1326.00	U001094	2	Color was different from images.	2025-10-14T12:03:56.749446
2	5792.00	O00002627	329.00	U001858	4	Highly recommend this brand.	2024-10-09T08:04:50.171793
3	36604.00	O00016798	1160.00	U008109	4	Highly recommend this brand.	2024-06-03T05:11:16.787214
4	40163.00	O00018414	1427.00	U006835	5	Highly recommend this brand.	2024-02-12T06:41:50.215810
5	31127.00	O00014300	1639.00	U007148	3	Item arrived damaged.	2025-01-20T05:32:09.398860

	event_id	user_id	product_id	event_type	event_timestamp

Query executed successfully.

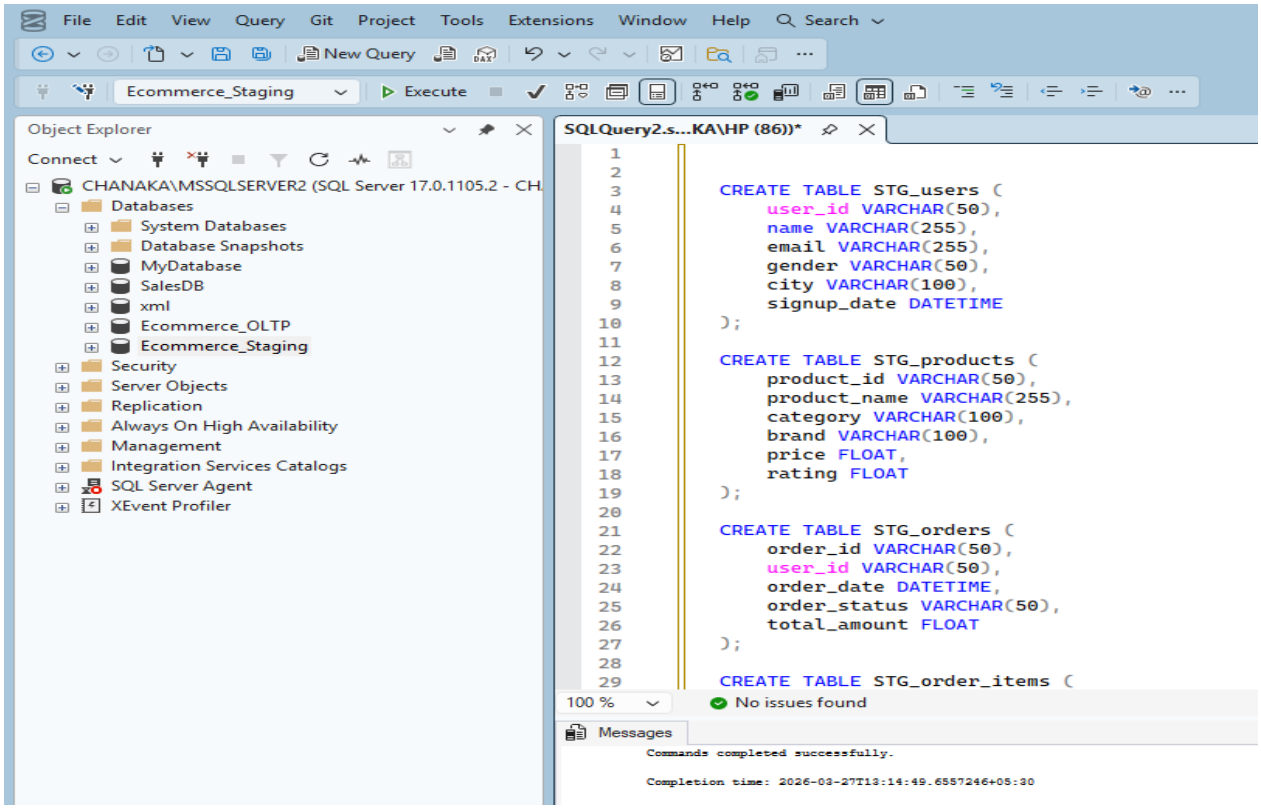
2.5 Query result for sample data

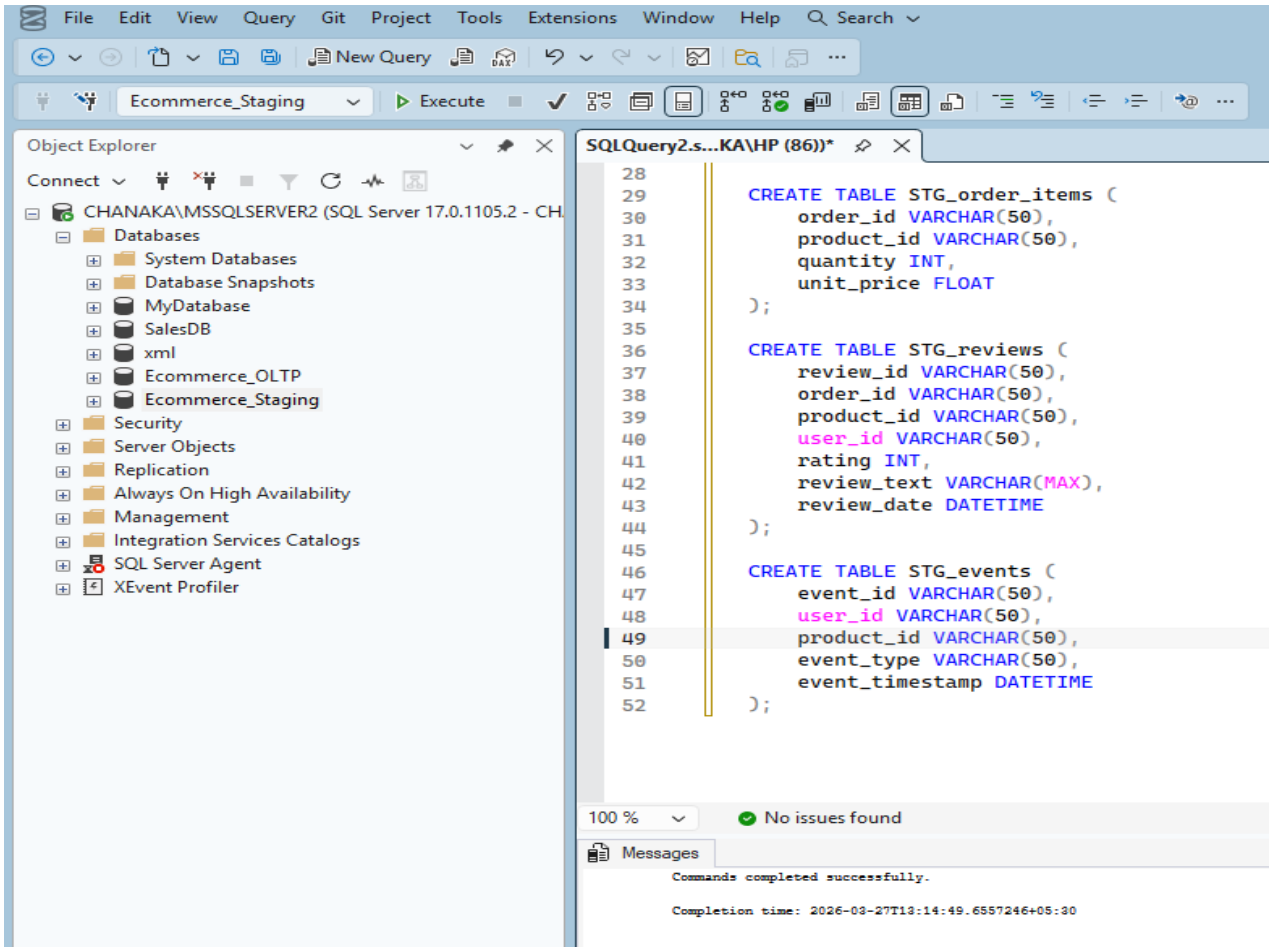
A staging database is used to temporarily store and prepare data before loading it into the data warehouse. It acts as a buffer area where data from multiple sources is collected, cleaned, transformed, and validated.



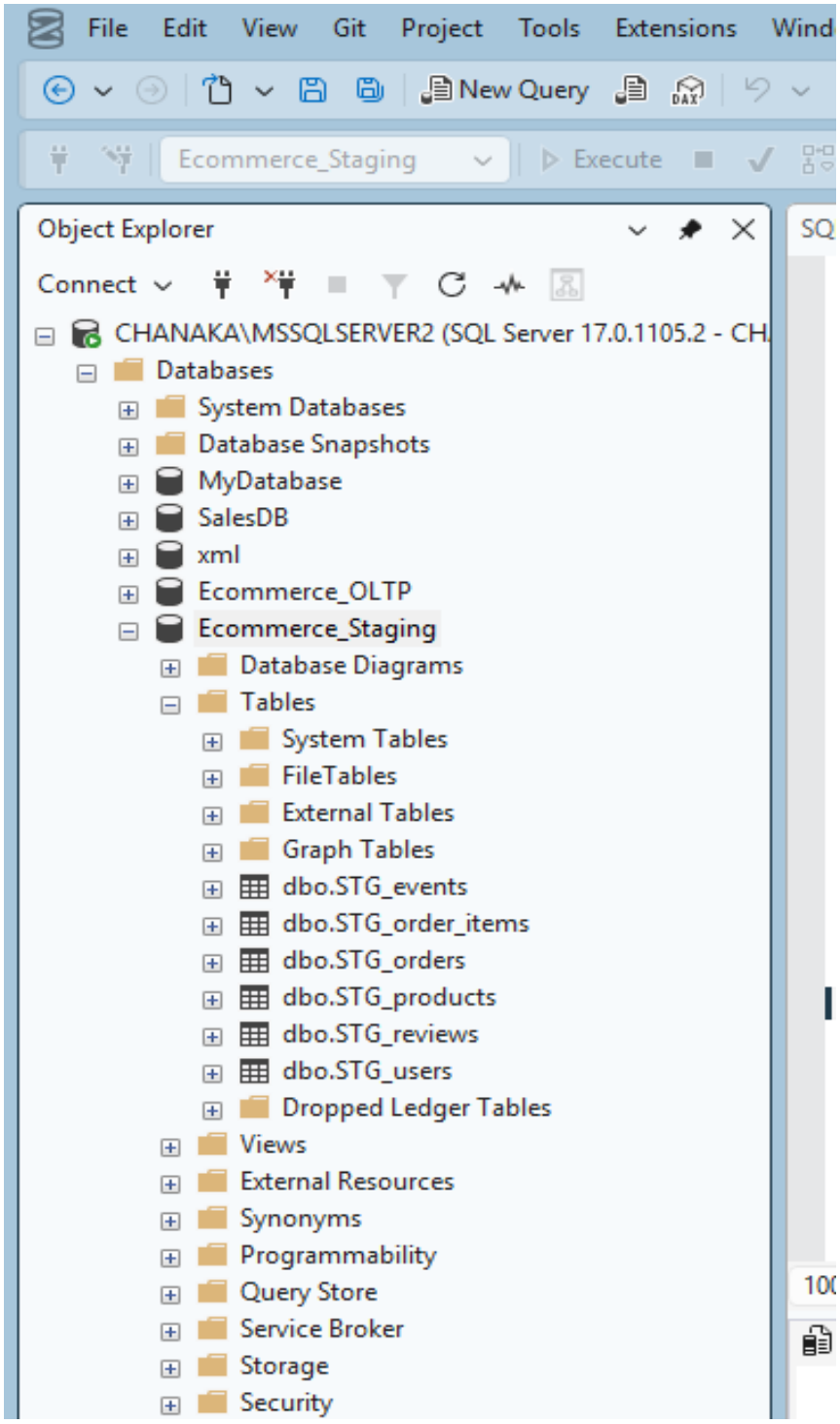
2.6 Create Staging Tables(BankRetail_Staging)

Now we will create tables inside Ecommerce_Stagingthat match the structure of the 6 source tables from Ecommerce_OLTP.



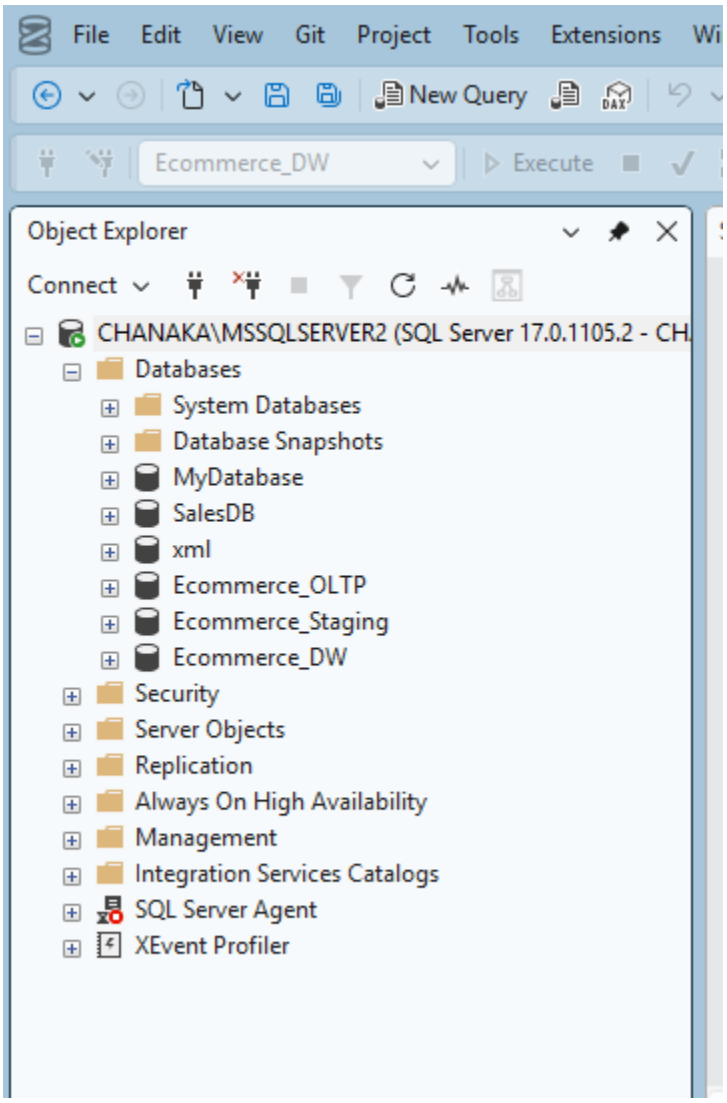


2.7 Tables of staging database



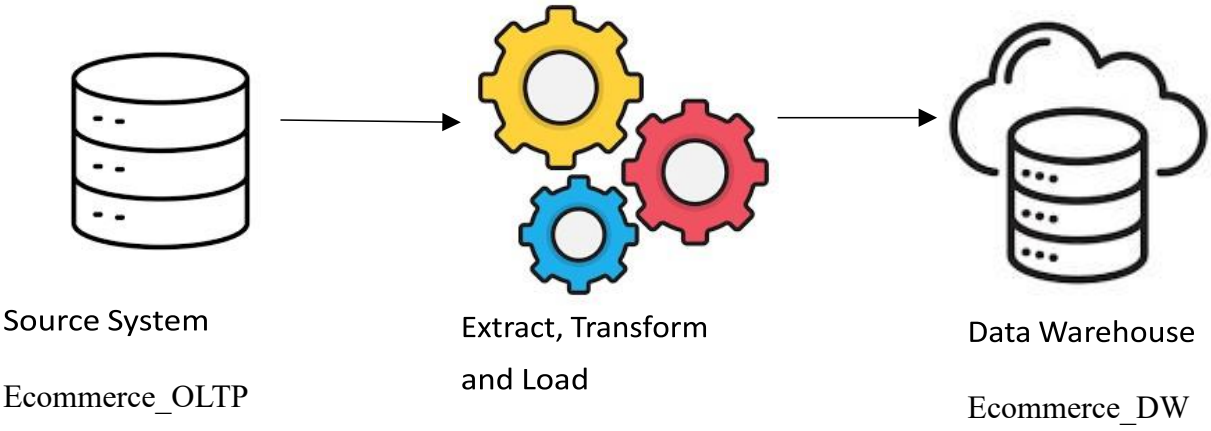
2.8 Final Output

Then the files were loaded to the “Ecommerce_OLTP” database.



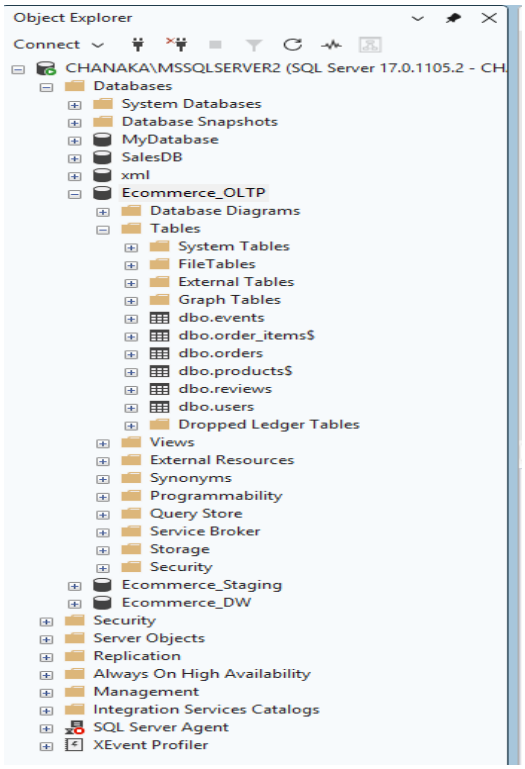
Step 3: Solution architecture

The below figure shows the high-level diagram for the process.(ETL Process)

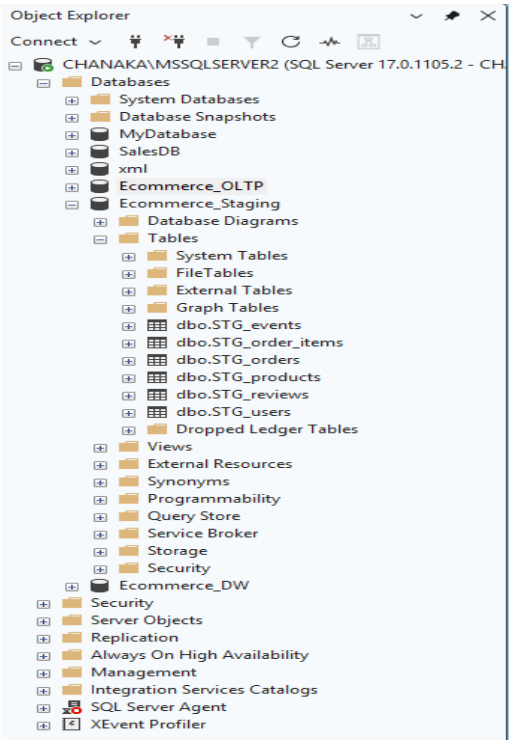


The solution architecture for this E-commerce data warehouse project follows a traditional ETL (Extract, Transform, Load) approach using Microsoft SQL Server and SSIS. Initially, raw data from flat files (CSV and Excel) is imported into an Operational Data Store named Ecommerce_OLTP. From there, SQL Server Integration Services (SSIS) is utilized to extract and load the data into a temporary Ecommerce_Staging database for cleansing and validation. Finally, the processed data is transformed and loaded into the central Ecommerce_DW database. This data warehouse is designed using a Star Schema dimensional model, providing a robust foundation for analytical processing and reporting via business intelligence tools like Power BI.

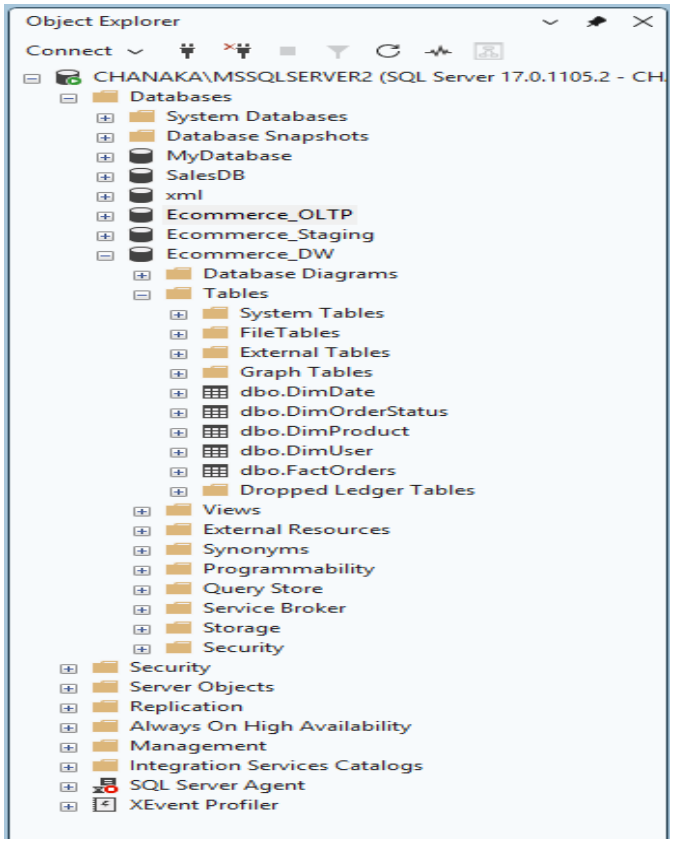
3.1 Database – Ecommerce_OLTP



3.2 Staging Database – Ecommerce_Staging



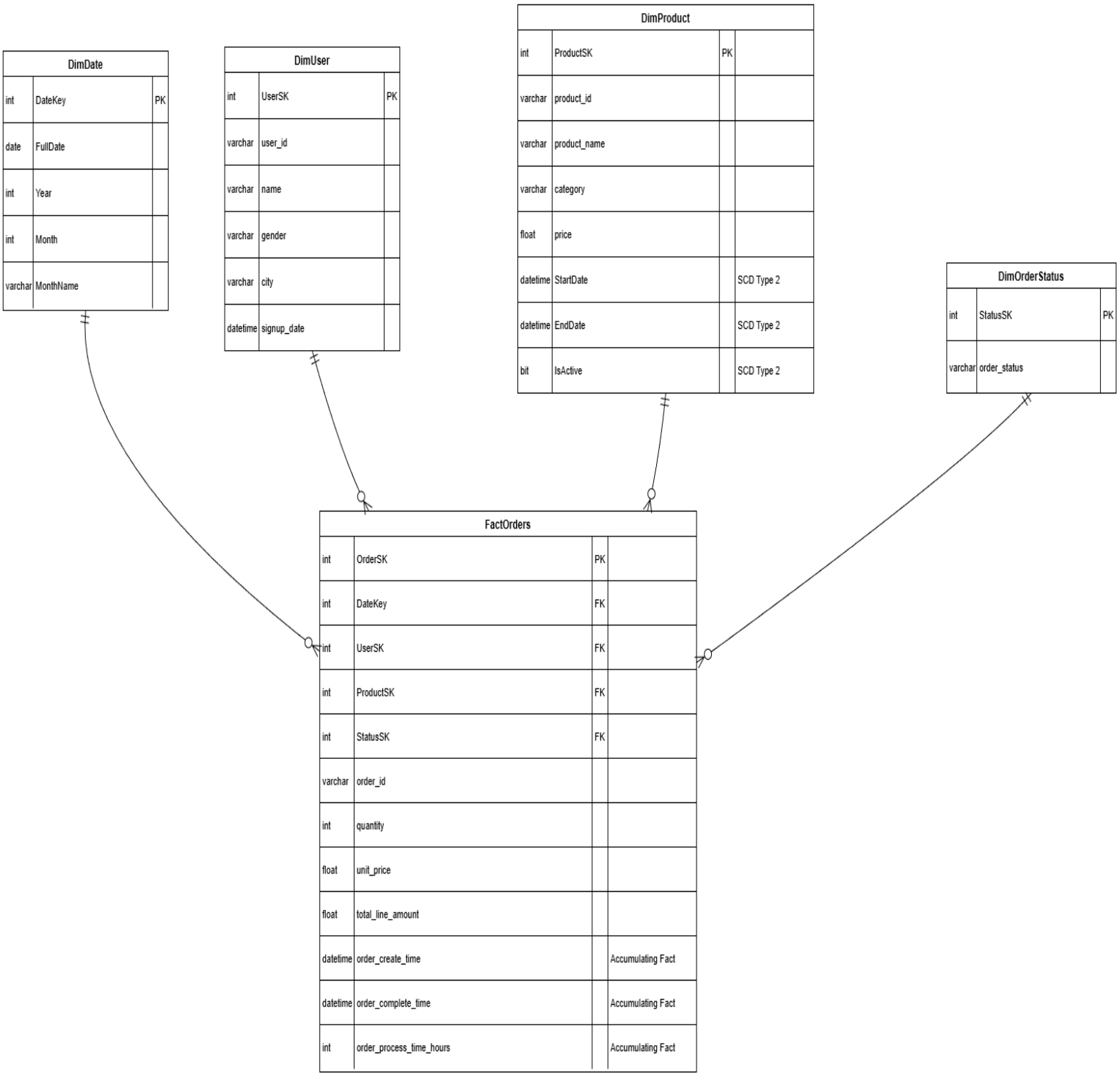
3.3 Data Warehouse- Ecommerce_DW(Data was transformed and loaded to datawarehouse)



3.4 Summarized Descriptions of Each Component

- **CSV and Excel Files:** The original raw files containing user, product, order, review, and event information.
- **Ecommerce_OLTP (Source Database):** A database where the extracted raw data from CSV and Excel files is loaded initially. It acts as the source system.
- **Ecommerce_Staging (Staging Database):** Temporary storage used for cleansing, validating, and transforming data before moving it into the data warehouse. Data quality issues are fixed here.
- **Ecommerce_DW (Data Warehouse):** The main data warehouse designed using a star schema, containing dimension and fact tables optimized for analytical querying.
- **SSAS Cube (OLAP Model):** An analytical model built using SQL Server Analysis Services (SSAS), creating cubes with hierarchies, measures, and aggregations for fast querying and slicing of data.
- **Power BI Reports/Dashboards:** Visualization layer where users interact with the data through dashboards, charts, graphs, and drill-through reports created using Power BI, connected to the OLAP cubes.

Step 4: Data warehouse design & development



The **key icon** should appear on the **dimension table** side (surrogate key), and the **infinity (∞)** symbol should appear on the **fact table** side (foreign key).

- A single central fact table, **FactOrders** (Stores comprehensive order lifecycle details, product quantities, and transactional amounts), was created during the warehouse design.
- Therefore, a traditional **Star Schema** dimensional model was used, which is highly optimized for analytical querying.
- The central **FactOrders** table is directly connected to four surrounding dimension tables: **DimUser**, **DimProduct**, **DimOrderStatus**, and **DimDate**.
- The **DimProduct** table is specifically implemented as a Slowly Changing Dimension (SCD Type 2) to accurately track historical product pricing changes over time.
- The fact table references **DimDate** through the DateKey foreign key. This acts as a comprehensive date table supporting year, quarter, month, and day granularity for precise time-based and accumulating snapshot analysis (e.g., tracking order processing times).

Key Elements of the Dimensional Model

Fact Tables ---- FactOrders

Dimensions ---- DimUser, DimProduct, DimOrderStatus, DimDate.

Slowly Changing Dimension (SCD)

DimProduct is implemented as a Slowly Changing Dimension (SCD Type 2) to capture changes like product price updates over time.

Assumption

In the data warehouse design, DimProduct was chosen as the Slowly Changing Dimension (SCD). This decision was made because product attributes, particularly the price, are subject to change over time. By implementing DimProduct as a Slowly Changing Dimension, it becomes possible to track historical changes in product pricing, ensuring that historical sales calculations remain accurate based on the price at the time of the order.

Step 5: : ETL development

In ETL (Extract, Transform, Load):

- **Extract** → Only collects raw data (no major changes).

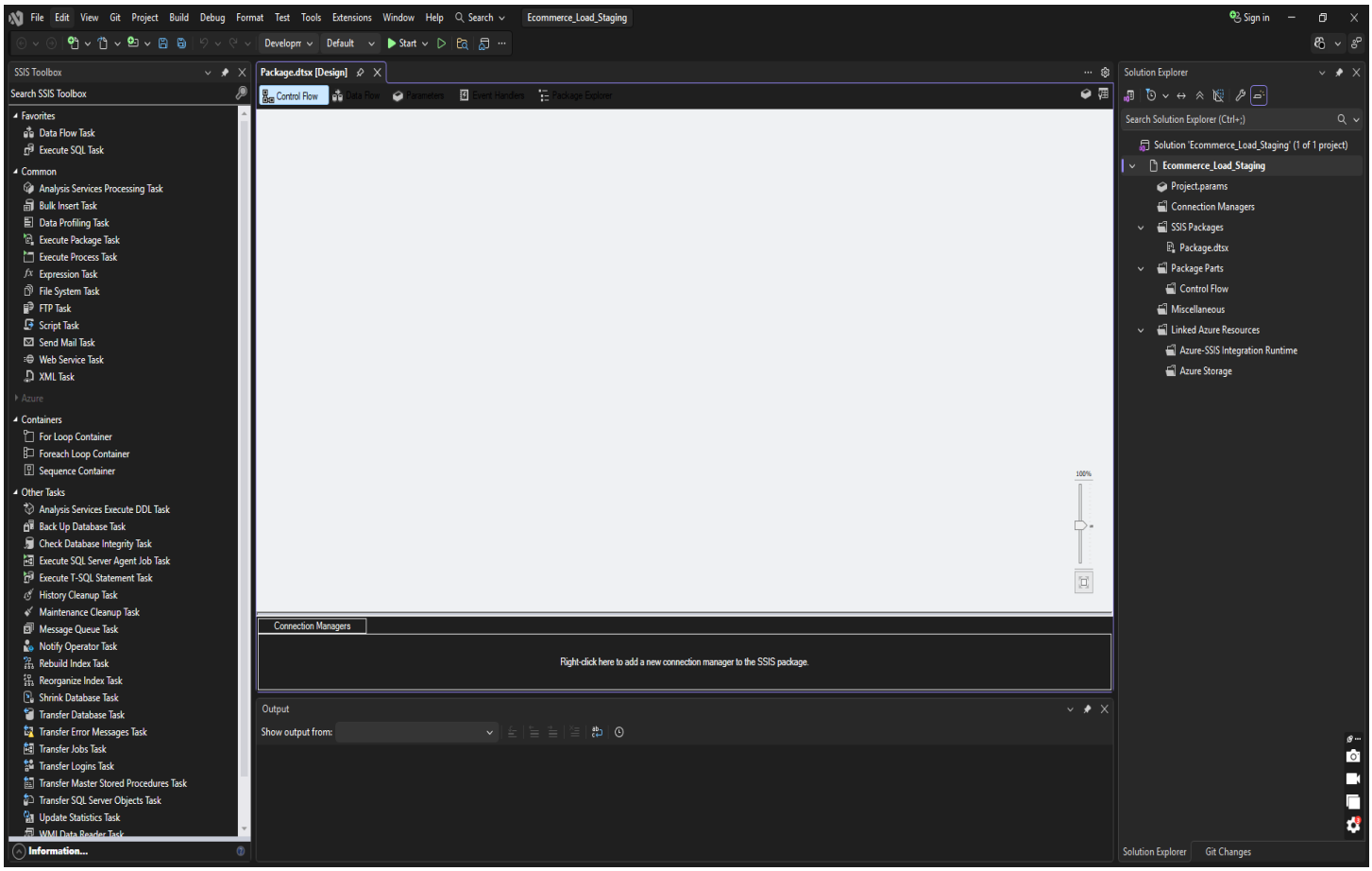
- **Transform** → Cleans, modifies, and corrects the data (this is where data conversion happens).
- **Load** → Inserts the clean, final data into the warehouse.

1) Extracting

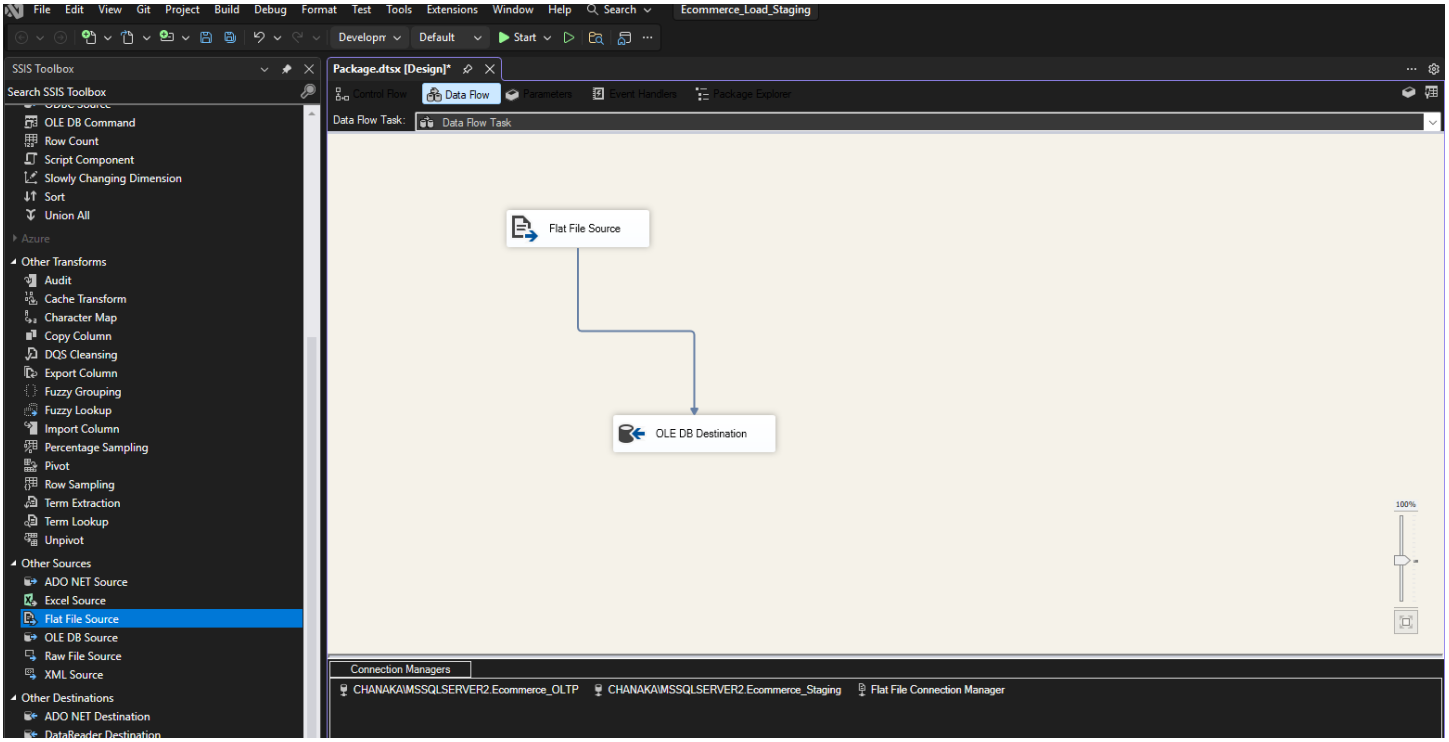
In this project, extraction was performed to collect and organize data from multiple source types, including CSV and Excel files, into the initial operational source database (**`Ecommerce_OLTP`**). The data was then extracted and loaded into specific staging tables within the **`Ecommerce_Staging`** database, accurately separating user, product, order, and status information. This structured extraction process enabled easier data cleaning, validation, and preliminary formatting before the final transformation and loading into the data warehouse (**`Ecommerce_DW`**). This ensures data consistency, completeness, and readiness for dimensional modeling.

Staging is done during the extracting step to safely store the raw extracted data in temporary tables before applying complex transformations such as Surrogate Key generation and Slowly Changing Dimension (SCD) logic.

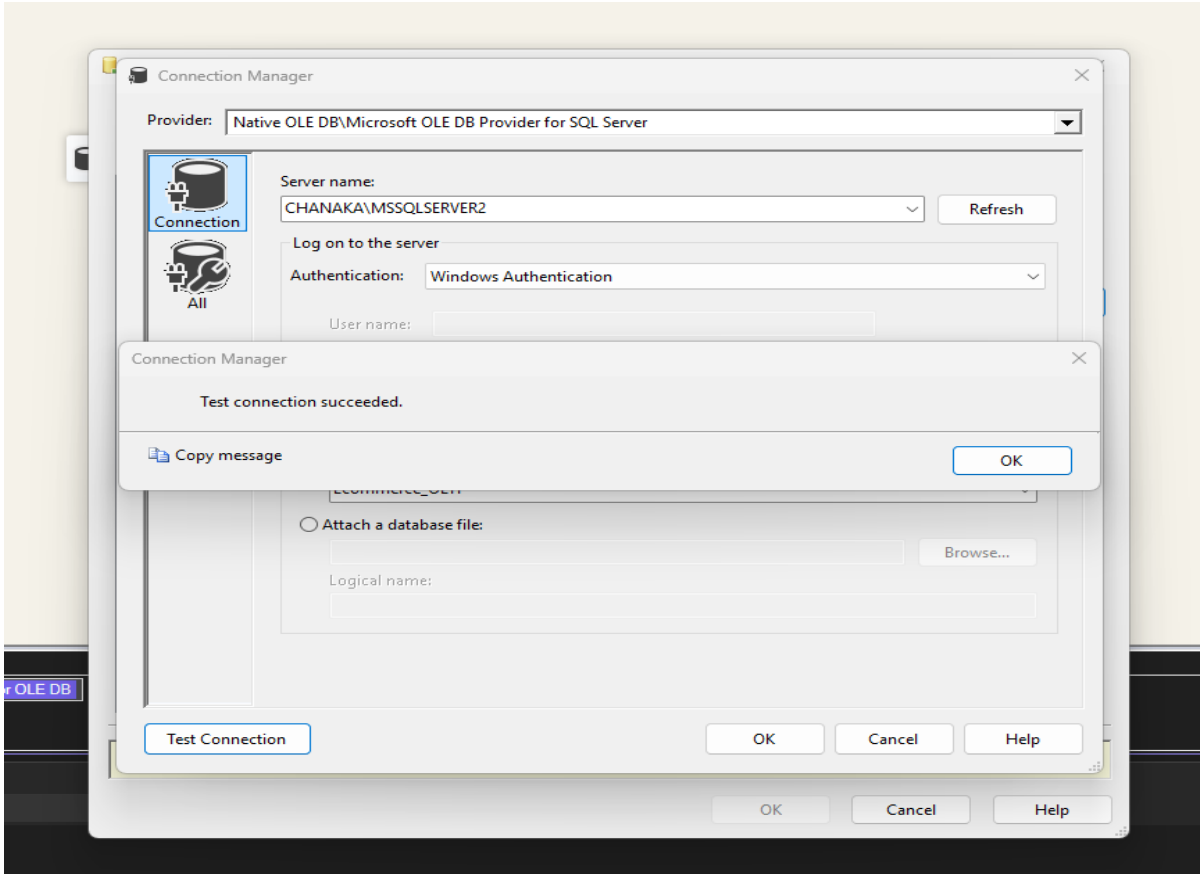
5.1.1 Create the SSIS project



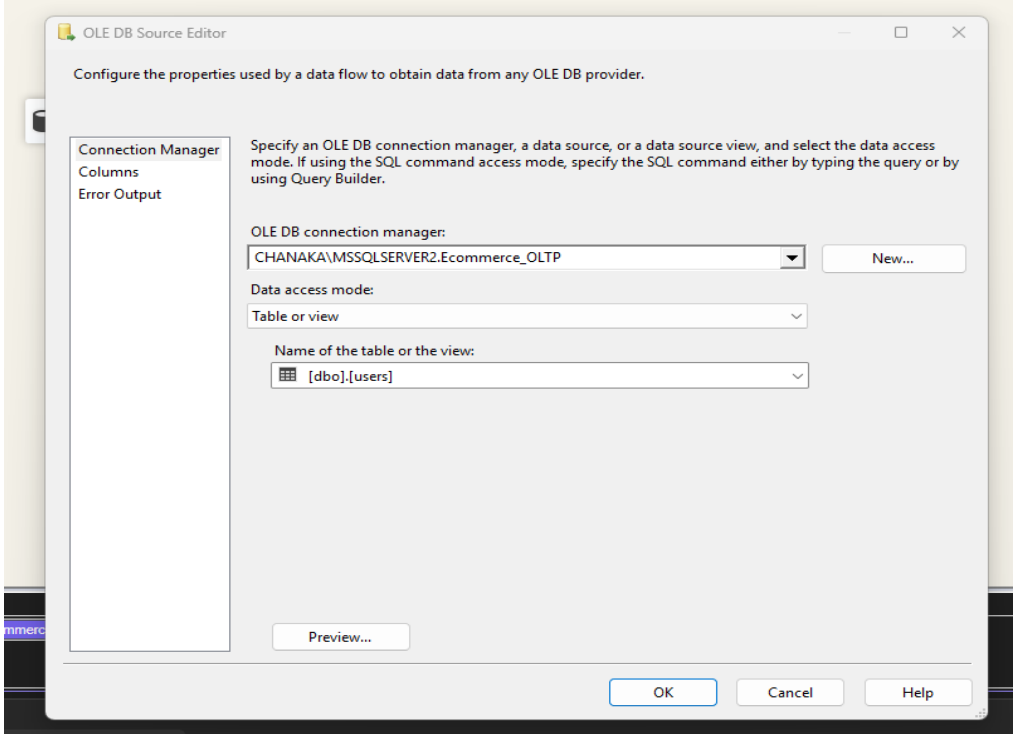
OLE DB Source is used to read data from a database. It would read data from your source table, e.g., users in the Ecommerce_OLTP source database.
OLE DB Destination is used to write (insert) data into a SQL table (like a staging or warehouse table). It would write the data into your staging table, e.g., stg_users in your Ecommerce_Staging database.



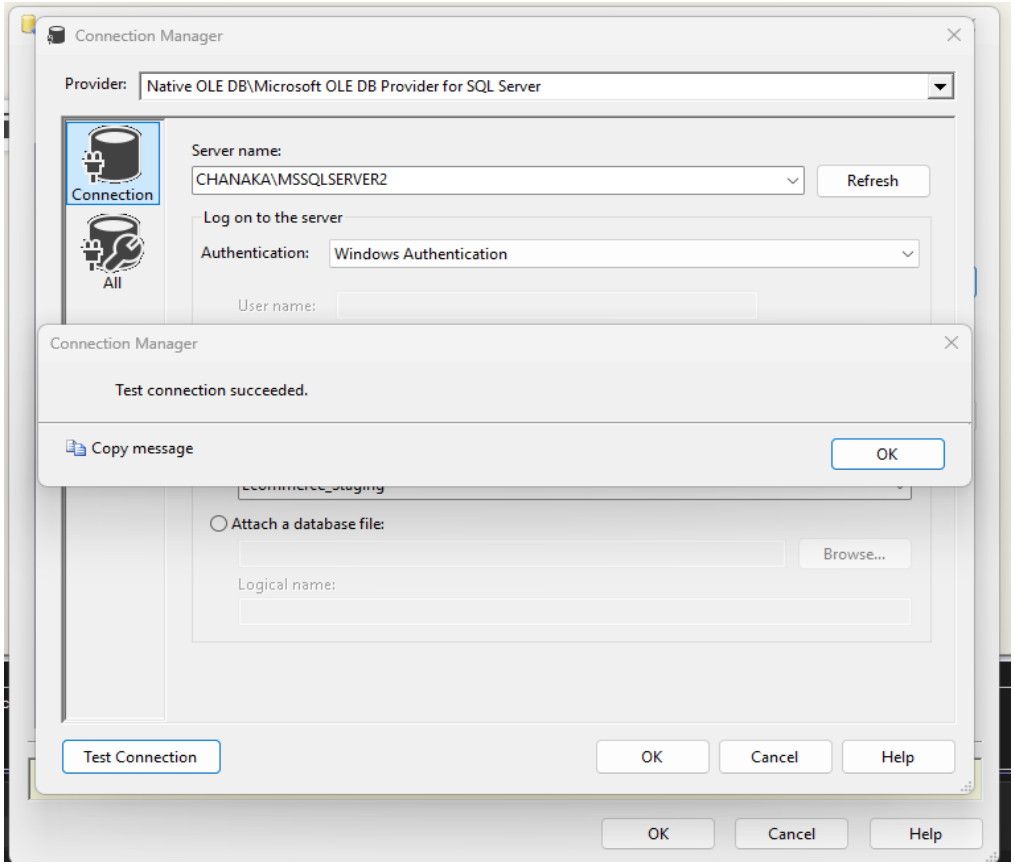
5.1.2 Made the Test Connection in OLE DB Source



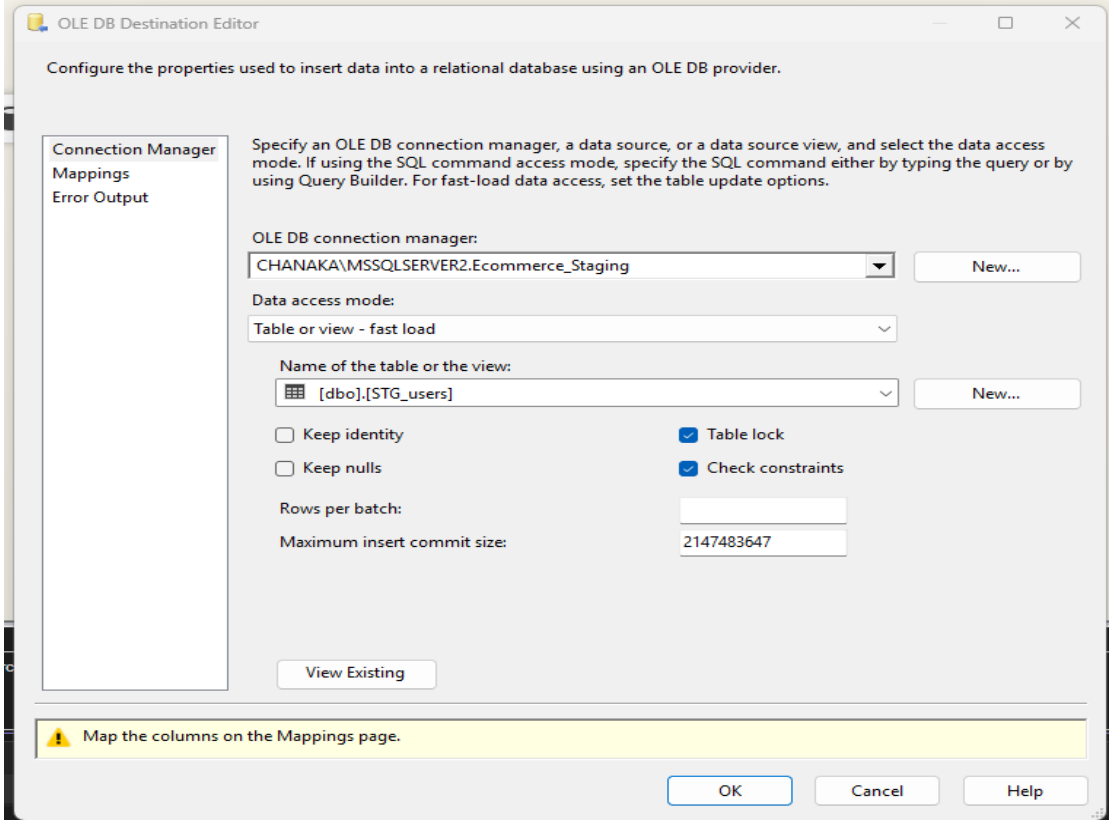
5.1.3 First the Users staging table



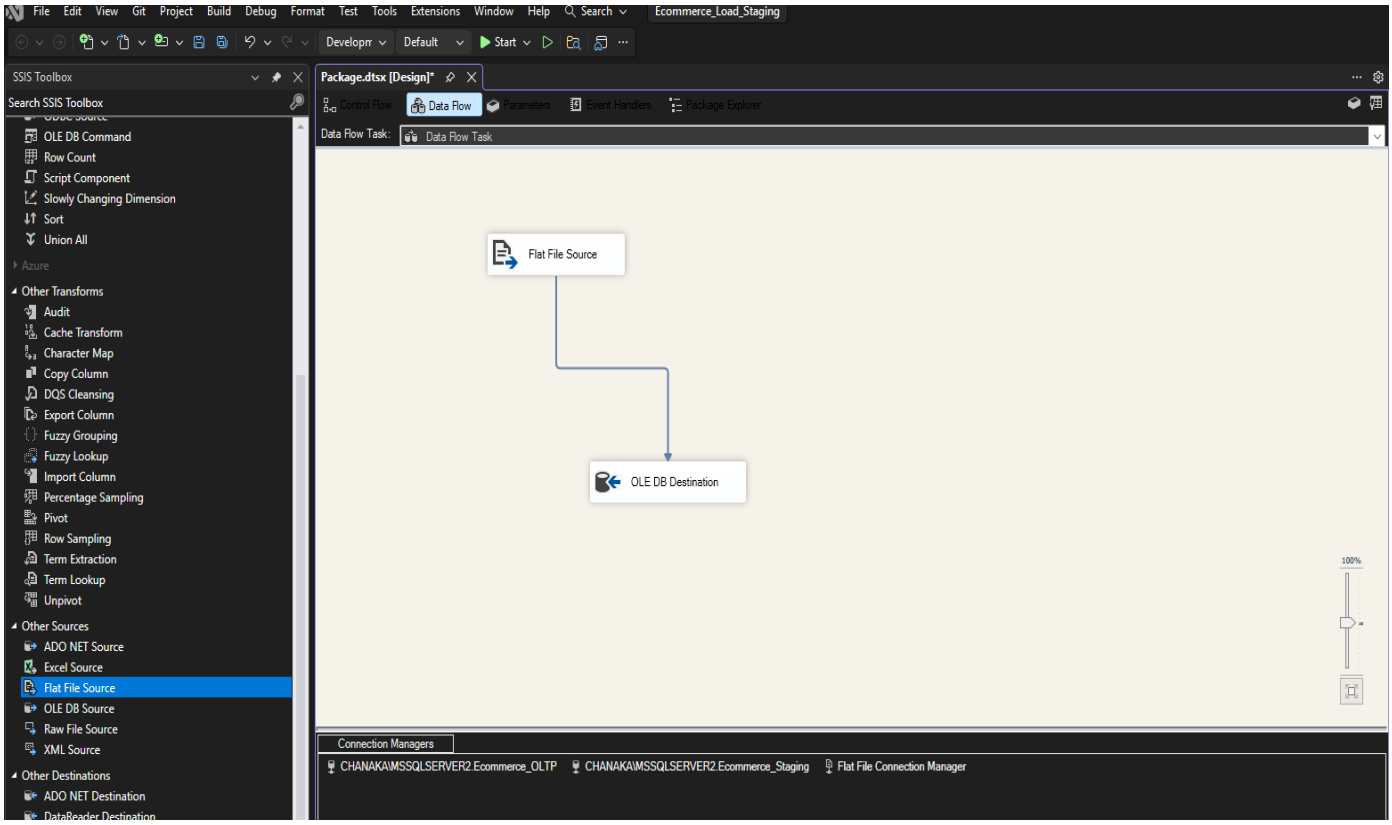
5.1.4 Making the Connection for OLE DB Destination.



5.1.5 Selected Completed Client Staging for OLE DB destination.

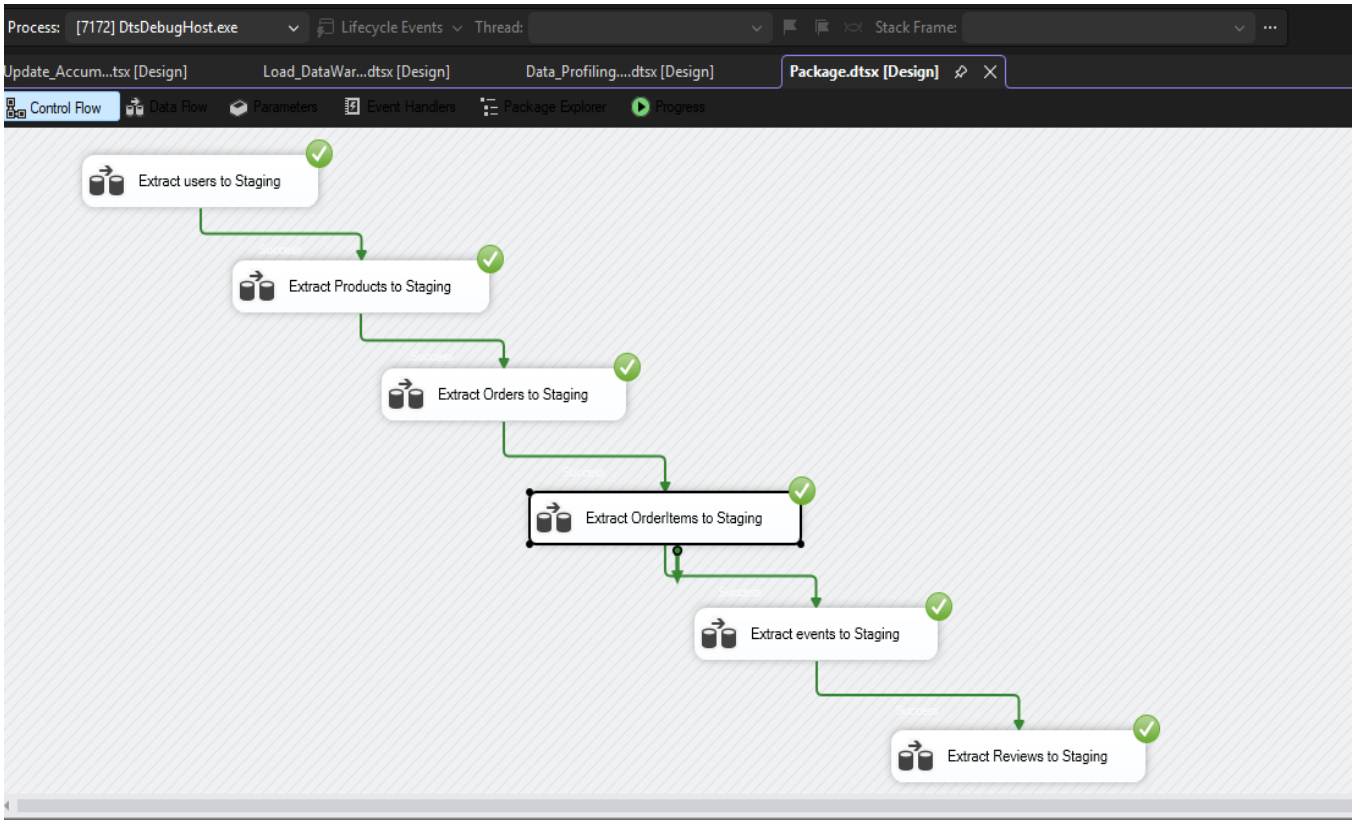


5.1.6 Final Output of data flow of staging table of Users.

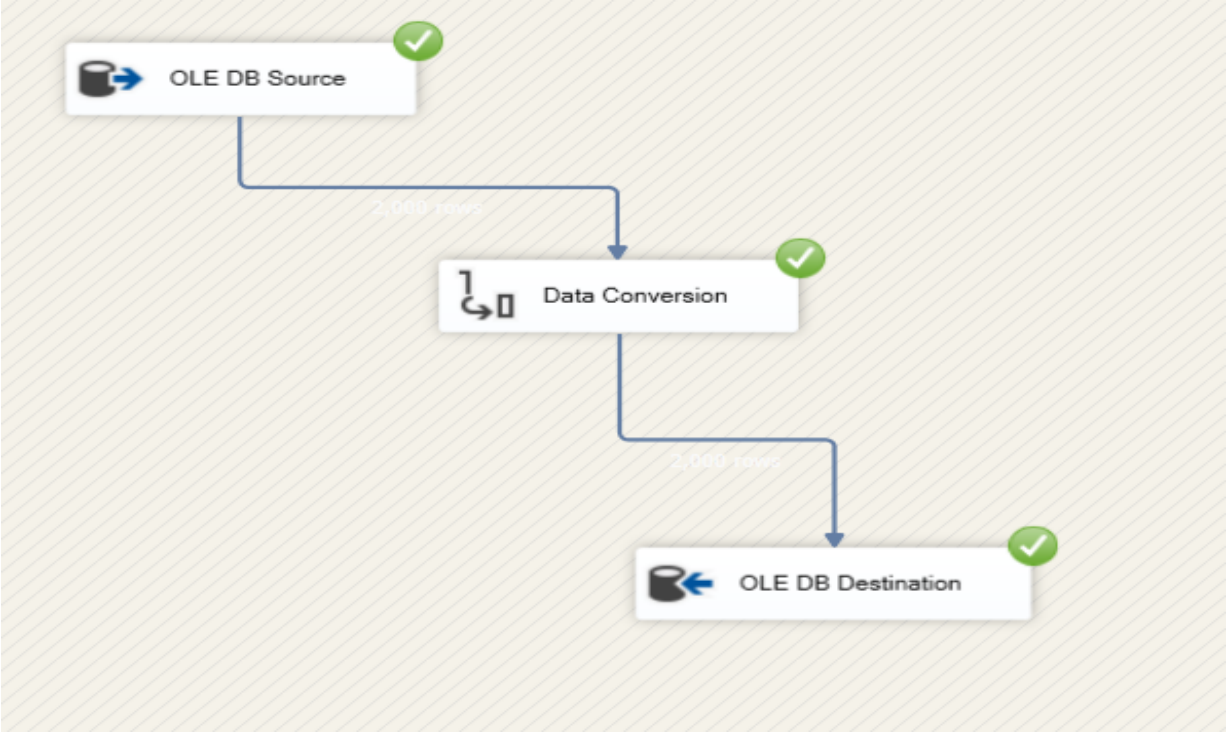


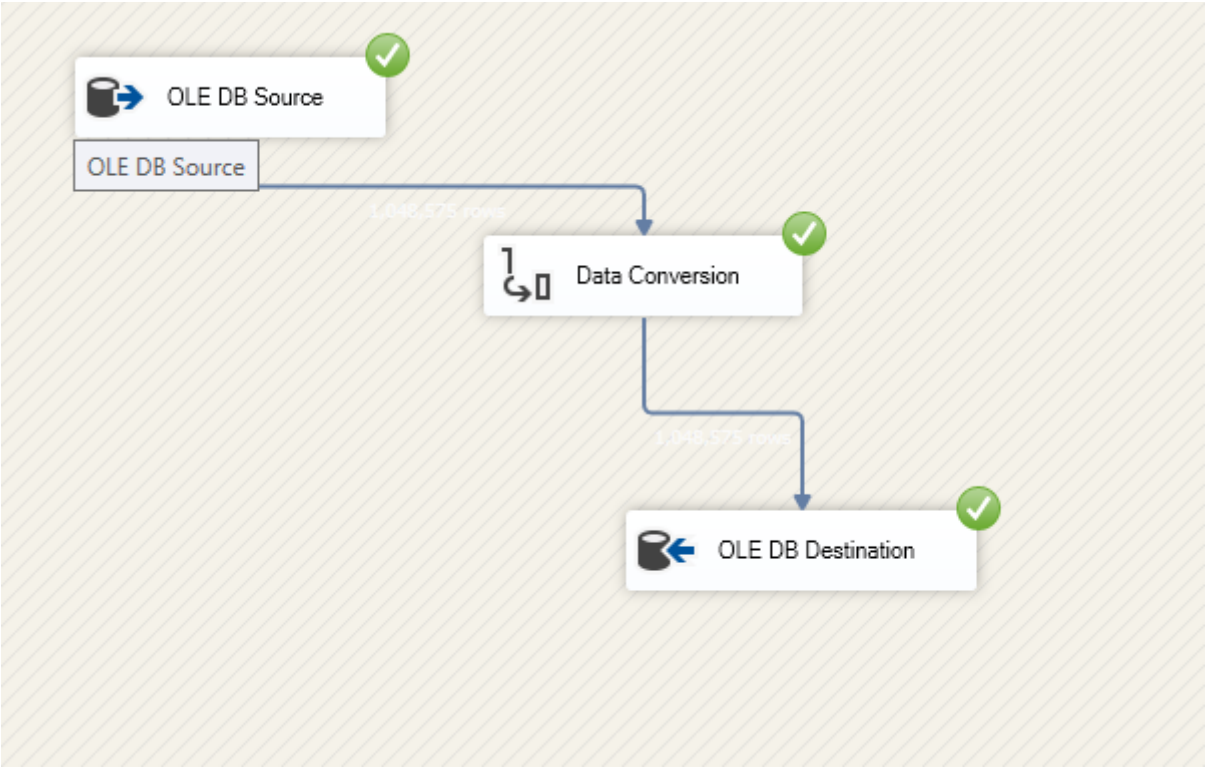
The same steps were repeated for every staging table and for each a data flow.

5.1.7 Final Output after Extracting,

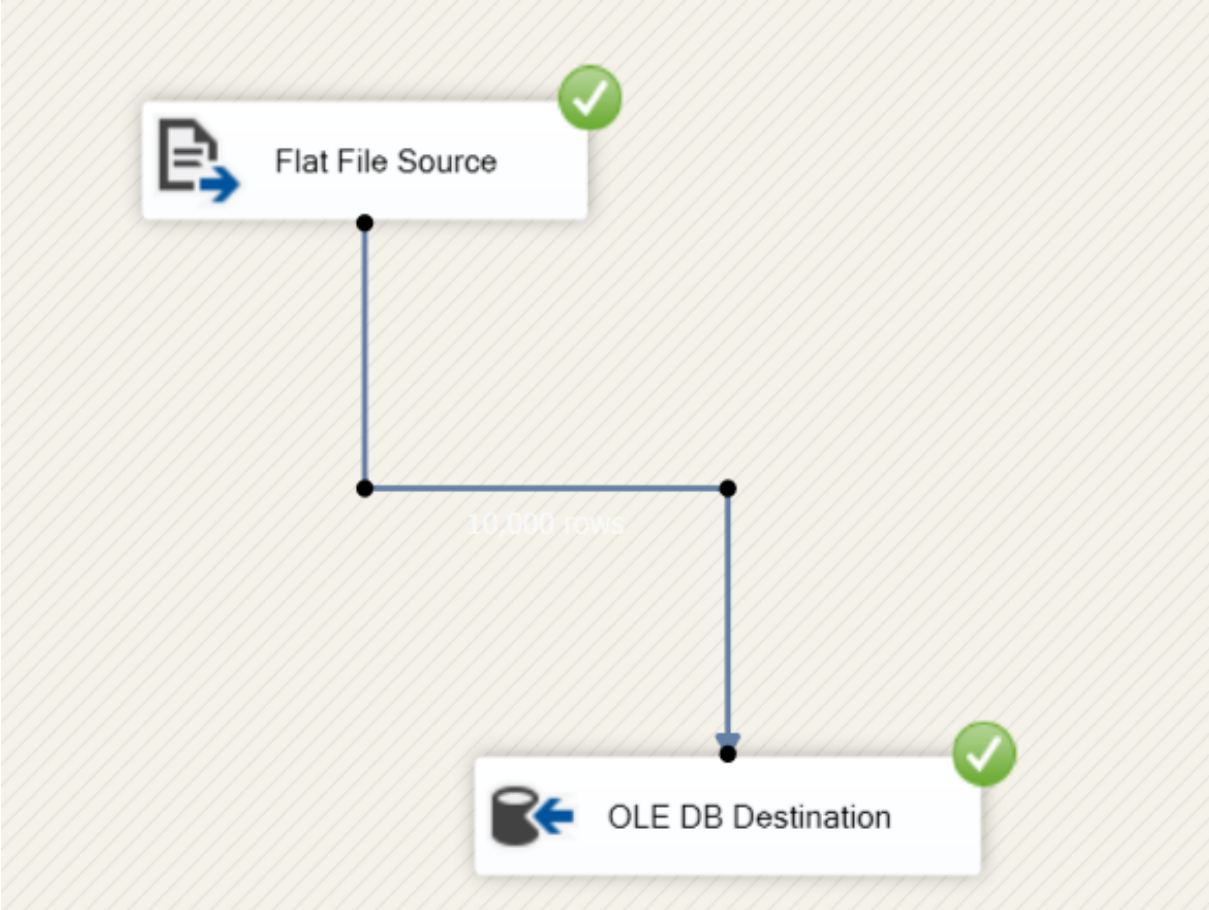


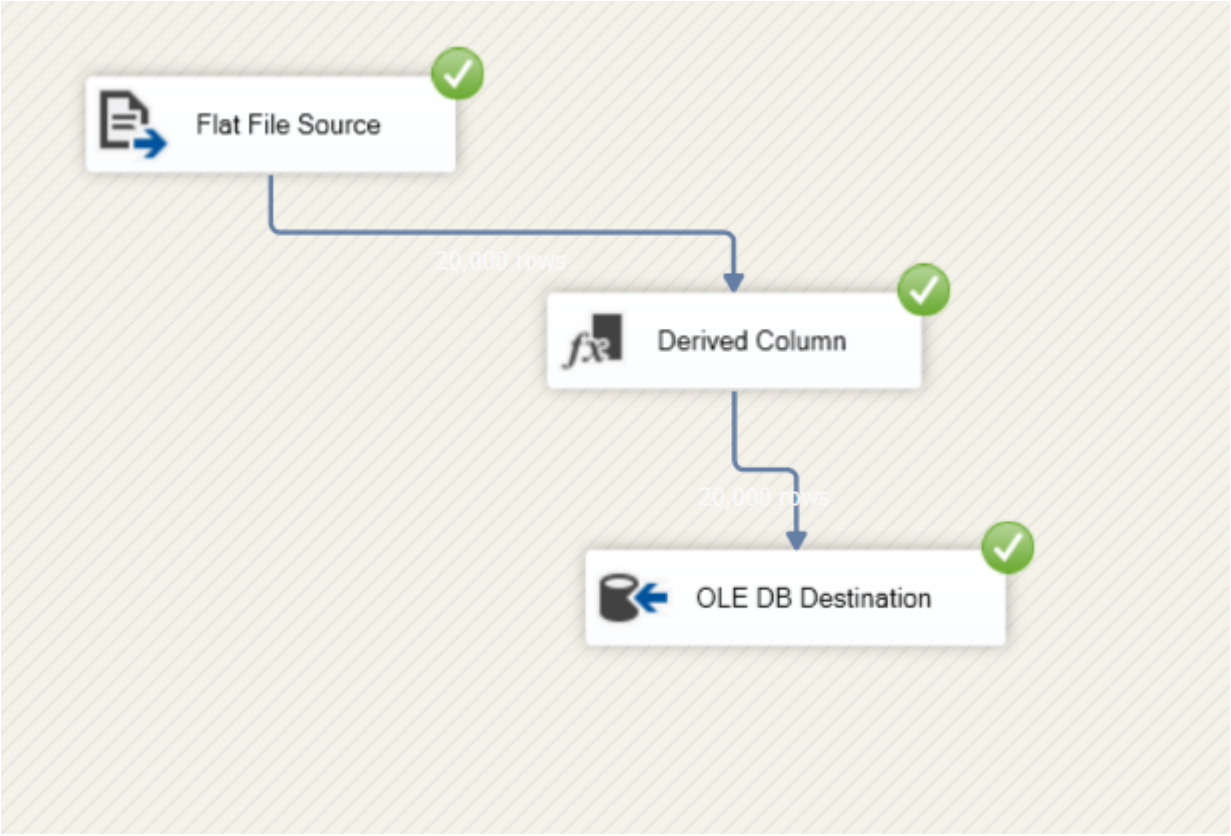
5.1.8 Data Flow Tasks for Excel Files depicted below,

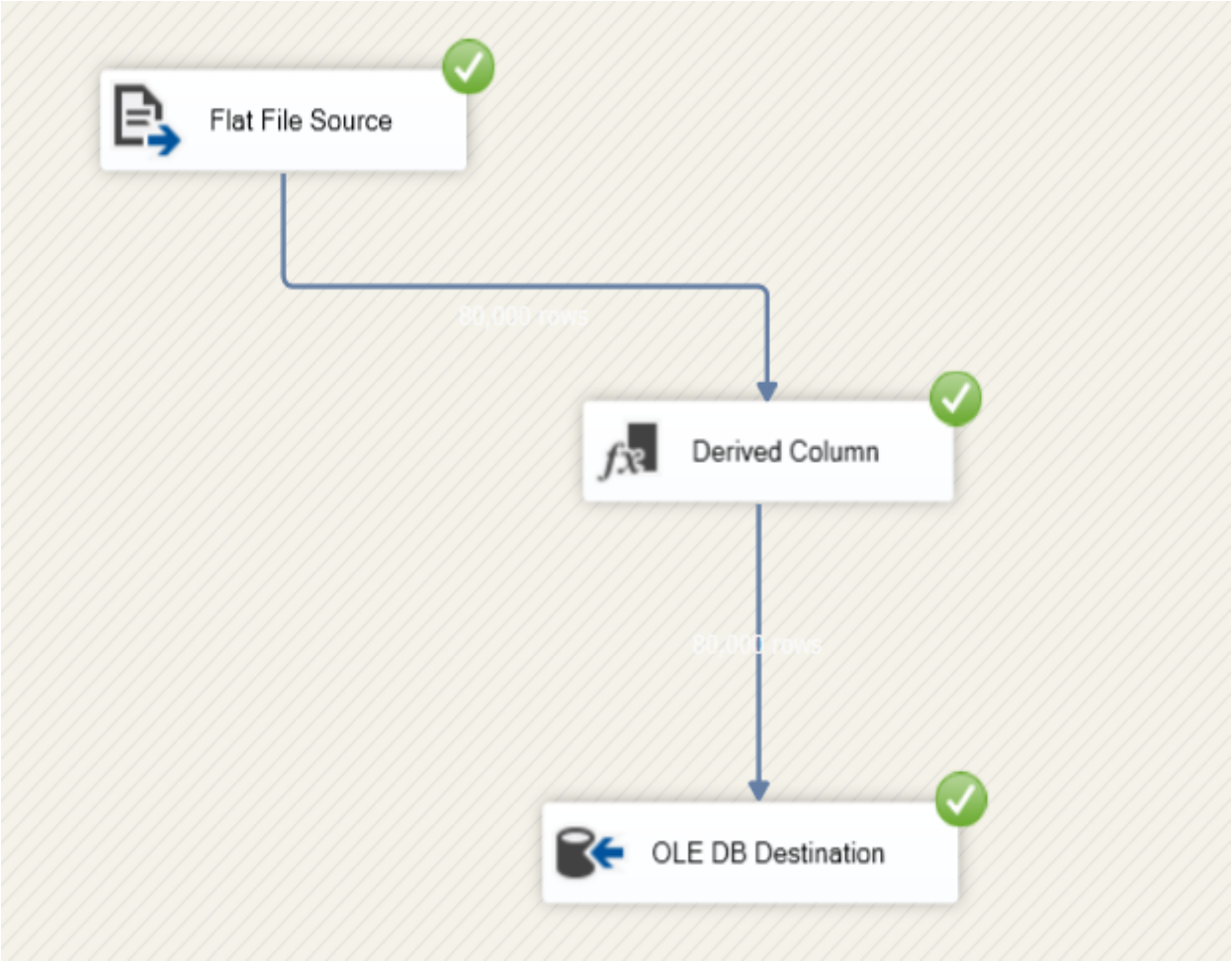


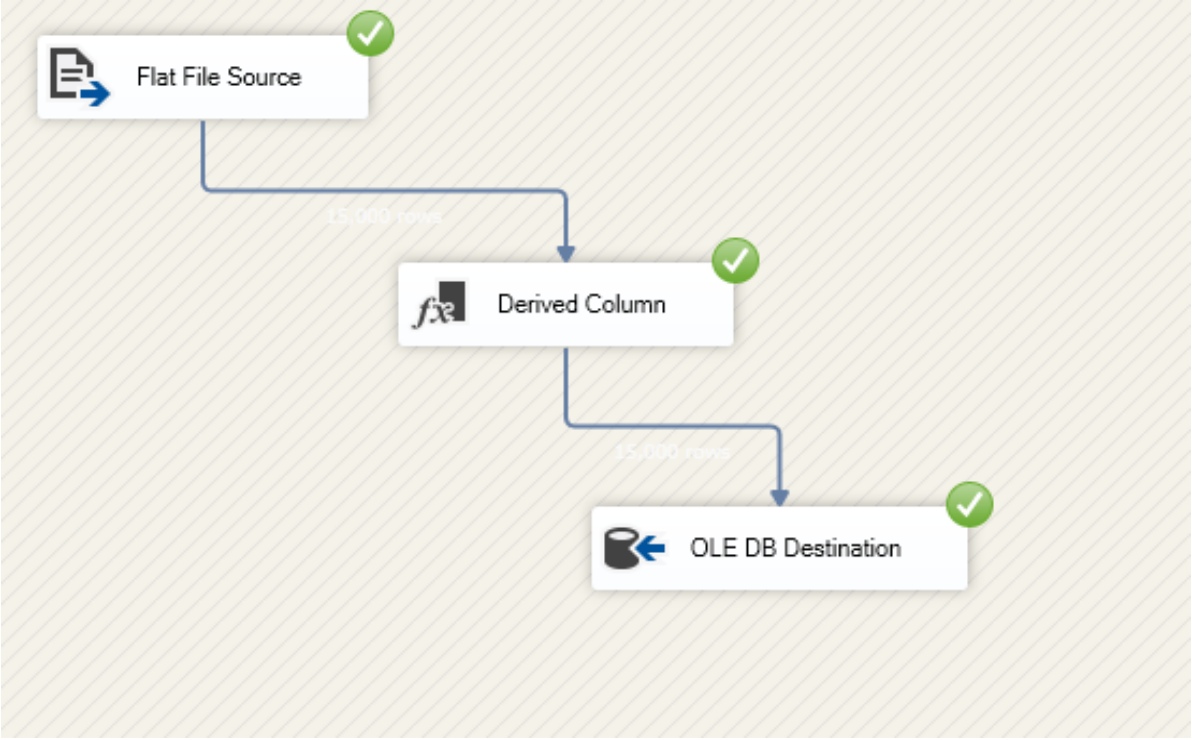


5.1.9 Data Flow Tasks for csv Files depicted below,




Flat File Source
20,000 rows
Derived Column
20,000 rows
OLE DB Destination


Flat File Source
80,000 rows
Derived Column
80,000 rows
OLE DB Destination

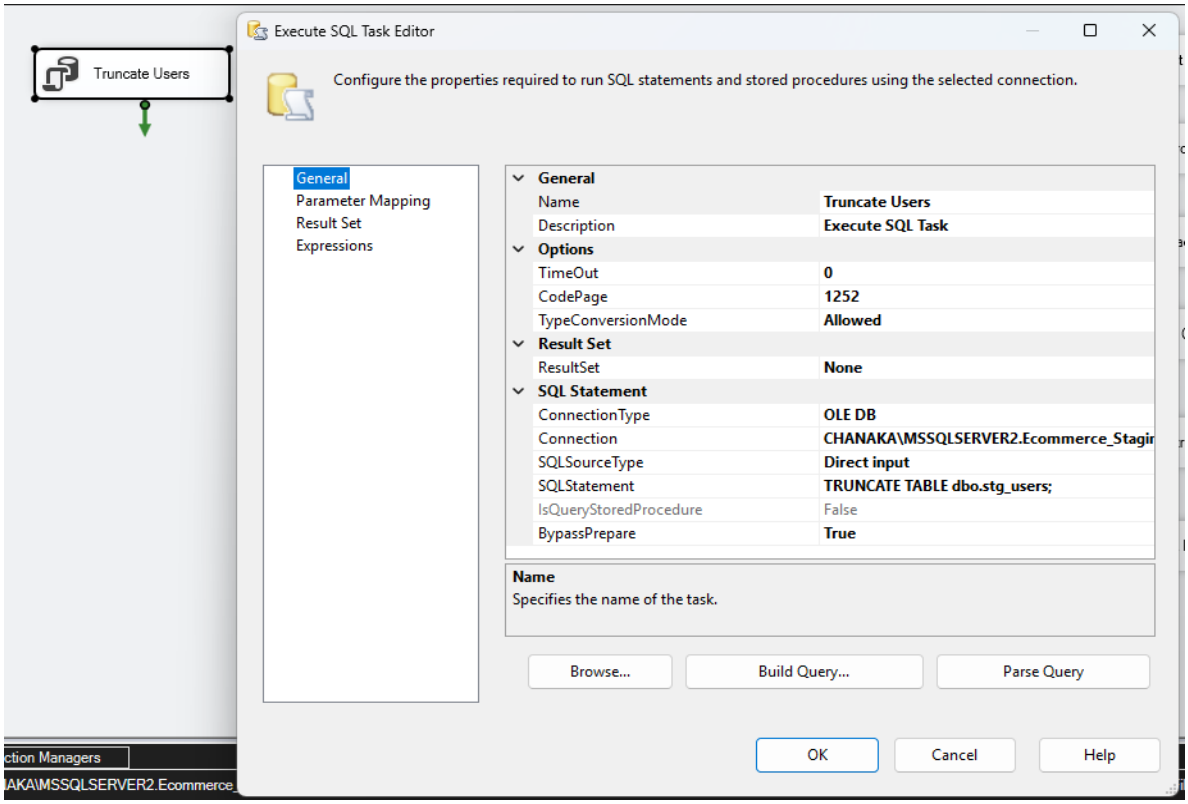


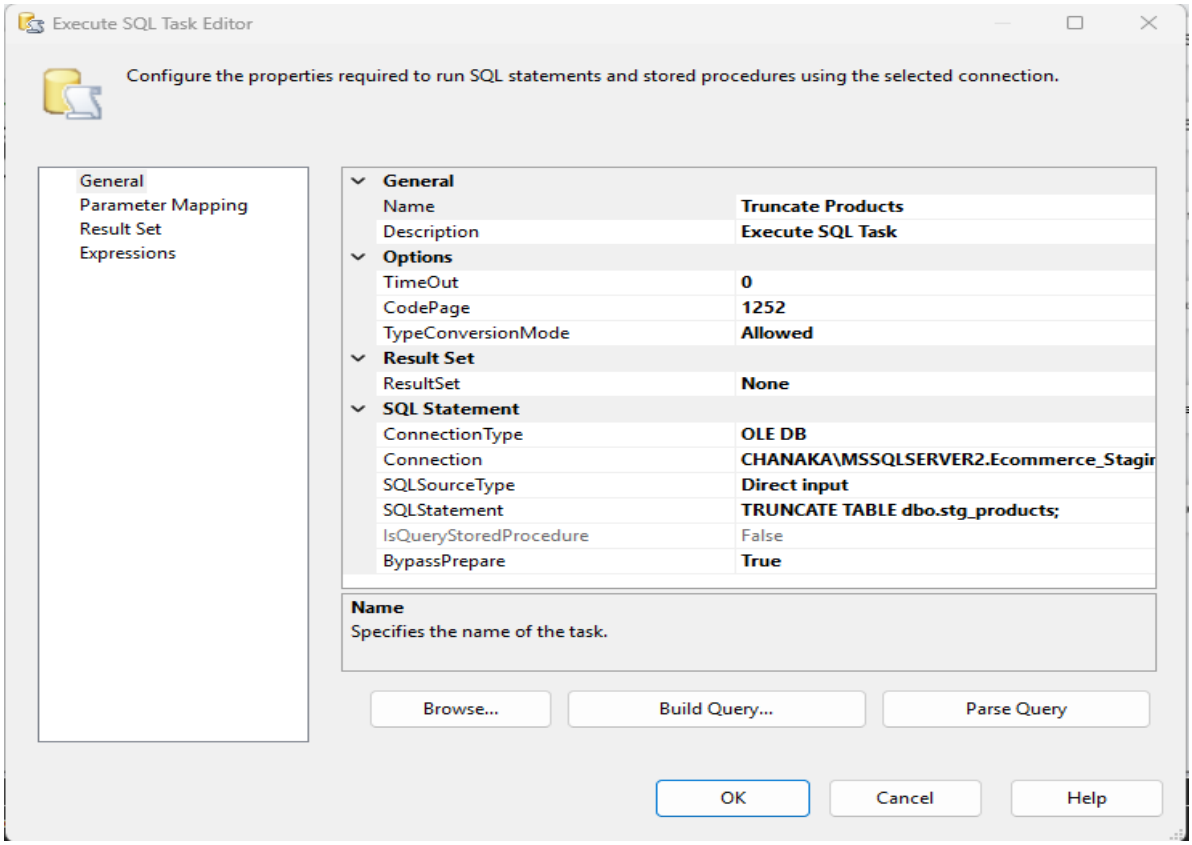
2)Truncate Tables

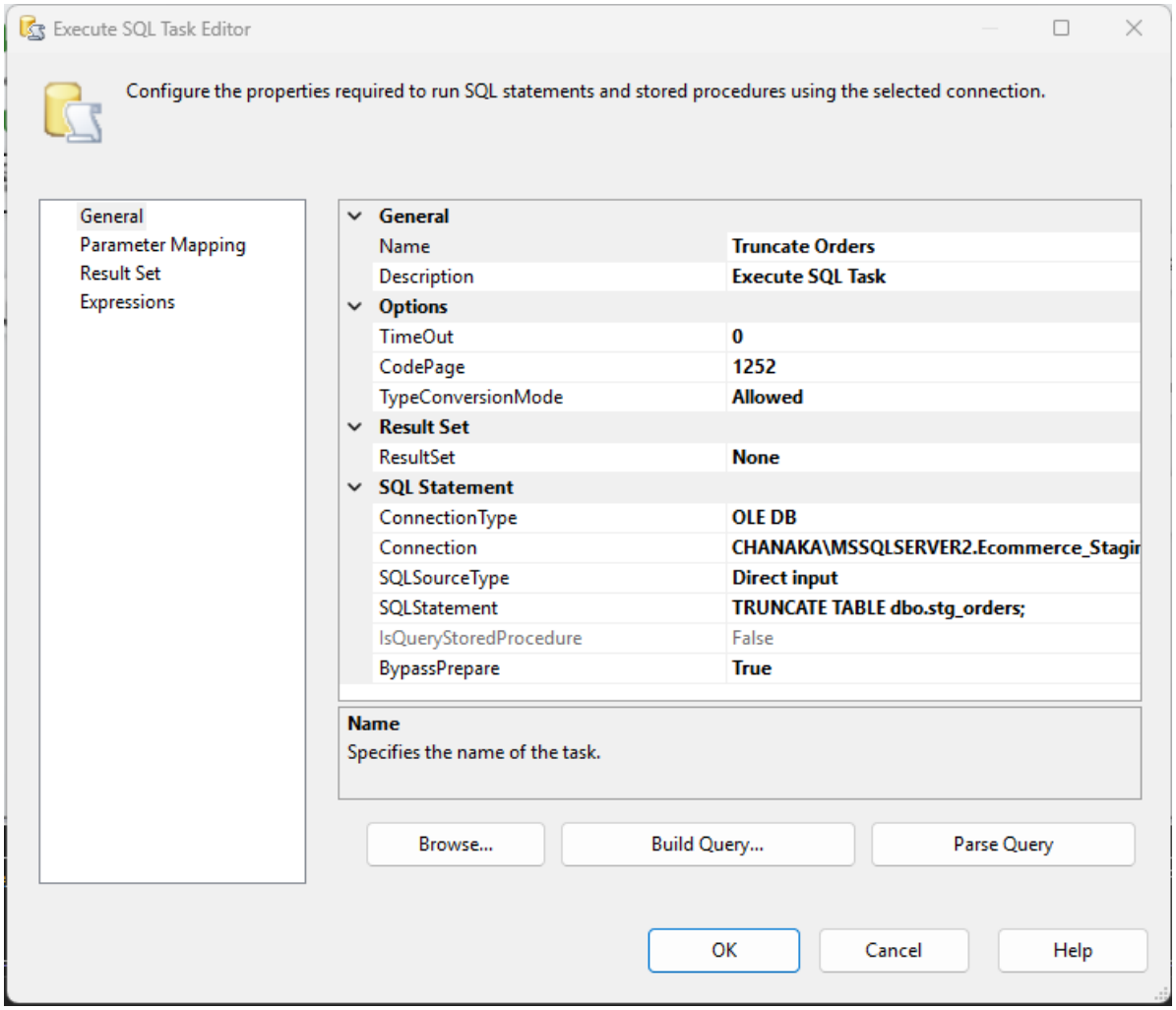
In this project, the **TRUNCATE TABLE** command was used during the ETL process to clear the staging tables in the **Ecommerce_Staging** database before reloading fresh data. This ensures that every time the ETL package runs, the staging tables start empty, preventing old or duplicate data from remaining in the staging area.

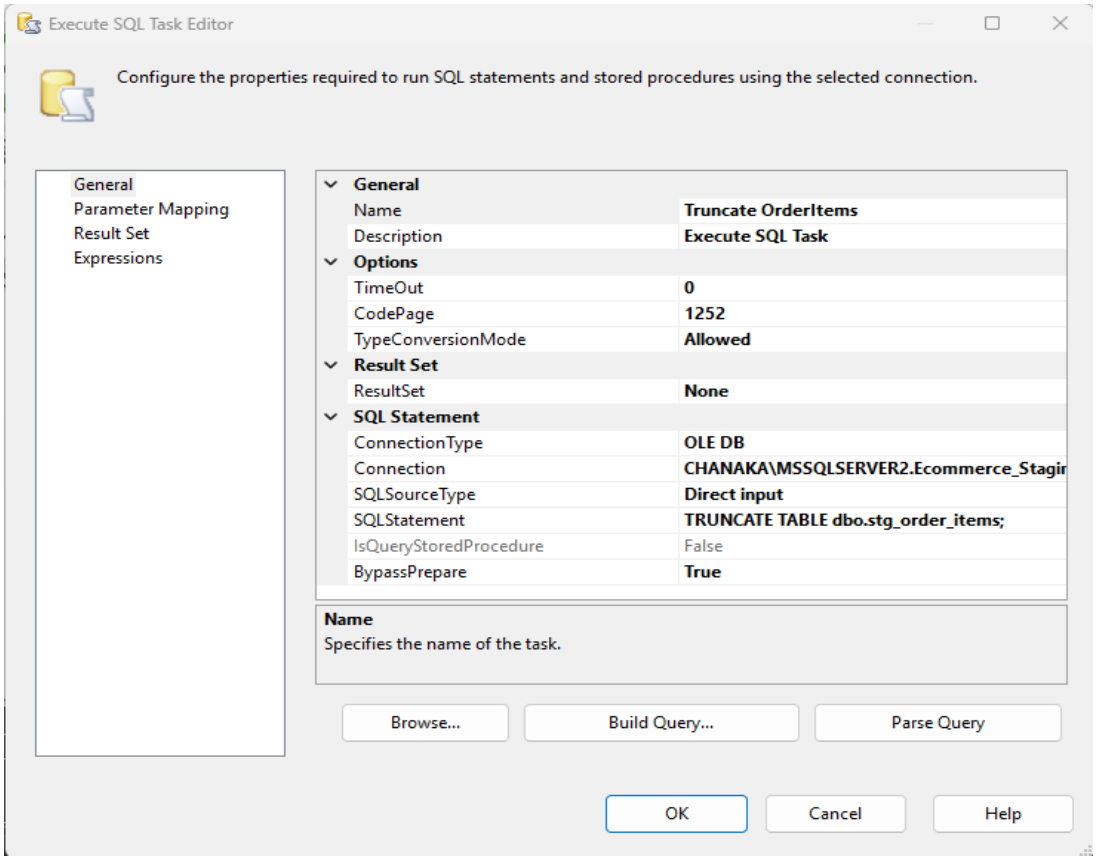
By truncating staging tables before each data load, the data extraction and transformation processes could proceed cleanly, guaranteeing data consistency, accuracy, and avoiding duplication in the final Data Warehouse (**Ecommerce_DW**).

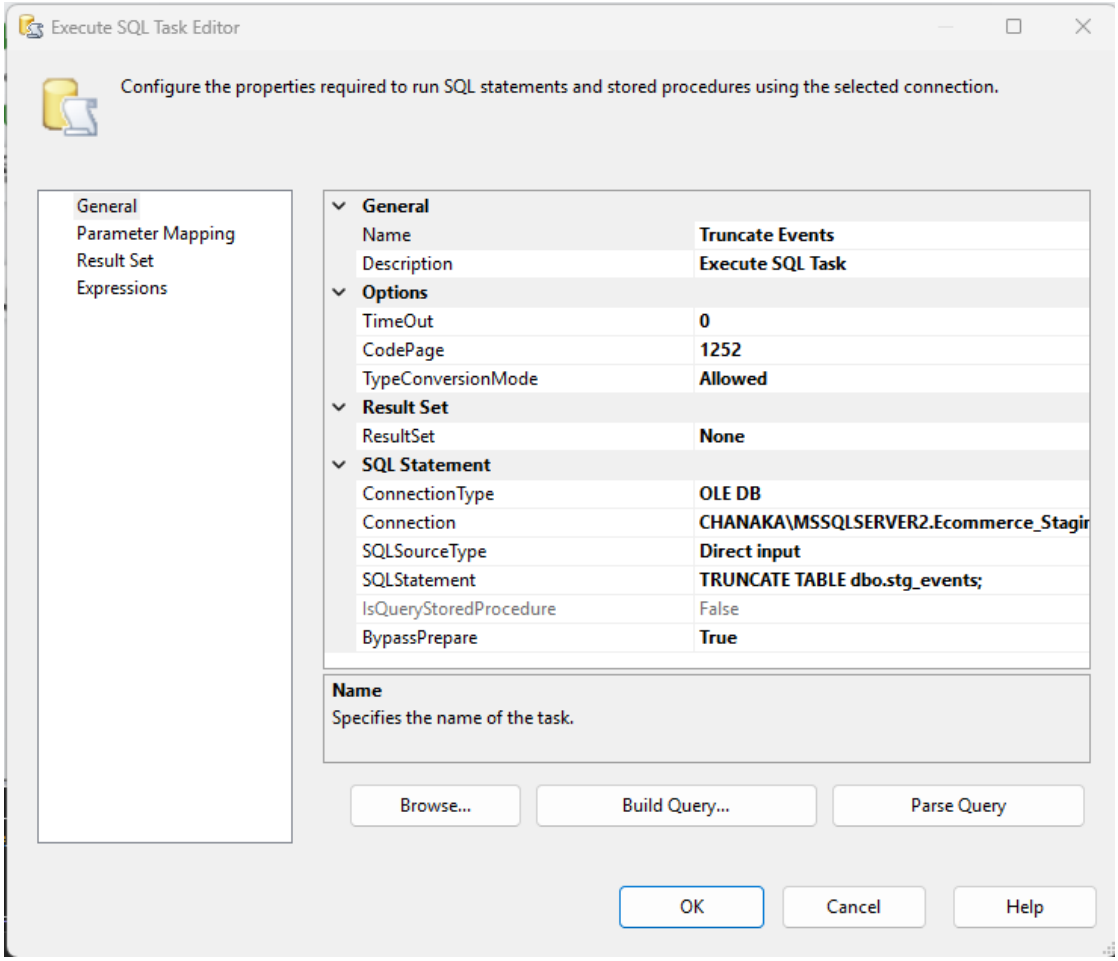
Truncate Table were done to all tables as depicted below,

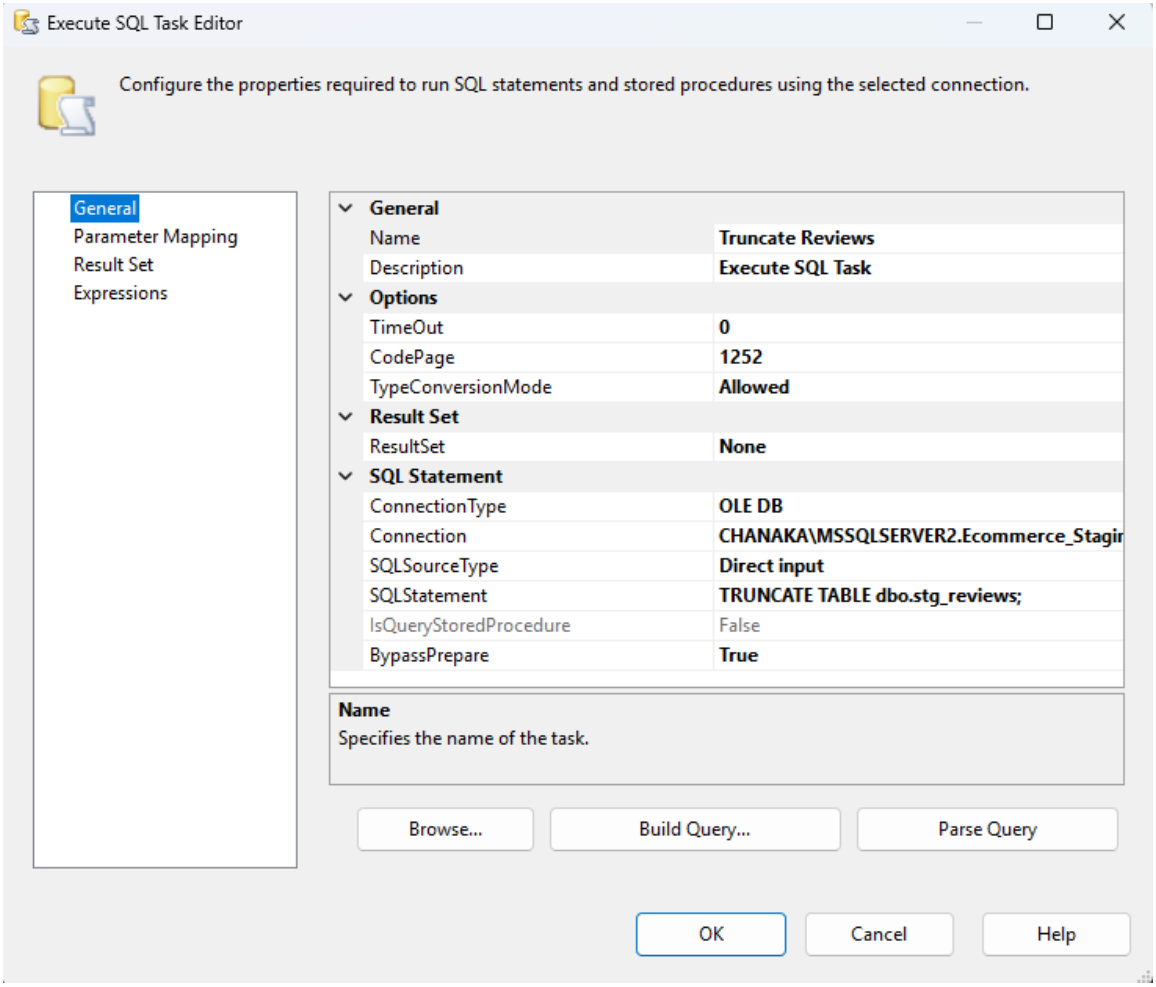



Execute SQL Task Editor
Configure the properties required to run SQL statements and stored procedures using the selected connection.
General
Parameter Mapping
Result Set
Expressions
General
Name
Truncate Products
Description
Execute SQL Task
Options
TimeOut
0
CodePage
1252
TypeConversionMode
Allowed
Result Set
ResultSet
None
SQL Statement
ConnectionType
OLE DB
Connection
CHANAKA\MSSQLSERVER2.Ecommerce_Stagir
SQLSourceType
Direct input
SQLStatement
TRUNCATE TABLE dbo.stg_products;
IsQueryStoredProcedure
False
BypassPrepare
True
Name
Specifies the name of the task.
Browse...
Build Query...
Parse Query
OK
Cancel
Help

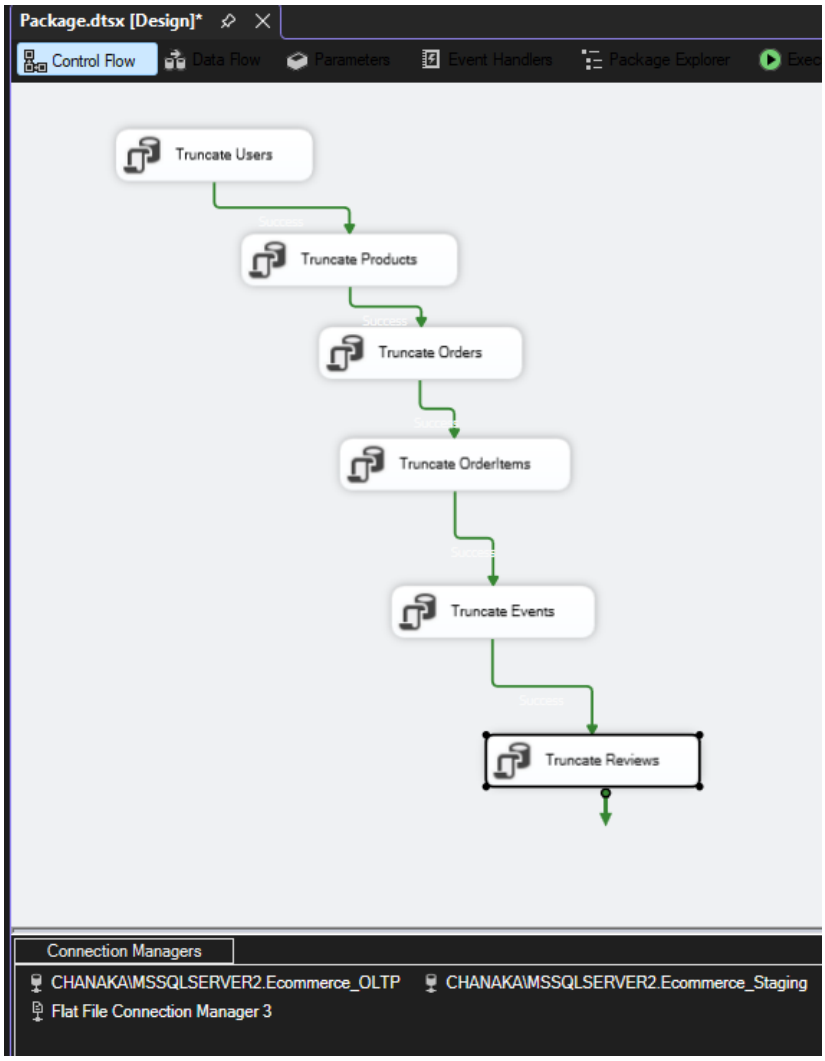

Execute SQL Task Editor
Configure the properties required to run SQL statements and stored procedures using the selected connection.
General
Parameter Mapping
Result Set
Expressions
General
Name
Truncate Orders
Description
Execute SQL Task
Options
TimeOut
0
CodePage
1252
TypeConversionMode
Allowed
Result Set
ResultSet
None
SQL Statement
ConnectionType
OLE DB
Connection
CHANAKA\MSSQLSERVER2.Ecommerce_Stagir
SQLSourceType
Direct input
SQLStatement
TRUNCATE TABLE dbo.stg_orders;
IsQueryStoredProcedure
False
BypassPrepare
True
Name
Specifies the name of the task.
Browse...
Build Query...
Parse Query
OK
Cancel
Help


Execute SQL Task Editor
Configure the properties required to run SQL statements and stored procedures using the selected connection.
General
Parameter Mapping
Result Set
Expressions
General
Name
Truncate OrderItems
Description
Execute SQL Task
Options
TimeOut
0
CodePage
1252
TypeConversionMode
Allowed
Result Set
ResultSet
None
SQL Statement
ConnectionType
OLE DB
Connection
CHANAKA\MSSQLSERVER2.Ecommerce_Stagir
SQLSourceType
Direct input
SQLStatement
TRUNCATE TABLE dbo.stg_order_items;
IsQueryStoredProcedure
False
BypassPrepare
True
Name
Specifies the name of the task.
Browse...
Build Query...
Parse Query
OK
Cancel
Help


Execute SQL Task Editor
Configure the properties required to run SQL statements and stored procedures using the selected connection.
General
Parameter Mapping
Result Set
Expressions
General
Name
Truncate Events
Description
Execute SQL Task
Options
TimeOut
0
CodePage
1252
TypeConversionMode
Allowed
Result Set
ResultSet
None
SQL Statement
ConnectionType
OLE DB
Connection
CHANAKA\MSSQLSERVER2.Ecommerce_Stagir
SQLSourceType
Direct input
SQLStatement
TRUNCATE TABLE dbo.stg_events;
IsQueryStoredProcedure
False
BypassPrepare
True
Name
Specifies the name of the task.
Browse...
Build Query...
Parse Query
OK
Cancel
Help



Final Output after Truncating the table,

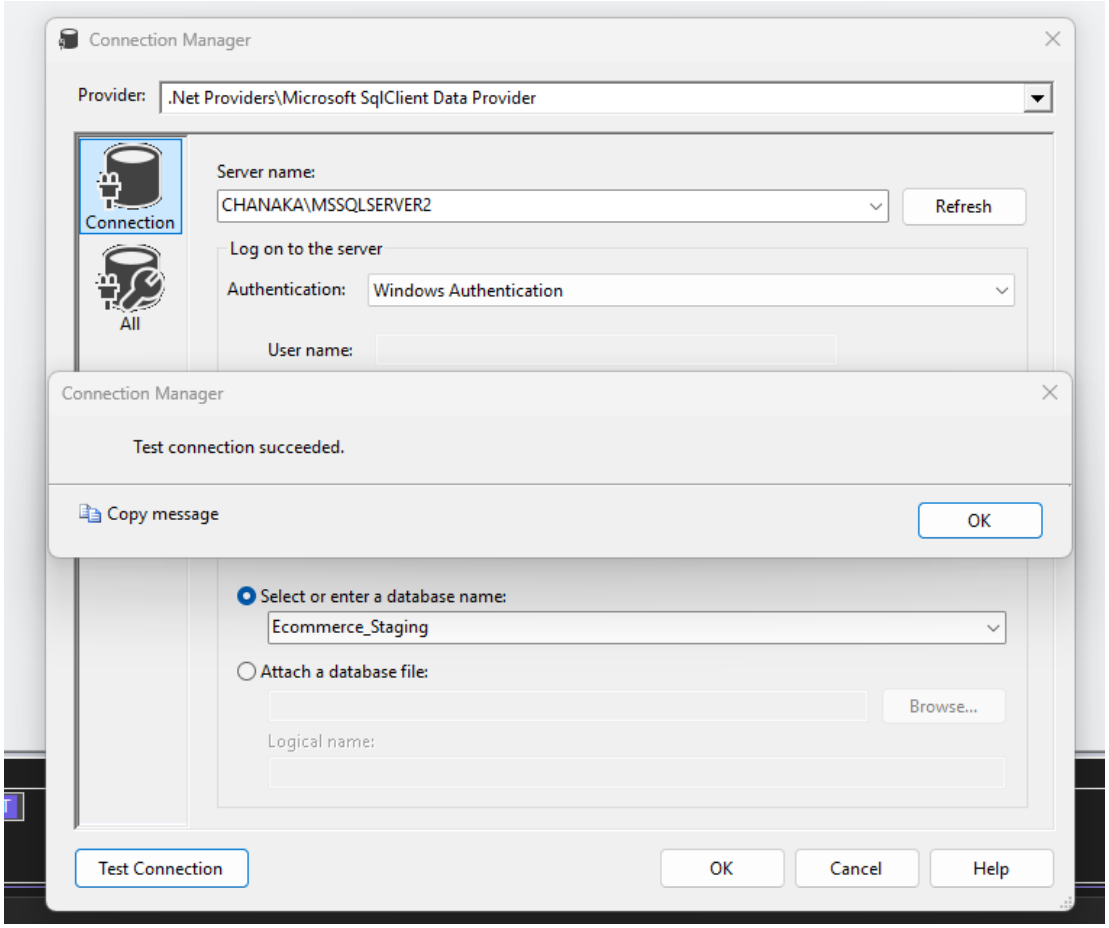


3)Data Profiling

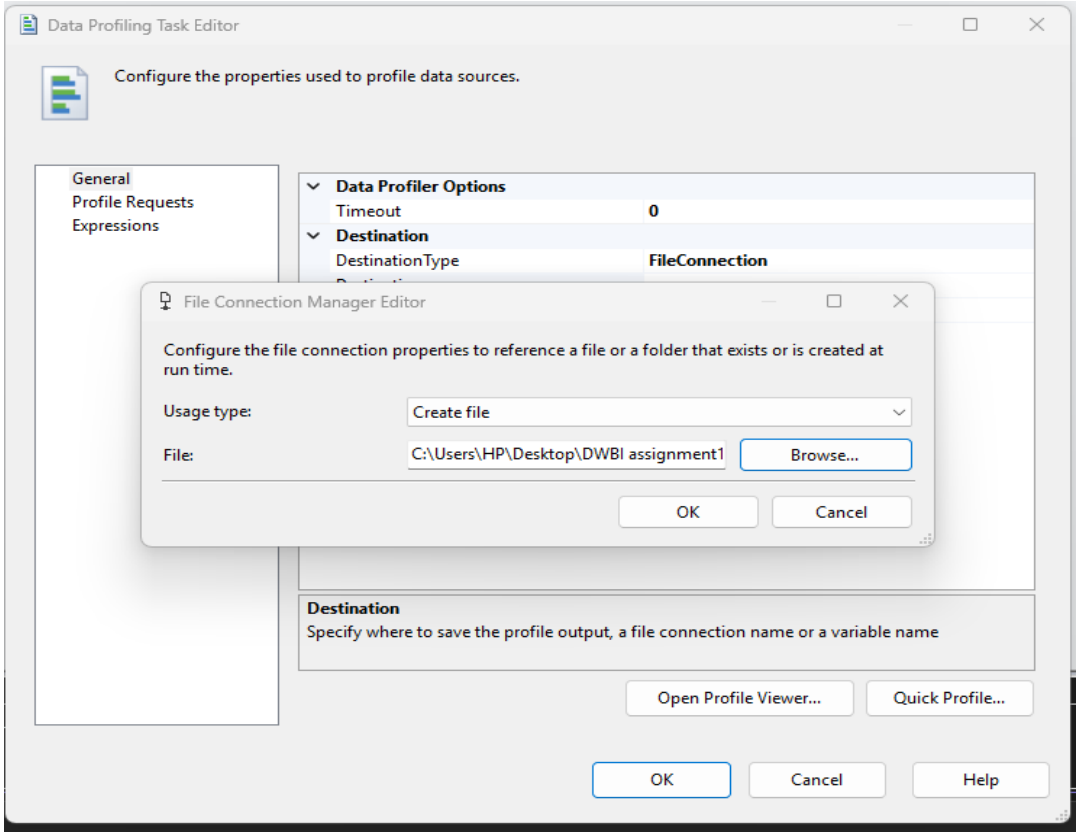
Data profiling was performed on the staging tables of the database following the extraction process. This step was crucial to evaluate the quality, consistency, and completeness of the sports tool data loaded from CSV and Excel sources. By utilizing the **Data Profiling Task** in SSIS, several data quality aspects were analyzed within the **Ecommerce_Staging** environment.

Key metrics such as **Null Ratio Profiles** were checked for critical fields like user_id and product_id, and **Column Statistics Profiles** were used to verify that numerical values like price and total_amount were within expected ranges. Identifying inconsistent formats or outliers at this stage allowed for necessary cleaning and transformations before the data reached the **Ecommerce_DW** data warehouse. This proactive approach ensures high-quality data for analytical reporting and decision-making.

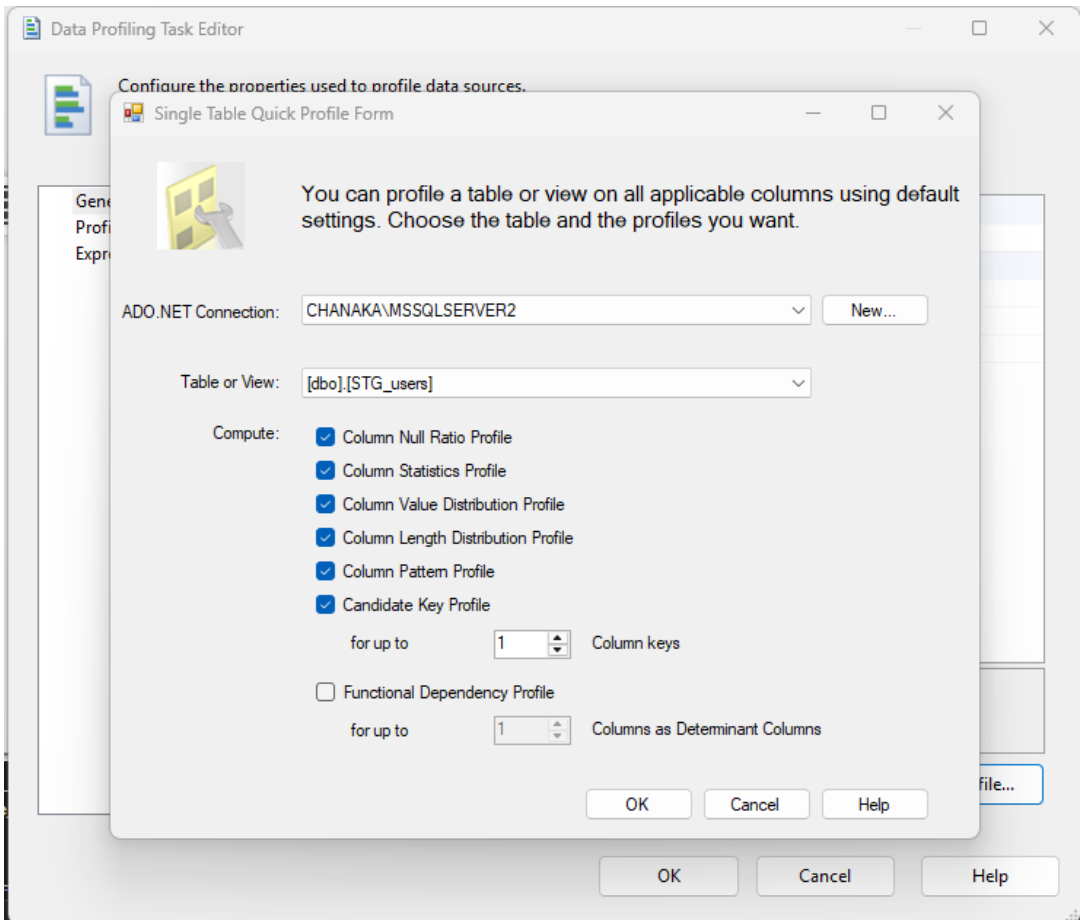
1. Connection Establishment and Verification



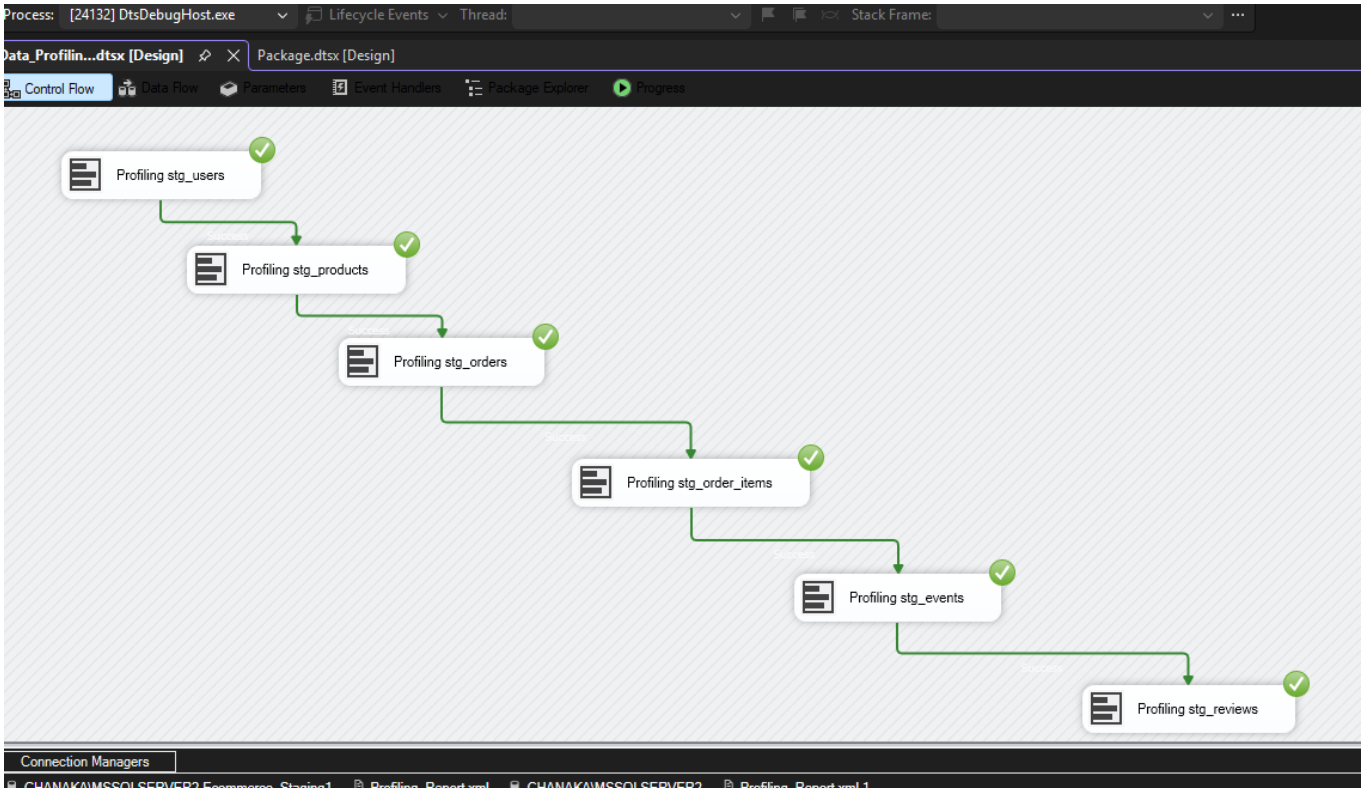
2. Destination Configuration for Profile Output



3.Configuring Data Profile Requests



4. Execution and Success Verification



It was repeated to all 6 tables.

4)Transform and Load Data

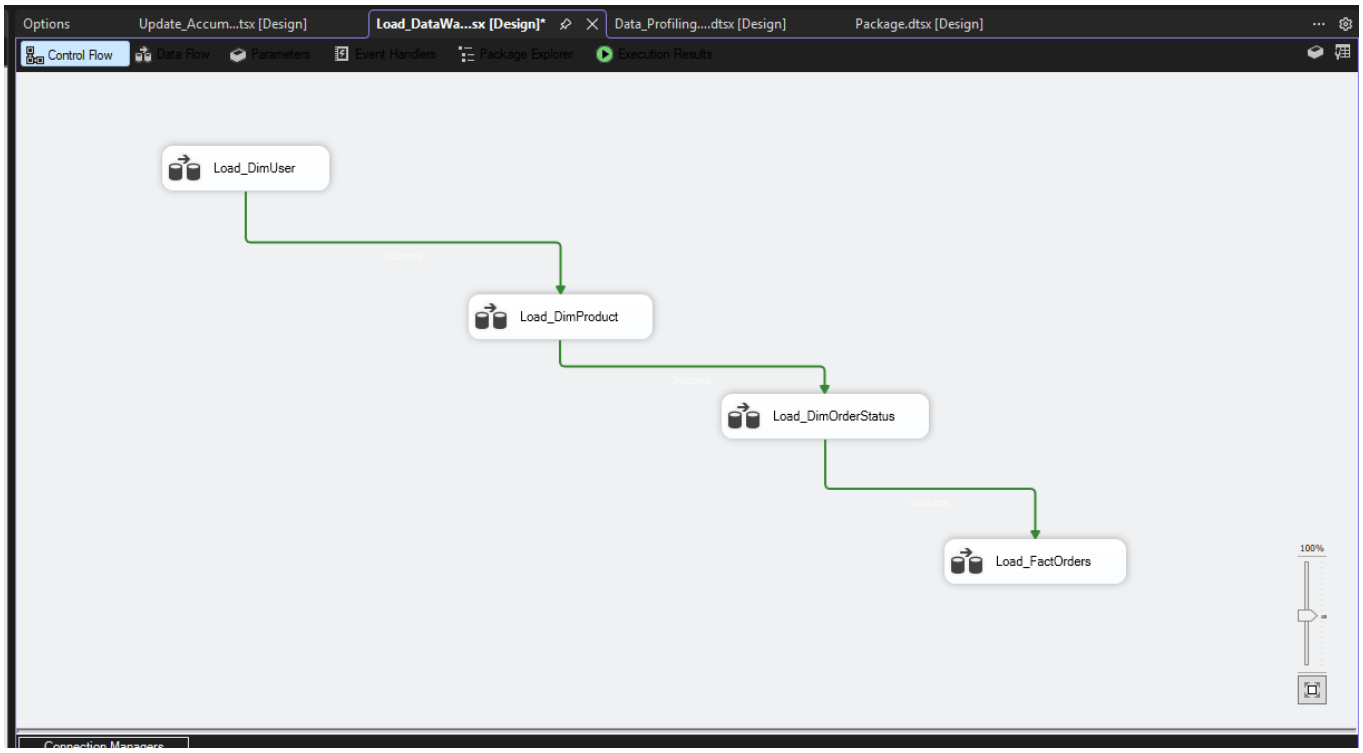
After extracting and profiling the data in the staging area (**Ecommerce_Staging**), various transformations were applied to clean and standardize the data. Data conversion tasks were performed to fix datatype mismatches, such as converting price fields to numeric and handling null values in product descriptions.

Once the data was properly transformed, it was loaded into the final data warehouse (**Ecommerce_DW**). During the loading process, surrogate keys were generated for dimension tables like **DimUser** and **DimProduct**. Fact tables like **FactSales** were populated by correctly establishing relationships with dimension tables using lookup transformations. This ensured the data was clean, consistent, and optimized for analytical reporting.

4.1 Transform and Loading DimProduct

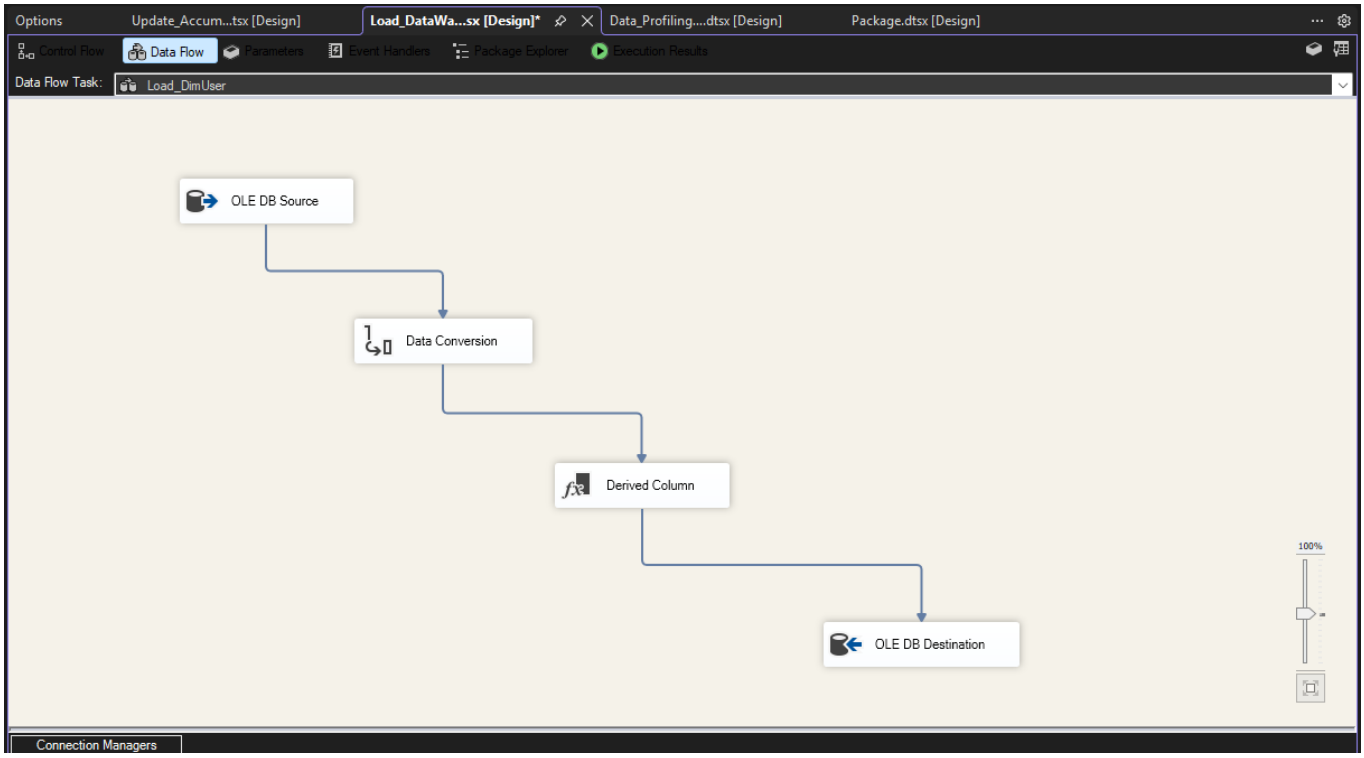
O We will use the DimUser dimension to illustrate the standard ETL process followed in this project. This involves source extraction, data type standardization, metadata enrichment, and final loading.

Step 01: Creating the Control Flow



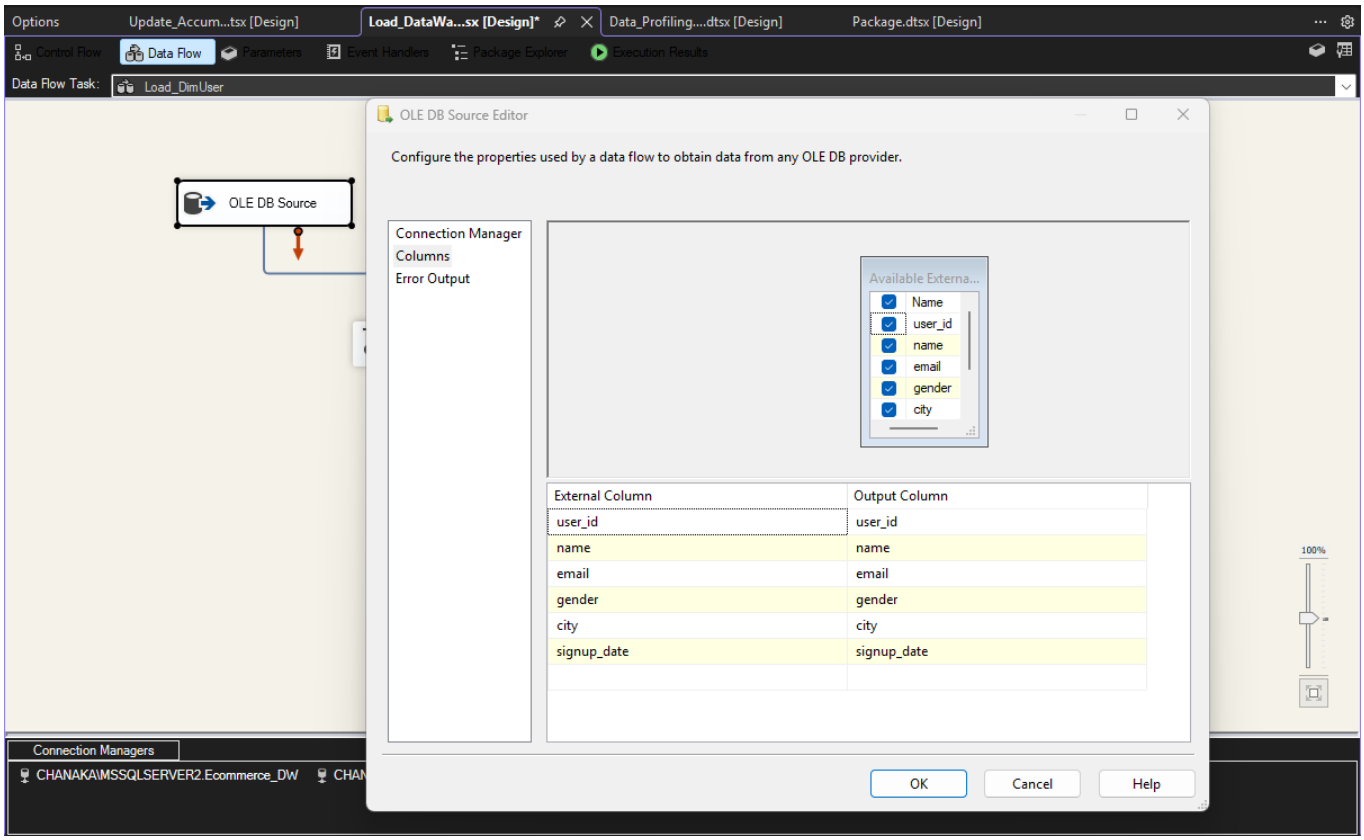
The ETL process starts at the Control Flow level. A **Data Flow Task** named Load_DimUser was created to encapsulate all transformation logic for the user dimension. This organizes the package and allows for sequential execution of different dimension loads.

Step 02: Designing the Data Flow Pipeline



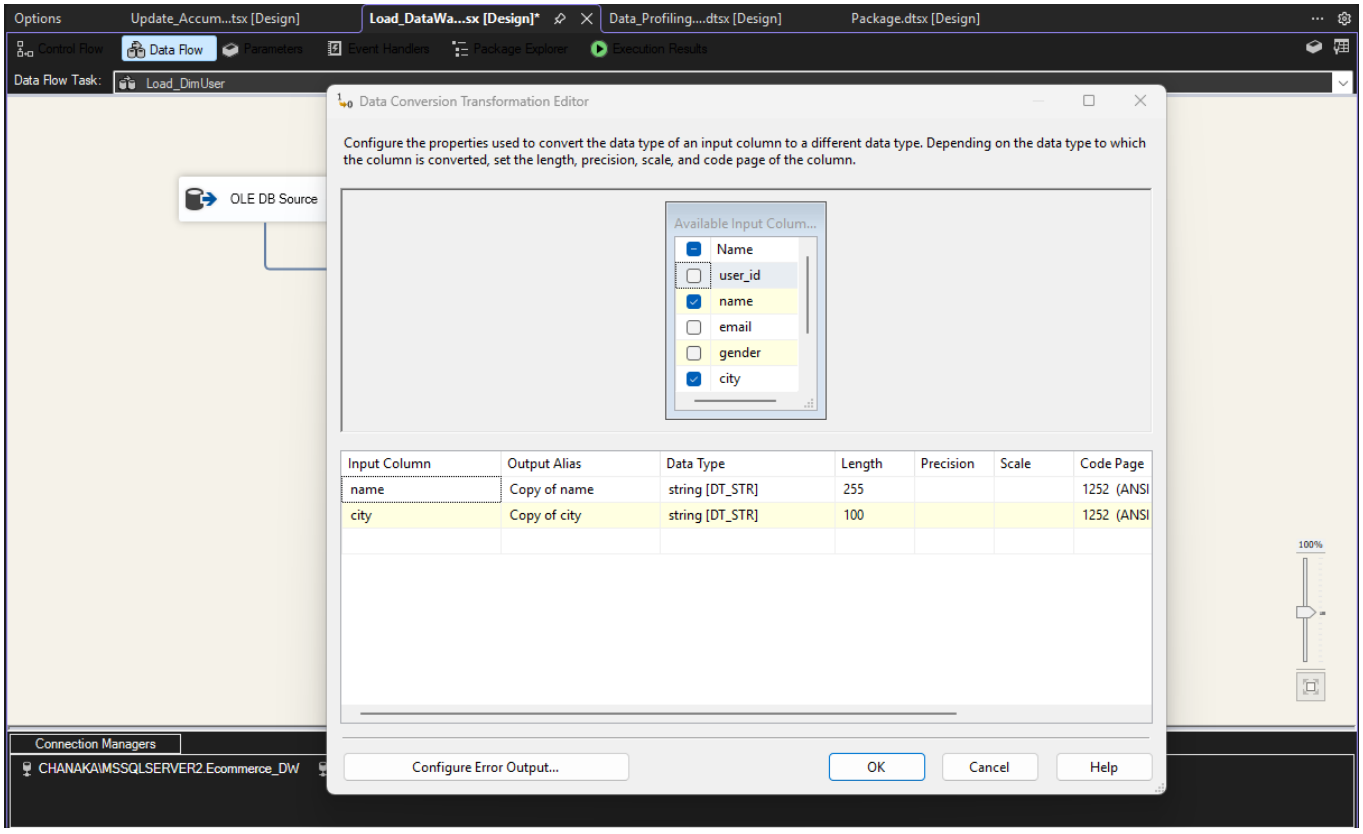
Inside the Data Flow Task, a pipeline was designed consisting of an **OLE DB Source**, **Data Conversion**, **Derived Column**, and **OLE DB Destination**. This pipeline ensures that data flows smoothly from the staging environment to the final warehouse while undergoing necessary changes.

Step 03: Source Extraction from Staging



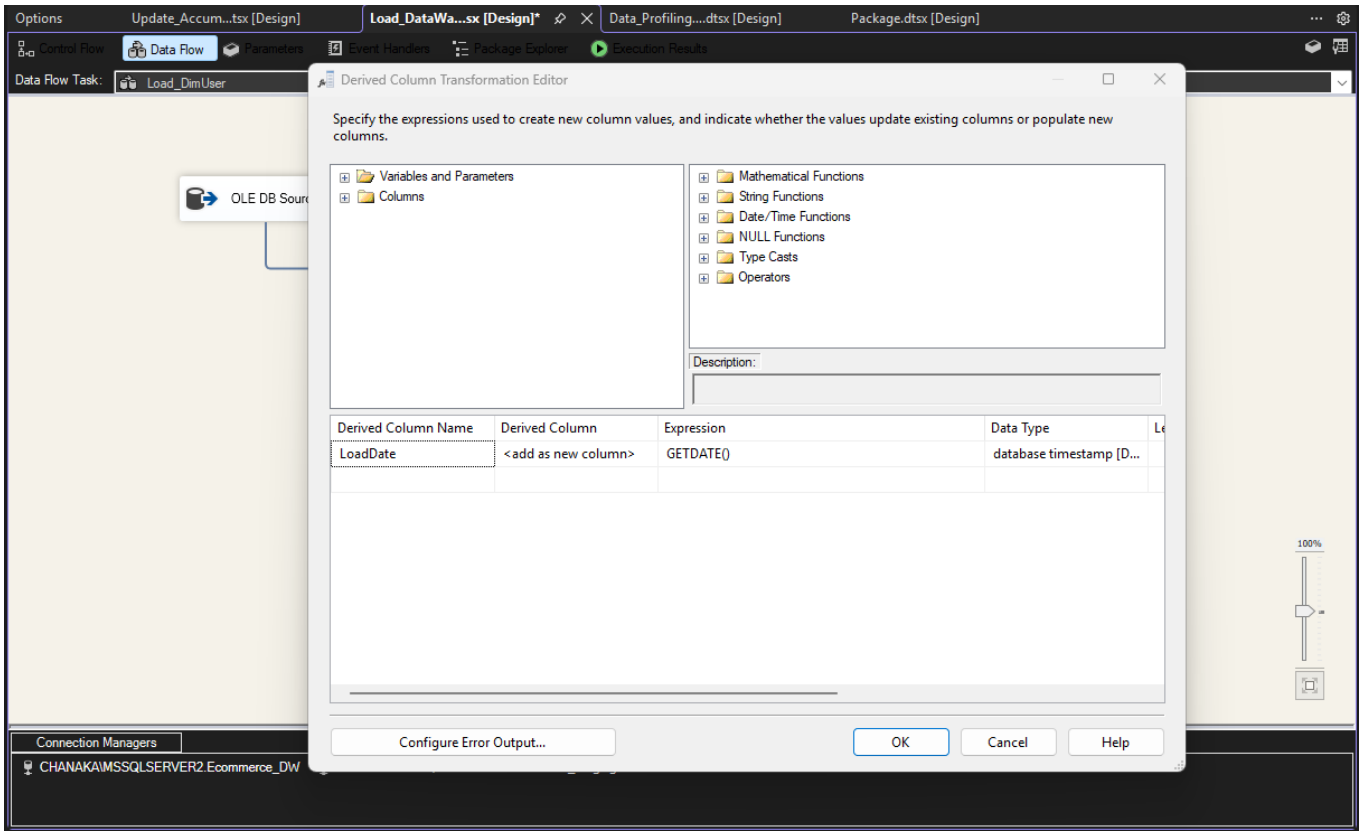
The **OLE DB Source** is configured to connect to the Ecommerce_Staging database. The [dbo].[STG_users] table is selected as the source. All relevant columns, such as user_id, name, email, and city, are extracted for further processing.

Step 04: Data Type Conversion



Since data types in the staging area might differ from the Data Warehouse schema, a **Data Conversion** transformation was applied. For instance, name and city columns were converted to string [DT_STR] with specific lengths (255 and 100, respectively) to ensure they match the destination table's requirements and prevent truncation errors.

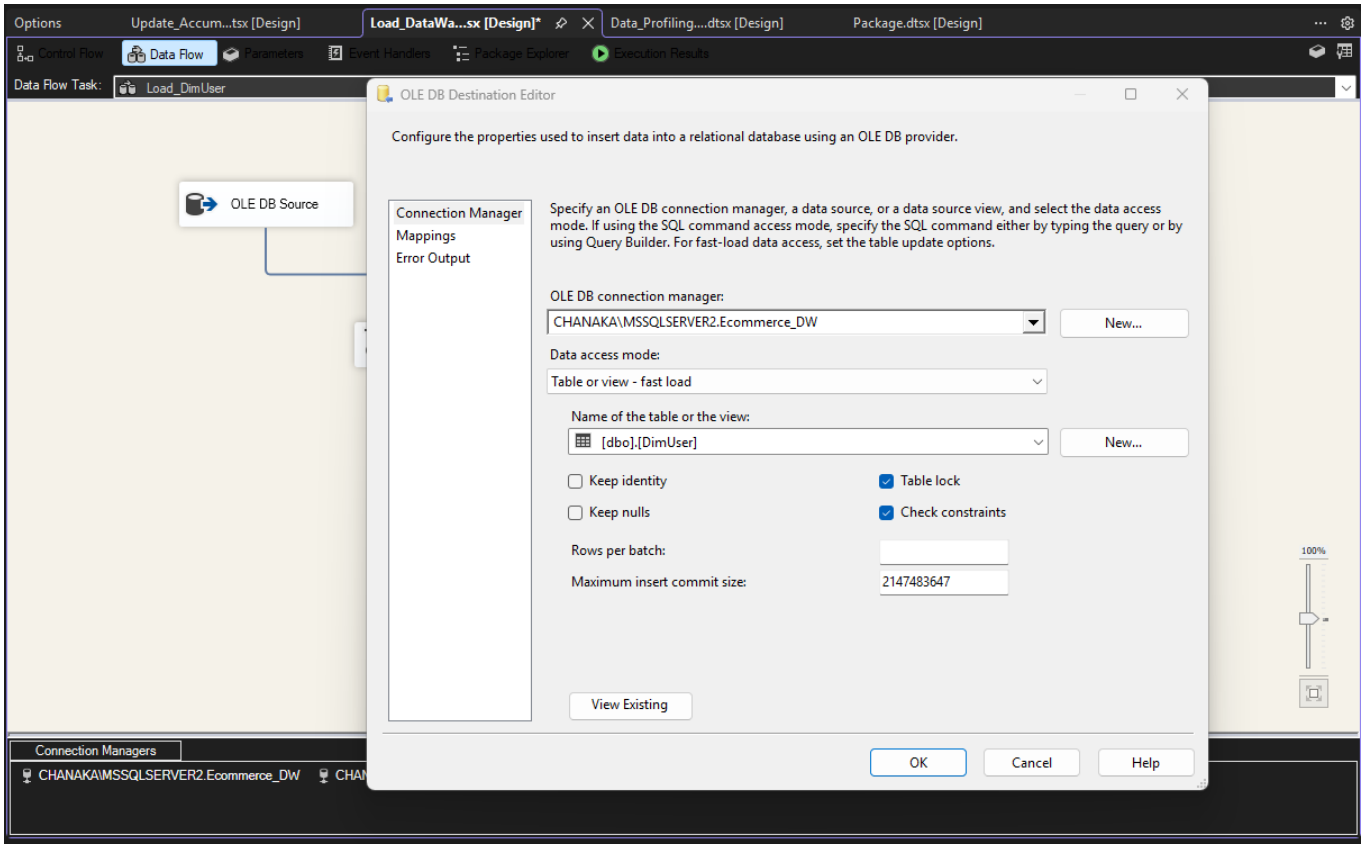
Step 05: Adding Metadata with Derived Columns



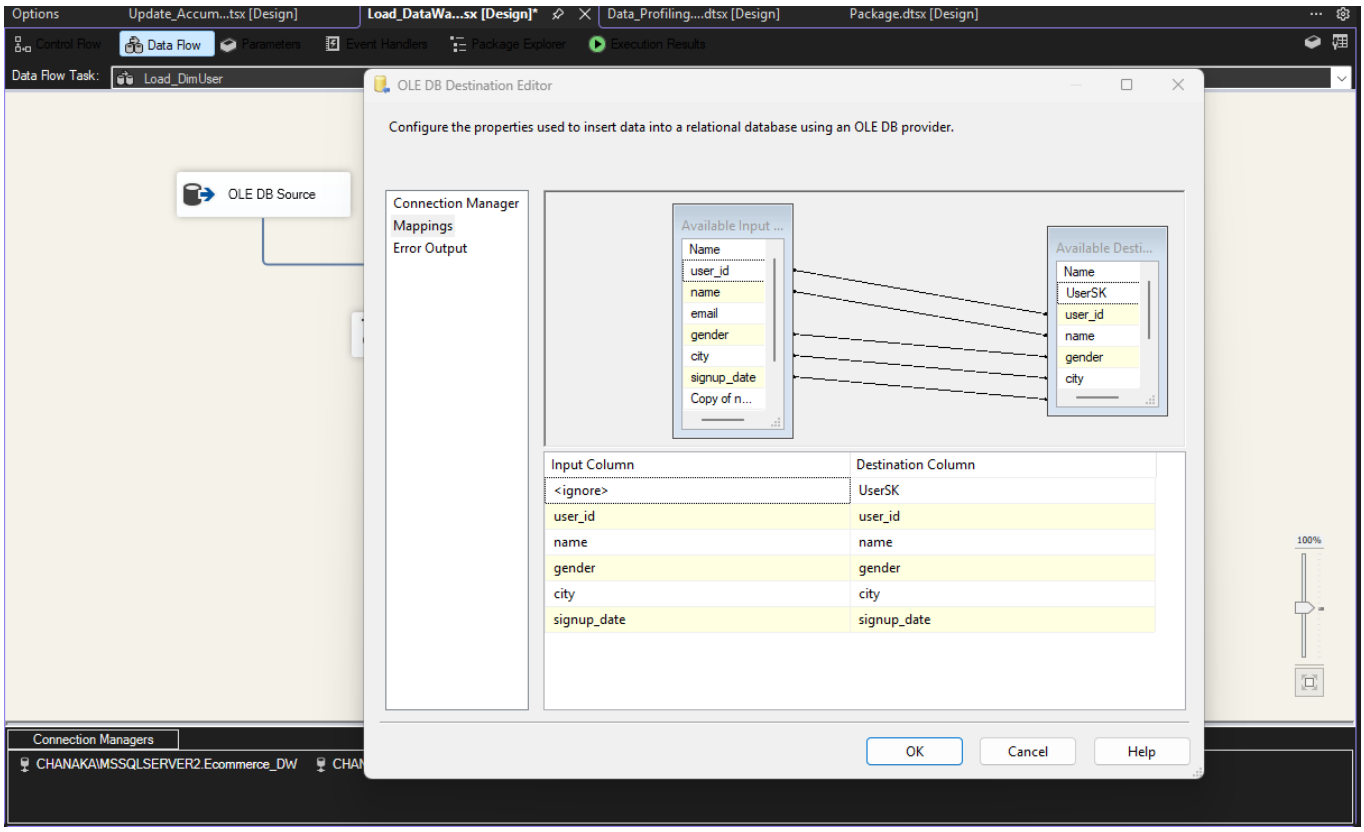
A **Derived Column** transformation was utilized to add an audit timestamp. Using the GETDATE() expression, a new column named LoadDate was created. This helps in tracking when each record was loaded into the data warehouse, which is a best practice in ETL design.

Step 06: Data Type Conversion

The OLE DB Destination is pointed toward the [dbo].[DimUser] table in the Ecommerce_DW database. The data access mode is set to "Table or view - fast load" to optimize performance during bulk inserts.

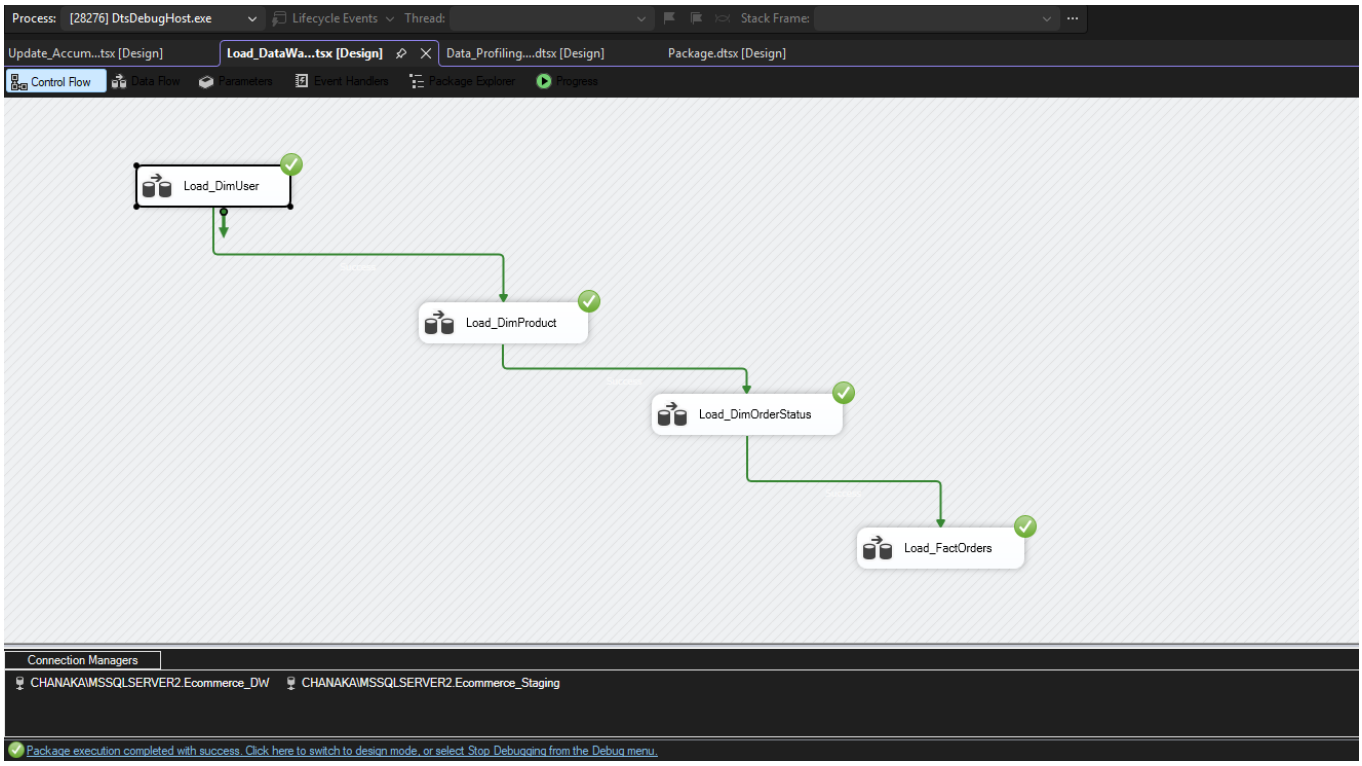


Step 07: Mapping Columns and Surrogate Keys



In the Mappings tab, the input columns from the transformation pipeline are mapped to the destination columns. The surrogate key UserSK is set to <ignore> because it is an identity column that is automatically generated by the SQL Server database upon insertion.

Step 08: Successful Execution

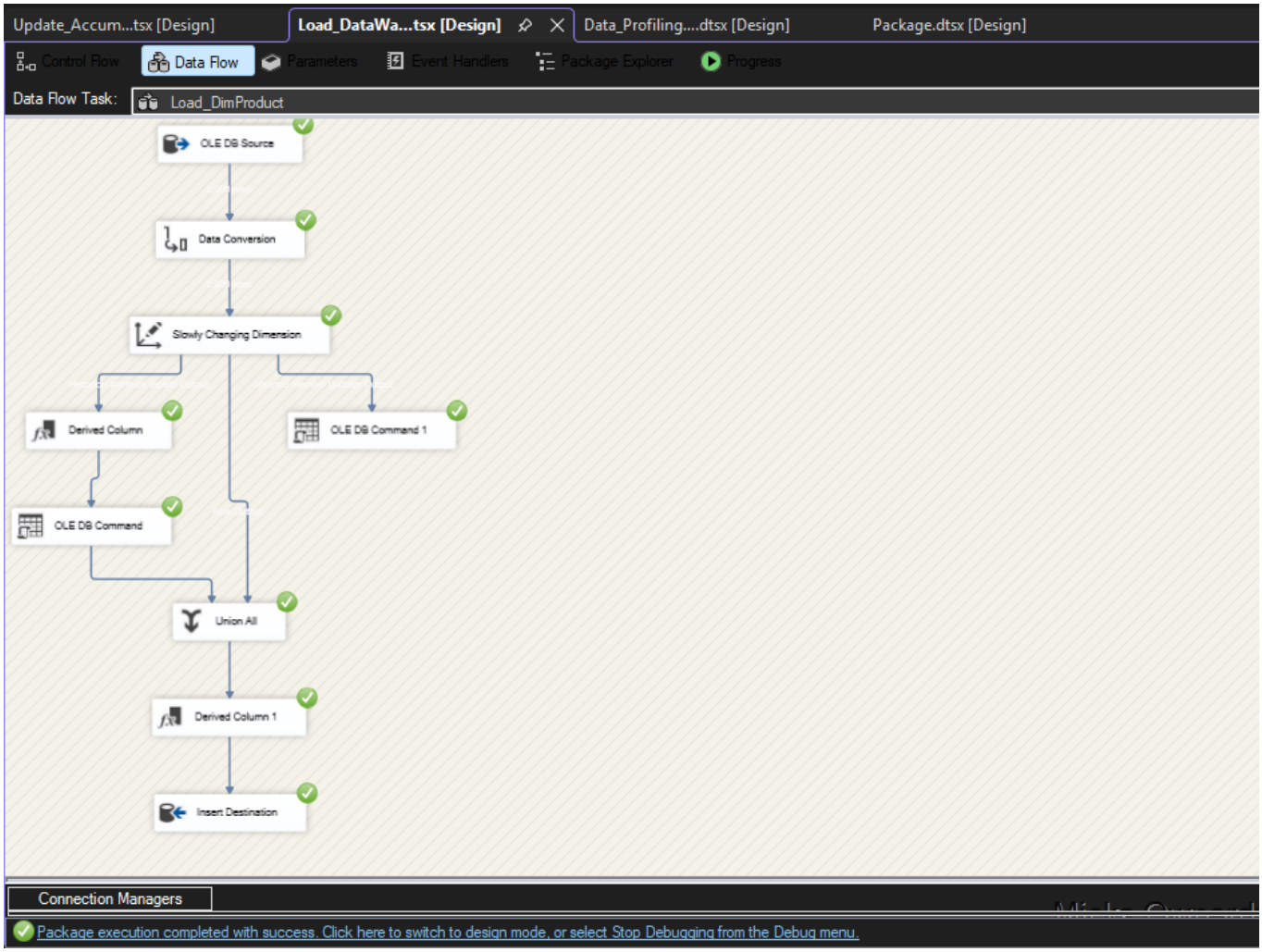


After configuration, the package was executed. The green checkmarks indicate that the Load_DimUser task (and subsequent tasks) completed successfully, confirming that the data was transformed and loaded into the warehouse without errors.

4.2 Transform and Loading DimProduct

This dimension acts as a lookup table for order states, requiring only unique values from the transactional data.

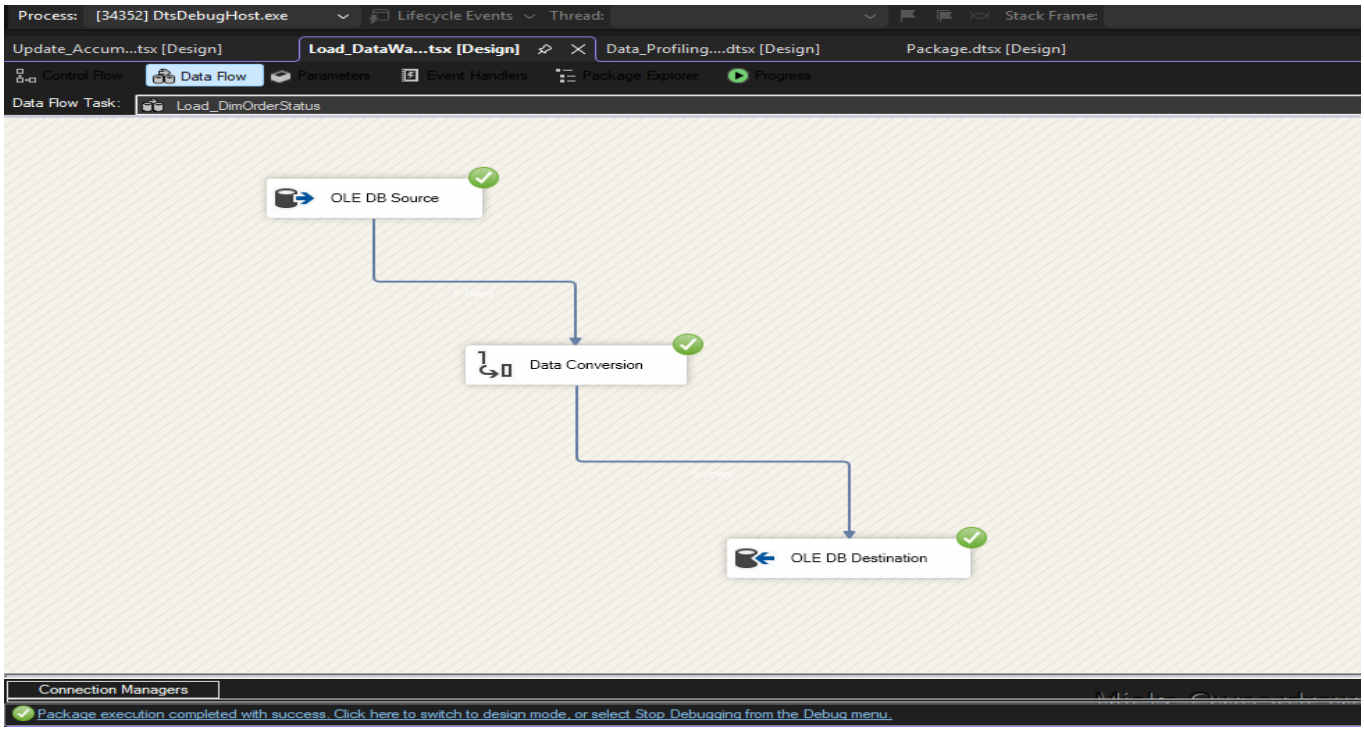
Transformation Logic: An **OLE DB Source** with a SELECT DISTINCT SQL command was used to extract unique status values from the stg_orders table. This ensures the dimension table is not cluttered with duplicate entries and provides a clean reference for the fact table.



4.3 Transform and Loading DimOrderStatus

This dimension acts as a lookup table for order states, requiring only unique values from the, transactional data.

Transformation Logic: An **OLE DB Source** with a SELECT DISTINCT SQL command was used to extract unique status values from the stg_orders table. This ensures the dimension table is not cluttered with duplicate entries and provides a clean reference for the fact table.



4.4 Transform and Loading FactOrders

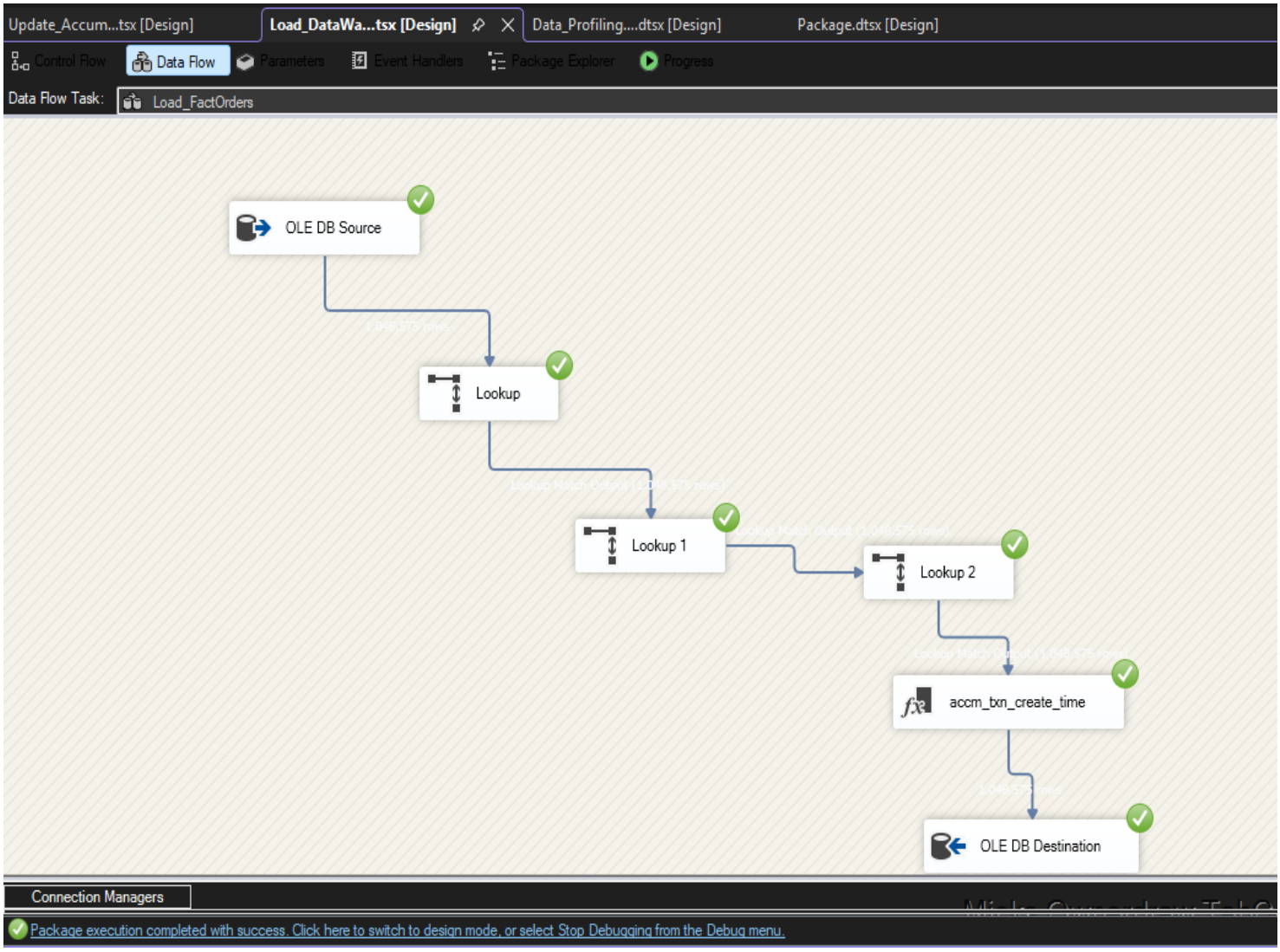
The fact table is the heart of the star schema, storing quantitative data and links to all dimensions.

Transformation Logic:

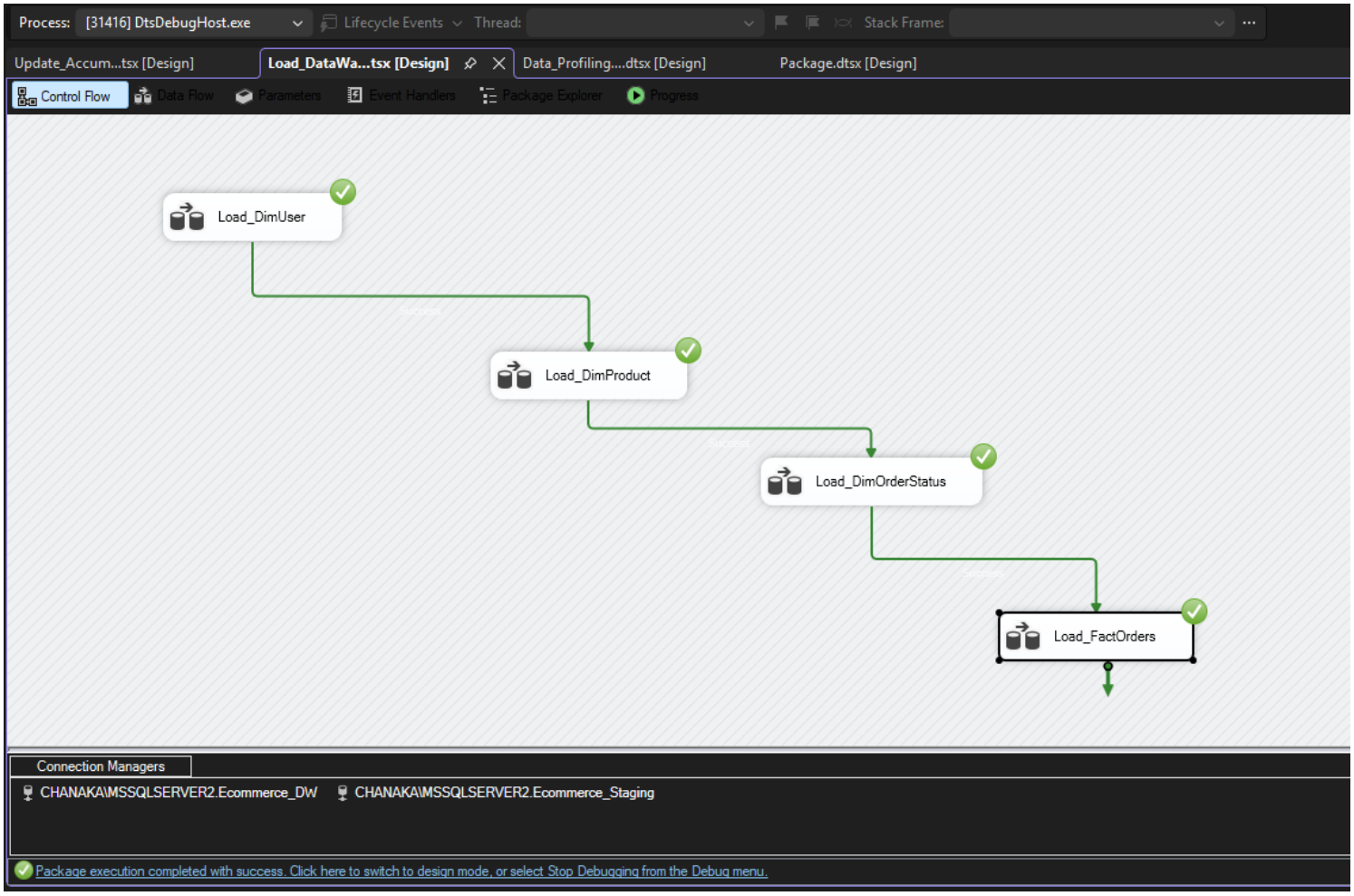
1. **Data Integration:** A complex SQL **JOIN** was used to combine order headers (stg_orders) with line items (stg_order_items).

2. **Surrogate Key Resolution: Lookup Transformations** were utilized to replace natural keys (like user_id and product_id) with their respective surrogate keys (UserSK, ProductSK) from the dimension tables.

3. **Loading:** The final enriched data was loaded using the **Fast Load** option to handle large volumes of transactional records efficiently.



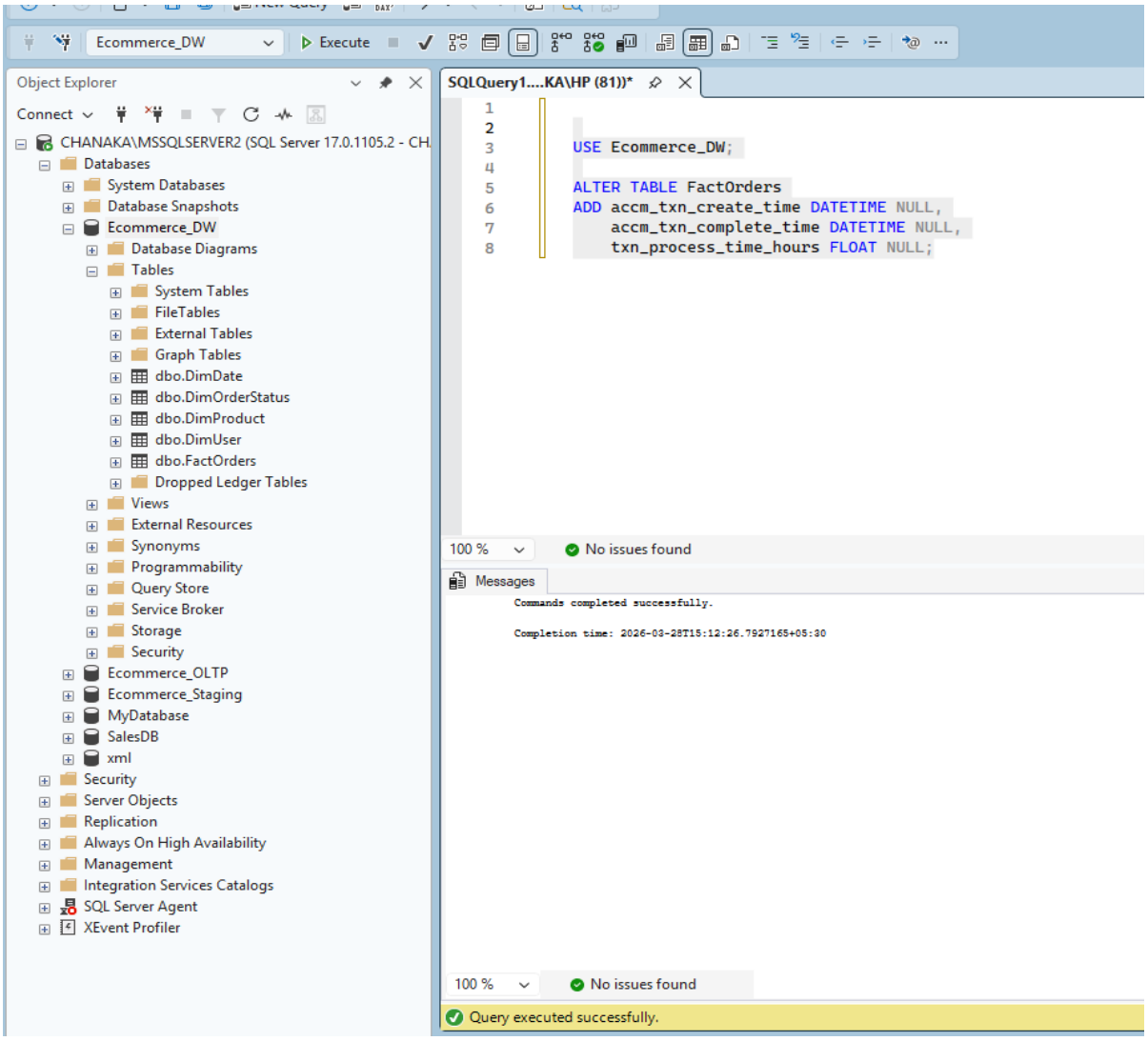
4.5 Final Package Execution and Success Verification



Step 6: : ETL development- Accumulating fact tables

Step 01 - Modifying the Fact Table Schema

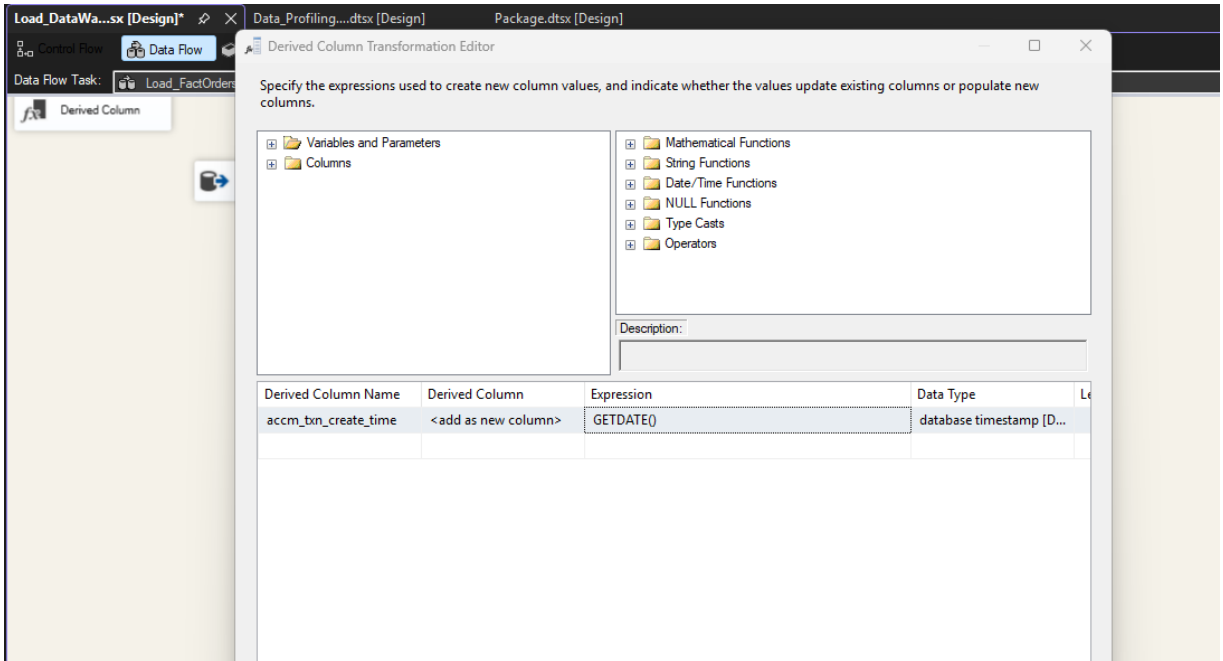
Schema Enhancement for Accumulating Measures

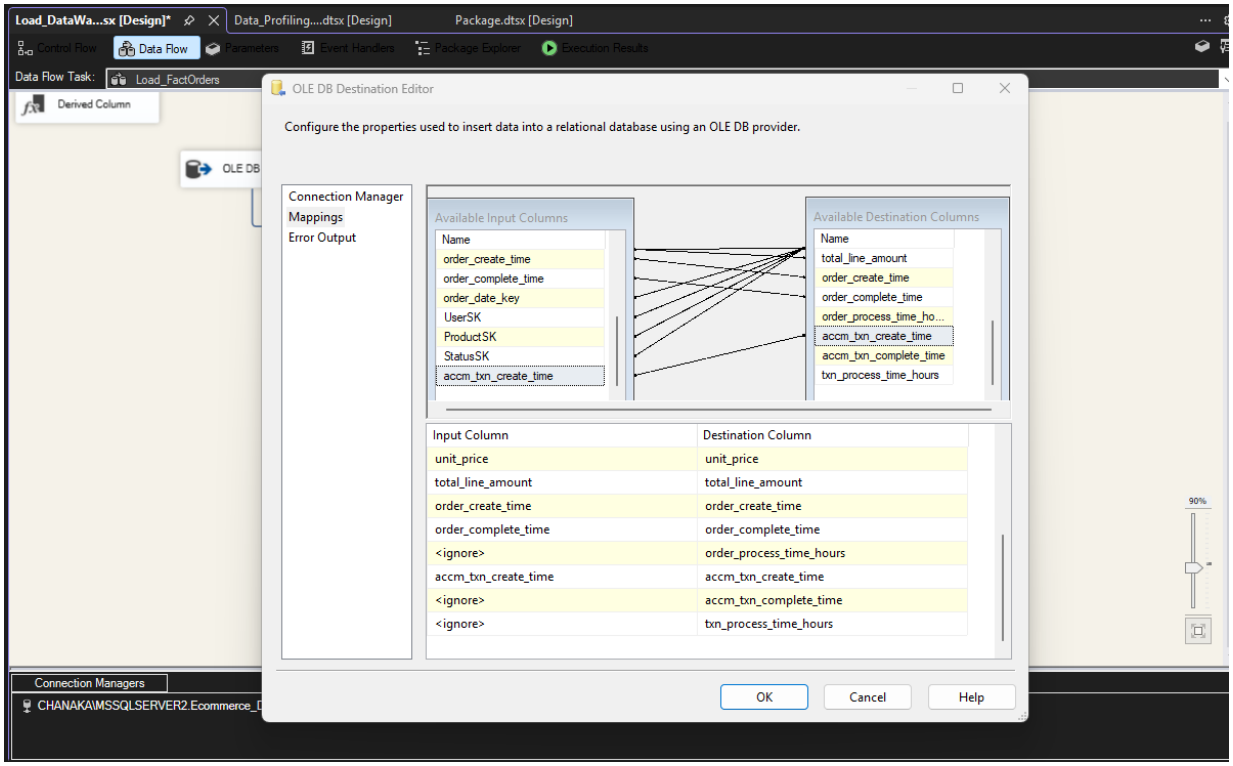


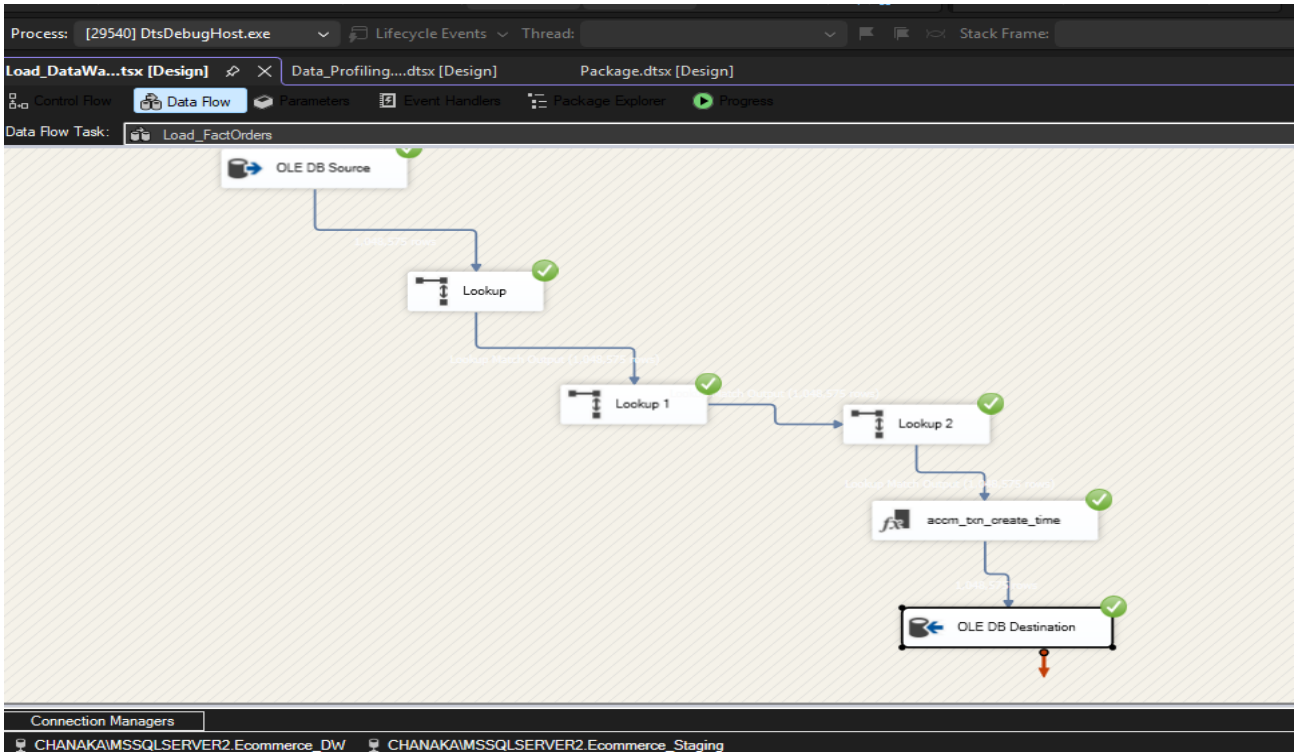
The existing FactOrders table was altered to include three new columns: accm_txn_create_time to store the initial entry timestamp, accm_txn_complete_time for the completion timestamp, and txn_process_time_hours to hold the calculated duration. These columns allow the data warehouse to track the lifecycle efficiency of each order, specifically measuring the time taken from order placement to fulfillment.

Step 02 - Initial Timestamp Capture (SSIS Changes)

Capturing Creation Timestamps in Initial ETL



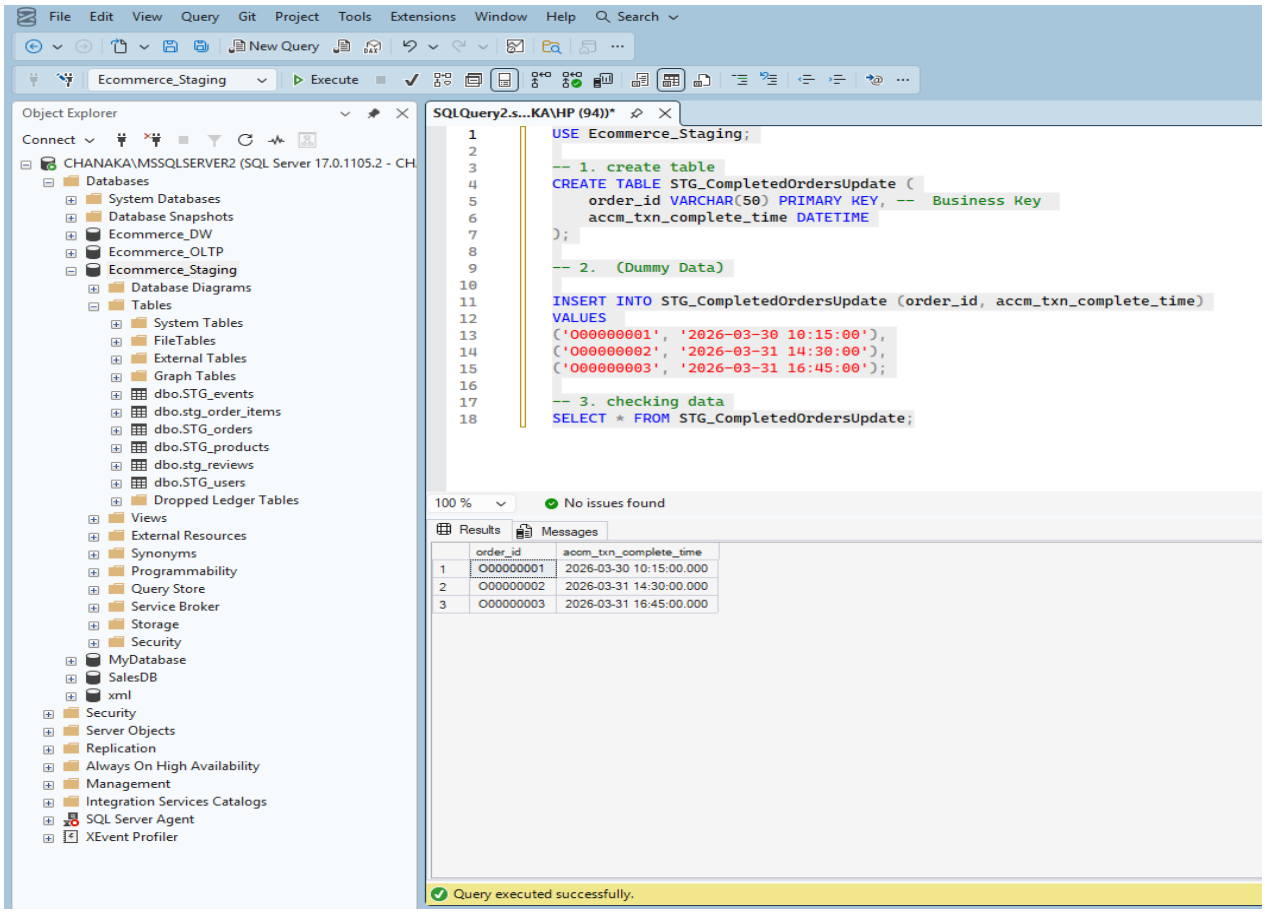




Within the main Load_FactOrders SSIS package, a **Derived Column** transformation was added to the data flow pipeline. The GETDATE() function was utilized to populate the accm_txn_create_time field during the initial data ingestion phase. This ensures that every new record entered into the fact table is timestamped with its exact entry time, providing a baseline for calculating processing durations in the accumulating fact table model.

Step 03 - Creating the Update Staging Table

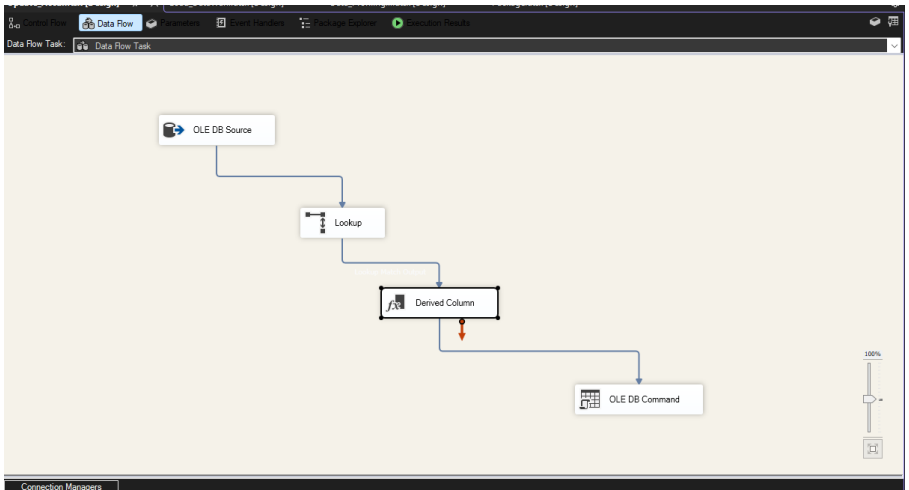
Staging Environment for Transaction Updates

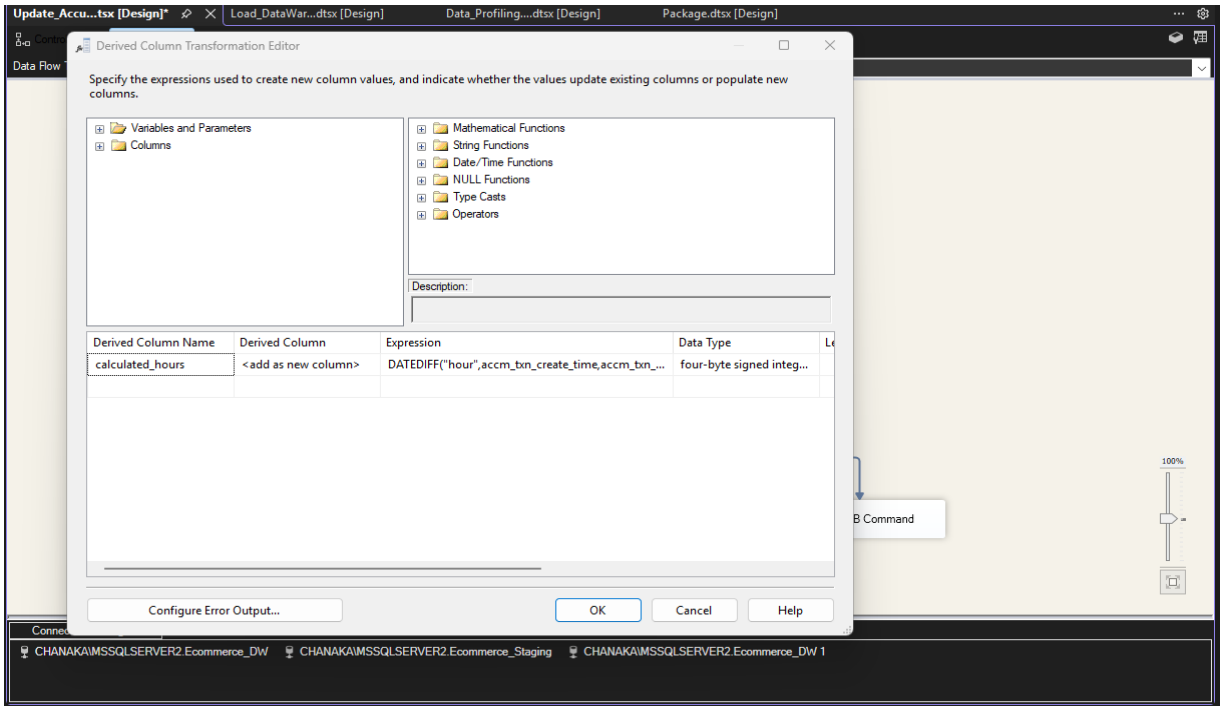


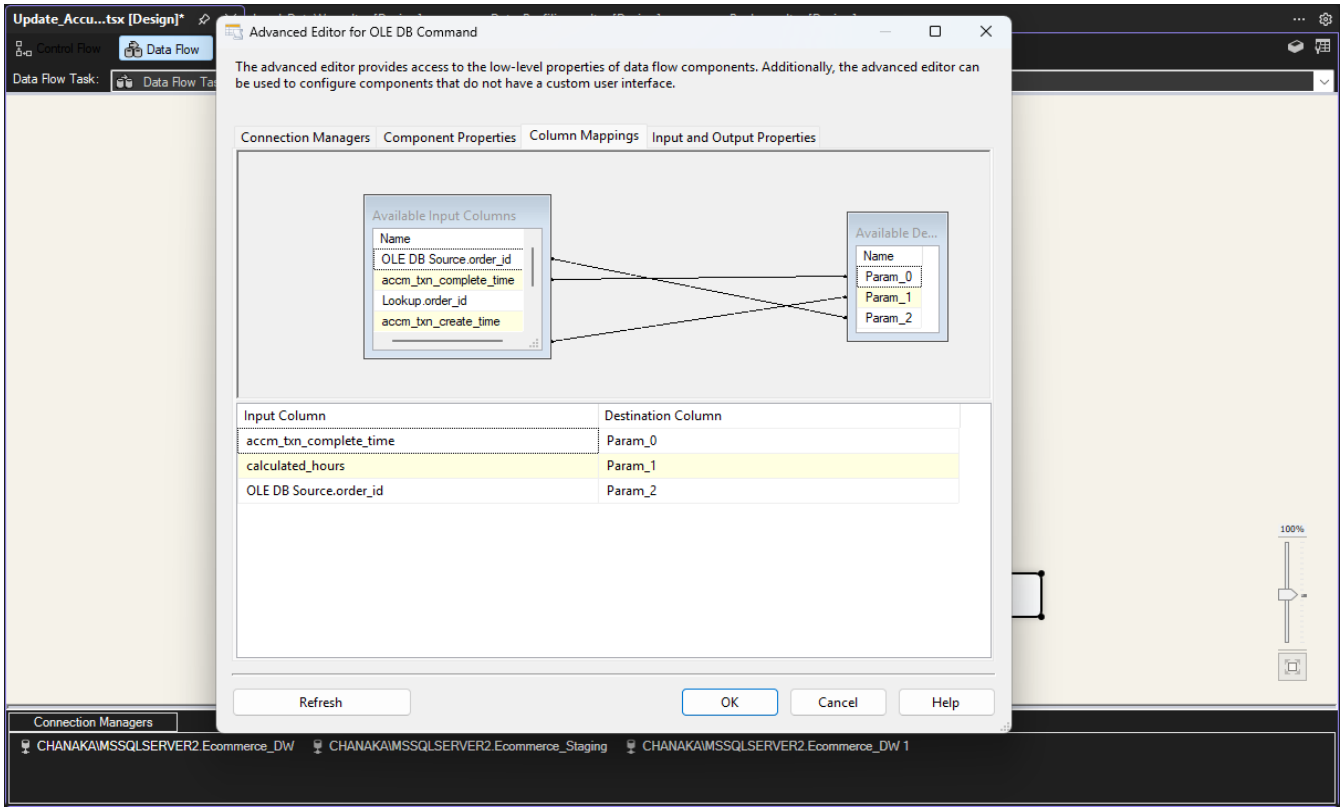
A separate staging table named STG_CompletedOrdersUpdate was created in the staging database. This table acts as a landing zone for late-arriving data regarding order completions. Since order fulfillment is an asynchronous process in e-commerce, this table temporarily stores the order_id (Business Key) and the corresponding accm_txn_complete_time. These records will be used in the subsequent ETL process to update the existing entries in the fact table and calculate processing durations.

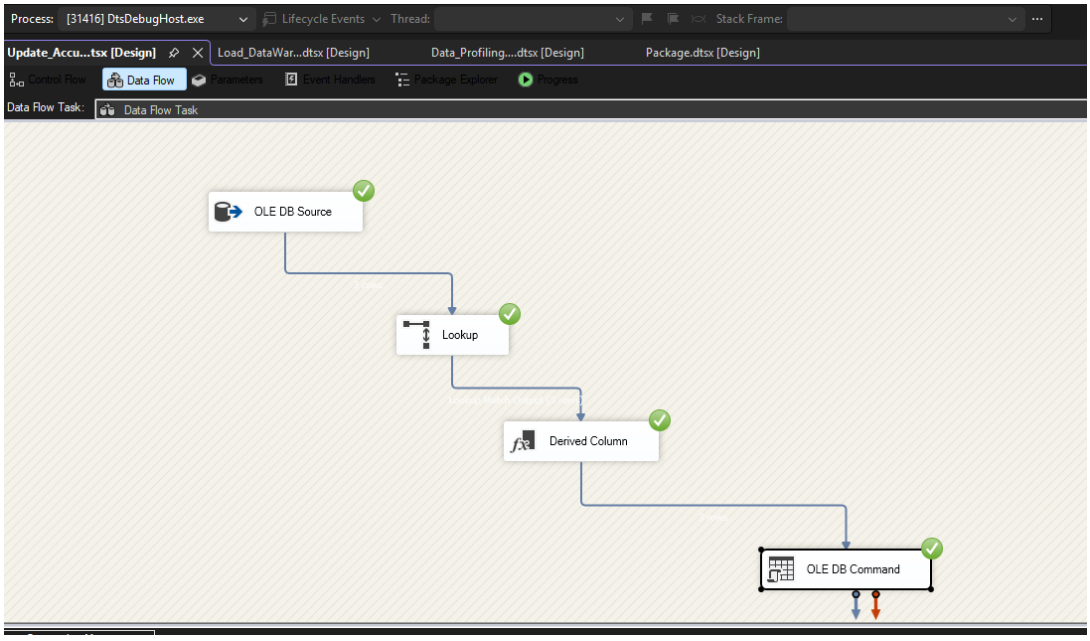
Step 04 - Designing the Update Pipeline (OLE DB Command)

Developing the Accumulating Update Pipeline




Update_Accu...tsx [Design]*
Load_DataWar...dtsx [Design]
Data_Profiling....dtsx [Design]
Package.dtsx [Design]
Data Flow
Derived Column Transformation Editor
Specify the expressions used to create new column values, and indicate whether the values update existing columns or populate new columns.
Variables and Parameters
Columns
Mathematical Functions
String Functions
Date/Time Functions
NULL Functions
Type Casts
Operators
Description:
Derived Column Name
Derived Column
Expression
Data Type
calculated_hours
<add as new column>
DATEDIFF("hour",accm_txn_create_time,accm_txn_...
four-byte signed integ...
Configure Error Output...
OK
Cancel
Help
B Command
100%
CHANAKA\MSSQLSERVER2.Ecommerce_DW
CHANAKA\MSSQLSERVER2.Ecommerce_Staging
CHANAKA\MSSQLSERVER2.Ecommerce_DW 1

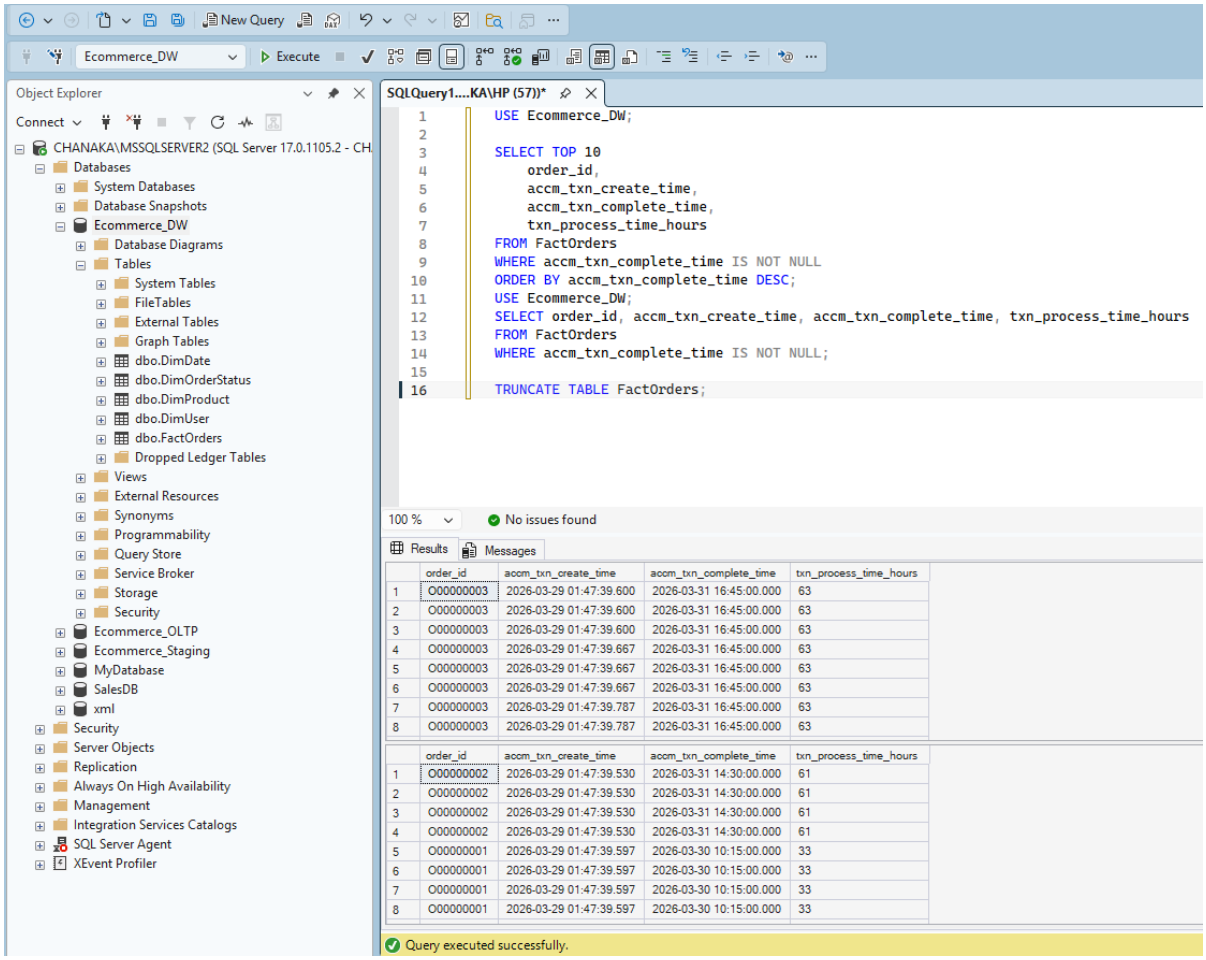

Update_Accu...tsx [Design]*
Control Flow
Data Flow
Data Flow Task:
Advanced Editor for OLE DB Command
The advanced editor provides access to the low-level properties of data flow components. Additionally, the advanced editor can be used to configure components that do not have a custom user interface.
Connection Managers
Component Properties
Column Mappings
Input and Output Properties
Available Input Columns
Name
OLE DB Source.order_id
accm_txn_complete_time
Lookup.order_id
accm_txn_create_time
Available De...
Name
Param_0
Param_1
Param_2
Input Column
Destination Column
accm_txn_complete_time
Param_0
calculated_hours
Param_1
OLE DB Source.order_id
Param_2
Refresh
OK
Cancel
Help
100%
Connection Managers
CHANAKA\MSSQLSERVER2.Ecommerce_DW
CHANAKA\MSSQLSERVER2.Ecommerce_Staging
CHANAKA\MSSQLSERVER2.Ecommerce_DW 1


Process: [31416] DtsDebugHost.exe
Lifecycle Events
Thread:
Stack Frame:
Update_Accu...tsx [Design]
Load_DataWar...dtsx [Design]
Data_Profiling....dtsx [Design]
Package.dtsx [Design]
Control Flow
Data Flow
Parameters
Event Handlers
Package Explorer
Progress
Data Flow Task:
Data Flow Task
OLE DB Source
Lookup
Derived Column
OLE DB Command

A specialized ETL pipeline was constructed to manage the accumulating nature of the fact table. The process extracts order completion data from the staging area and identifies the corresponding existing records in the FactOrders table via a **Lookup Transformation**. To ensure high performance and avoid common OLE DB metadata conversion errors (DT_DBTIMESTAMP vs DT_R8), the transaction duration was pre-calculated within the SSIS engine using a **Derived Column** transformation with a DATEDIFF expression. Finally, an **OLE DB Command** was implemented to execute a parameterized SQL UPDATE statement, updating the fact table with the completion timestamp and the calculated processing hours. The **DelayValidation** property was also enabled to ensure smooth execution by bypassing rigid design-time type checks.

Step 05 - Final Execution and Verification

Final Verification of Accumulating Fact Calculations



The final phase involved executing the end-to-end ETL process and verifying the data integrity within the FactOrders table. After running the update package, a verification query was executed in SQL Server Management Studio (SSMS). The results confirmed that the accm_txn_complete_time was successfully populated from the staging source, and the txn_process_time_hours was accurately calculated based on the interval between creation and completion. This successful synchronization validates the accumulating fact model, providing a complete audit trail of the order processing lifecycle.